

P3166R0: Static Exception Specifications

Table of Contents

- [1. Abstract](#)
- [2. Synopsis](#)
- [3. Overview](#)
- [4. Motivation](#)
 - [4.1. Alternative error-signalling mechanisms have problems too](#)
 - [4.1.1. Composition of facilities that use different error-signaling mechanisms](#)
 - [4.1.2. Runtime overhead due to additional branching, packing/unpacking](#)
 - [4.1.3. May require an additional "invalid" state, additional preconditions](#)
 - [4.2. Making exceptions fast by-design](#)
 - [4.3. Supporting exceptions in freestanding and real-time environments](#)
 - [4.4. Reducing bugs caused by failure to handle error-conditions](#)
 - [4.5. Describing the contract in code instead of in documentation](#)
 - [4.5.1. Uses in generic code](#)
 - [4.6. Reducing boiler-plate when writing functions that are transparent to exceptions](#)
- [5. Proposal](#)
 - [5.1. Overview](#)
 - [5.2. \(Re\)Adding throw specifiers](#)
 - [5.3. Static and dynamic exception specifications](#)
 - [5.3.1. Types in a throw-specification form an unordered set](#)
 - [5.3.2. Handling of `std::any\_exception` in the throw-specifier](#)
 - [5.3.3. The types in the throw specification describe all concrete types that may be thrown](#)
 - [5.3.4. Exception types may not be references, cv-qualified, or void](#)
 - [5.3.5. Static exception specifications are part of the function type](#)
 - [5.3.6. Deducing the throw-specifier from a function signature](#)
 - [5.3.7. `throw\(auto\)` - Deducing exception-specifications from the body of a function](#)
 - [5.3.8. Forward declarations of `throw\(auto\)` functions](#)
 - [5.3.9. Deduced exception-specifications and recursive functions](#)
 - [5.3.10. Delayed computation of deduced throw specifications](#)
 - [5.3.11. Do we also need `noexcept\(auto\)`?](#)
 - [5.4. Querying the throw-specification](#)
 - [5.4.1. `declthrow` of a call to a `throw\(...\)` function](#)
 - [5.4.2. Mixed dynamic and static exception specifications](#)
 - [5.4.3. Order of the exception types](#)
 - [5.4.4. Exception specifications of defaulted special member functions](#)
 - [5.4.5. Introducing a pack outside of a template](#)
 - [5.4.6. Packs of `declthrow` packs](#)
 - [5.4.7. Availability of the `declthrow` keyword](#)
 - [5.4.8. Alternative Syntaxes Considered](#)
 - [5.4.9. Filtering the set of exceptions](#)
 - [5.5. Checking the throw-specification of a function](#)
 - [5.6. Computing the set of potentially-thrown exception types](#)
 - [5.6.1. Statement Reachability](#)
 - [5.6.2. *function-body*](#)
 - [5.6.3. *statement*](#)
 - [5.6.4. *expression-statement*](#)
 - [5.6.5. *compound-statement*](#)
 - [5.6.6. *selection-statement*](#)
 - [5.6.7. *iteration-statement*](#)

- [5.6.8. *jump-statement*](#)
 - [5.6.9. *declaration-statement*](#)
 - [5.6.10. *try-block*](#)
 - [5.6.11. *init-statement*](#)
 - [5.6.12. *expression*](#)
- [5.7. Template handlers](#)
 - [5.7.1. Template argument deduction](#)
 - [5.7.2. Matching dynamic exception objects](#)
 - [5.7.3. Ordering of instantiated handlers](#)
- [5.8. Virtual Functions](#)
 - [5.8.1. ABI of virtual functions with differing exception specifications](#)
- [5.9. Concepts](#)
- [5.10. Coroutines](#)
 - [5.10.1. Coroutine promise object handling for static exception objects](#)
 - [5.10.2. Computing the exception specification of a coroutine with a deduced throw specification](#)
- [5.11. Type traits](#)
 - [5.11.1. Nothrow queries](#)
 - [5.11.2. `std::is\_function` and `std::is\_member\_function\_pointer`](#)
 - [5.11.3. Additional traits](#)
- [5.12. Freestanding](#)
- [6. Prior Work](#)
 - [6.1. Throw specifications and `noexcept`](#)
 - [6.2. Java Checked Exceptions](#)
 - [6.2.1. Background On How Exceptions work in Java](#)
 - [6.2.2. Criticisms of Java Checked exceptions](#)
 - [6.3. Midori](#)
 - [6.4. Deducing exception specifications](#)
 - [6.5. Faster Exceptions](#)
- [7. Design Discussion](#)
 - [7.1. Code Evolution](#)
 - [7.2. A new calling convention / function-type](#)
 - [7.3. Changing the implementation changes the calling convention](#)
 - [7.4. How accurate does the potentially-thrown exception type calculation need to be?](#)
 - [7.5. Permission to throw types derived from `std::bad\_alloc` / `std::bad\_cast`](#)
 - [7.6. Avoiding dependencies on thread-locals](#)
 - [7.6.1. Avoiding overhead implied by `std::current\_exception\(\)` and `throw;`](#)
 - [7.6.2. Avoiding overhead implied by `std::unhandled\_exceptions\(\)`](#)
 - [7.7. Calling C functions](#)
 - [7.8. Interaction with Contracts](#)
 - [7.9. Standard Library Impacts](#)
 - [7.9.1. Whether implementations add `noexcept` to "Throws: Nothing" functions can now affect well-formedness of a program](#)
 - [7.9.2. Should implementations be allowed to strengthen "Throws: something" exception specification to a static exception specification?](#)
 - [7.9.3. Usage of the standard library in functions with static throw-specifications will be painful without more `noexcept`](#)
 - [7.9.4. Function-wrapper types](#)
 - [7.9.5. Algorithms](#)
- [8. Implementation Strategies](#)
 - [8.1. Multiple return-paths/return-addresses](#)
 - [8.1.1. Inline Jump Tables](#)
 - [8.1.2. "NOP Stuffing" Jump Tables](#)
 - [8.2. Multiple return-value-slots](#)
 - [8.3. Passing an exception-value-slot parameter](#)

- [8.4. Walking stack to find storage](#)
 - [8.5. Unwinding through scopes with destructors](#)
 - [8.6. Interop between static-exception functions and dynamic-exception functions](#)
 - [8.7. Virtual function calls](#)
 - [8.8. Interaction with `std::uncaught\_exceptions\(\)`](#)
 - [8.9. Static exceptions and the Itanium C++ ABI](#)
 - [9. Acknowledgements](#)
 - [10. References](#)
-

Document **P3166R0**

Date **2024-03-16**

Reply To **Lewis Baker** <lewissbaker@gmail.com>

Audience **EWGI, LEWGI**

1. Abstract

Exceptions are the C++ language's primary tool for communicating errors to callers. However, the dynamic exceptions we have today come with runtime overheads that cause many people to avoid using, or even completely disable exceptions in domains that are performance-sensitive.

This paper proposes adding new language features for annotating functions with additional type information, in the form of static exception specifications, that should allow exception objects to be returned on the stack with performance similar to that of normal return-paths.

These features include:

- Adding the ability to add throw-specifiers to function declarations as a way of declaring the complete set of exception types that the function may exit with.
- Allow declaring a function as `throw(auto)` to deduce the set of exception types that may be thrown based on the body of the function.
- Add a new `declthrow()` syntax for querying the set of exceptions that an expression may exit with.
- Add a `template catch` syntax to allow writing exception handlers that are templates which are instantiated based on the set of exception types that may be thrown from the associated try-block.
- Enable static checking of exception specifications for function definitions with a static throw-specifier to ensure that they can not exit with exceptions not listed in their throw specifier. Functions definitions with a `throw(auto)`, `throw(...)` or `noexcept` specifier are not checked, even if they call functions with static exception specifications.

This paper has the following main goals:

1. Eliminate the performance-related reasons for not using exceptions in embedded and other performance critical code.
2. Enable the use of exception-based error handling in freestanding environments by enabling exceptions to be thrown/caught without a need for heavy-weight runtime components or dynamic allocation.
3. Eliminate classes of bugs relating to unhandled exceptions and error code-paths by making it possible to identify at compile-time code that is failing to handle some exception-types that might be thrown and make it ill-formed.

This proposal is still in the early stages and is being published to solicit feedback on direction and also on potential interaction with other in-flight papers, such as contracts and discussions around `noexcept` policy in the standard library.

2. Synopsis

Example 1: Explicit throw specifications on declarations

```
void throws_nothing() throw(); // empty static exception specification – equivalent to nothrow
void throws_a_b() throw(A, B); // static exception specification
void throws_any() throw(...); // dynamic exception specification – equivalent to noexcept
void throws_any2() throw(std::any_exception); // same as throw(...)
```

Example 2: Deduced throw specifications

```
template<std::ranges::range R, typename F>
requires std::invocable<F&, std::ranges::range_reference_t<R>>
void for_each(R rng, F func) throw(auto) {
    for (auto&& x : rng) {
        func(static_cast<decltype(x)>(x));
    }
}
```

Example 3: Querying the potentially-thrown exception types + use of template handlers

```
std::variant<std::monostate, decltype(throws_a_b())...> err; // variant<monostate, A, B>
try {
    throws_a_b();
} template catch (auto& e) { // instantiated for each static exception type in set { A, B }
    err.template emplace<decltype(auto(e))>(std::move(e));
}
```

Example 4: Checked exception specifications - Calling throw(A,B) function from throw() function

```
void nothrow_caller() throw() {
    throws_a_b(); // Ill-formed: A and B not handled locally and not listed in throw specification
}

try {
    throws_a_b(); // Ill-formed: A not handled locally and not listed in throw specification
} catch (B) {}

try {
    throws_a_b(); // OK: all potentially-thrown exceptions are handled locally
} catch (A) {}
catch (B) {}
}
```

Example 5: Checked exception specifications - Calling throw(...) function from throw() function

```
void nothrow_caller() throw() {
    throws_any(); // Ill-formed: dynamic exception not handled locally
}
```

```

try {
    throws_any(); // OK
}
catch(...) {}
}

```

Example 6: Checked exception specifications - Calling throw(A,B) function from throw(A) function

```

void caller() throw(A) {
    throws_a_b(); // Ill-formed: B not handled locally and not listed in throw specification

    try { throws_a_b(); /* OK */ }
    catch (B) {}

    try { throws_a_b(); /* OK */ }
    catch (A a) {
        if (!can_handle(a)) throw; // OK: rethrows A, which is allowed
    }
    catch (B) {}
}

```

Example 7: Checked exception specifications - Calling throw(...) function from throw(A) function

```

void caller() throw(A) {
    throws_any(); // Ill-formed: dynamic exception not handled locally

    try { throws_any(); /* OK - all exceptions handled */ }
    catch (const A&) {
        throw; // Ill-formed: Might rethrow type derived from A (unless A is final)
    }
    catch(...) {}

    try { throws_any(); /* OK - exceptions handled */ }
    catch(const A& a) {
        throw a; // OK - only throws A - slices types derived from A
    }
    catch(...) {}
}

```

Example 8: Simpler alternative to std::expected / std::variant code

An API that can fail but avoids using exceptions might look like:

```

std::expected<X, E1> get_x() noexcept;
std::expected<Y, E2> get_y() noexcept;

std::expected<Z, std::variant<E1, E2>> make_z() noexcept {
    auto x = get_x();
    if (!x) return {std::unexpected, std::move(x).error()};

    auto y = get_y();
    if (!y) return {std::unexpected, std::move(y).error()};

    return {std::in_place, std::move(x).value(), std::move(y).value()};
}

std::expected<Z, std::variant<E1, E2>> make_z_p2561() {
    return {std::in_place, get_x()??, get_y()??}; // ?? error-propagation operator from P2561
}

```

```

}

void consumer() noexcept {
    auto result = make_z();
    if (!result) {
        std::visit(overload(
            [&](const E1& e) noexcept { /* handle E1 */ },
            [&](const E2& e) noexcept { /* handle E2 */ },
            result.error()));
        return;
    }

    Z& z = result.value();
    // use z
}

```

which can instead be written using static exceptions, with equivalent (or better) performance:

```

X get_x() throw(E1);
Y get_y() throw(E2);

Z make_z() throw(E1, E2) {
    return Z{get_x(), get_y()}; // allows aggregate initialization
}

void consumer() throw() {
    try {
        Z z = make_z();
        // use z
    }
    catch (const E1& e) { /* handle E1 */ }
    catch (const E2& e) { /* handle E2 */ }
}

```

Example 9: Guaranteed deterministic local throw/catch, even in throw(...) functions

```

void example(std::vector<std::vector<std::string>> vec) {
    struct InvalidString {};
    try {
        for (auto& strings : vec) {
            for (auto& string : strings) {
                if (!is_valid(string))
                    throw InvalidString{}; // cost equivalent to local goto
                // process string...
            }
        }
    }
    catch (InvalidString) {
        // handle invalid input
    }
}

```

3. Overview

This paper proposes introducing *static exception specifications*, repurposing the throw-specification syntax which was removed in C++17, to provide additional type-information about the closed-set of possible exception types

that might be thrown from a function. The proposed design tries to avoid the shortcomings of the previous design of throw-specifications by requiring that checks are performed at compile-time instead of at runtime.

Function definitions with a throw specification are checked by the compiler at compile-time to ensure that there are no code-paths that might allow an exception to exit the function that would violate the declared throw-specification, rather than dynamically check this at runtime and terminate. Functions that fail this static exception specification check are ill-formed.

While we are yet to gain implementation experience for this proposal, the design for static exception specifications should permit implementation strategies that can achieve efficiency of throwing exceptions close to that of normal return values, with exception objects able to be returned on the stack or in registers using similar conventions to normal return-values. Some potential implementation strategies for this are described in the [Implementation Strategies](#) section.

The proposed design is also similar in many ways to the design proposed in [P0709R4 - Zero-overhead deterministic exceptions: Throwing values](#). However, it differs in a number of key areas:

- It allows users to specify a list of multiple exception types they might throw in the throw specifier rather than limiting them to throwing only `std::error` or, in the extension proposed at the end of P0709, a single error type.
- It does not require the introduction of a special `std::error` type that must be used to wrap all exceptions-as-values. Such a type could potentially be used in conjunction with this proposal via `throw(std::error)`, but would require explicit conversion between exception types at function boundaries.
- It does not require falling back to dynamic-allocation when propagating exception types that do not fit in the small-object optimisation built into P0709's proposed `std::error` type. All static exception objects are passed in automatic storage duration storage so can be as large or as small as you like. Throwing an empty exception object does not need to consume a register, throwing a large object does not require dynamic allocation.
- It provides the `template catch` facility to allow generically handling multiple static exception types - something P0709 does not provide as it expects you to just be propagating a single static exception type, `std::error`.

This design does not change the semantics of existing C++23 code. It can be incrementally adopted throughout a code-base by annotating functions with static exception specifications using the `throw(/type-list/)` syntax where appropriate in much the same way that code-bases could incrementally adopt `noexcept` specifiers in their code-base when they were introduced. Changing a function from `noexcept(false)` to `throw(A, B, C)` will likely be an ABI break, however, and so will require recompilation of any calling code.

This proposal aims to enable use of static exception specifications in freestanding environments which were traditionally unable to use exceptions. Static exception-specifications provide enough type-information to the compiler to allow exceptions to be thrown, propagated, caught and rethrown with minimal runtime machinery - without need for dynamic-allocation, runtime type information, or `dynamic_cast`.

If a program can avoid using facilities that require dynamic exceptions, such as `std::current_exception()` and `throw;` expressions that appear outside of the lexical scope of handlers, then the only dependency on thread-local storage is `std::uncaught_exceptions()`, which is not required in a lot of projects and could potentially be omitted from freestanding environments. See the section on [Avoiding dependencies on thread-locals](#) for more details.

The ability to opt-in to compile-time checking that all exceptions are handled helps to ensure that all error-conditions have been handled, potentially helping to improve the correctness, reliability and safety of code. Callers of a function cannot forget to handle errors like they can with other mechanisms for communicating errors.

It also allows a program to ensure that there are no hidden calls to `std::terminate()` inserted by the compiler due to unhandled exceptions, which can potentially remove some barriers to use of C++ exceptions in environments where termination should only be performed in the presence of unrecoverable errors.

Authors of functions that have checked exception specifications can still explicitly catch exceptions that represent fatal error conditions and insert an explicit call to `std::terminate()` to get the same behaviour as the dynamically-checked `noexcept` behaviour - only now these calls to `std::terminate()` appear in source code and so can be more easily audited than the implicit ones the compiler was inserting.

Adopting this paper would have some implications for the P2900 contracts proposal and general `noexcept` policy in the standard library. It forces us to confront the question of whether a function with a deduced exception specification that evaluates a `contract_assert` should be considered potentially-throwing. This paper discusses these question briefly and poses some potential directions, but does not seek to answer them at this stage.

This paper is structured into the following sections:

- [Motivation](#) - motivation for addition of this feature
- [Proposal](#) - describes the design of the features this paper is proposing
- [Prior Work](#) - a comparison of this work to prior-art in this area. e.g. to Java, Midori, C++98
- [Design Discussion](#) - further discussion of important design points, alternatives, future work, etc.
- [Implementation Strategies](#) - discusses potential strategies implementations could use to implement this design efficiently

4. Motivation

The paper [P0709R4] "Deterministic exceptions: throwing values" by Herb Sutter contains detailed motivation for improving exceptions, and covers the background and history of exceptions in C++ better than I could do here.

I agree with much of the philosophy and motivations expressed in P0709 and have used them as inspiration for the design for this paper. However, this paper takes a different approach to solving the issues raised.

The paper [P1947R0] "C++ exceptions and alternatives" by Bjarne Stroustrup argues that adding yet-another-error-handling mechanism is likely to further fracture the community because existing mechanisms and styles won't disappear and efficiency improvements will be patchy and limited.

P1947 advocates instead for improving the existing C++ exceptions that we have and highlights some problems with some of the alternative error-code based handling mechanisms.

Improvements in this space should try to be consistent with, as much as possible, the existing C++ exception design. For example, by ensuring that a thrown exception object of type, `T`, remains an exception object of type `T` as it propagates up the stack, regardless of whether callers have dynamic or static exception specifications.

4.1. Alternative error-signalling mechanisms have problems too

Code-bases that avoid using exceptions, for whatever reason, need to choose an alternative error-signalling mechanism to signal to the caller that an operation failed.

Popular choices include using `std::error_code` (either as a return-value or out-parameter), `boost::outcome`, `std::expected`, `std::optional`, etc.

Many of these error-handling mechanisms force the user to make other trade-offs in return for avoiding exceptions. If we can reduce the reasons for people to avoid using exceptions, we can potentially reduce the need to make some of these trade-offs.

4.1.1. Composition of facilities that use different error-signaling mechanisms

As pointed out in P1947, these alternative error mechanisms aren't universally usable in all places. For example, calls to constructors are unable to change the return-type to return a value-or-error object, and overload operators

are unable to take additional out-parameters that can receive error-codes.

Many of these error-signalling mechanisms don't force users to handle the errors, meaning users can mask problems by accidentally ignoring errors, making their use "error"-prone.

The growing number of error-handling mechanisms in use means it is harder to compose components that use different error-handling techniques - often requiring explicit conversion/adaptation between components that use different error-signalling mechanisms.

Many generic algorithms assume that if an operation doesn't throw an exception then it must have succeeded. Adapting these algorithms to other error-signalling mechanisms often requires reimplementing them.

4.1.2. Runtime overhead due to additional branching, packing/unpacking

The alternative error-handling mechanisms can also incur runtime overhead on the success-path compared to exception-based code.

They often require returning a value that can either represent a success or an error. The calling code then needs to inspect the value to see which kind of result it is and then branch on the answer rather than just continuing on the success path.

Alternatives can inhibit copy-elision of returned values. Results typically need to be packed into some container, such as `std::expected`, and then unpacked by the caller before being passed by-value to another function, or used to initialize a data-member with aggregate initialization.

When propagating error values through many layers of a call-stack each caller needs to inspect the result, unpack the error, repack into a new wrapper object of a different return-type.

Both cases require additional calls to move-constructors compared to returning the success value on its own, or compared to throwing an exception.

4.1.3. May require an additional "invalid" state, additional preconditions

Alternative error-handling mechanisms also typically require objects to have an invalid state.

A constructor that can fail but that does not throw an exception will generally require a subsequent query after calling the constructor to check whether construction succeeded before attempting to use that object.

This can potentially require that object to hold additional state to represent that invalid state.

It also requires adding additional pre-conditions on each operation performed on the object that check that the object is valid.

4.2. Making exceptions fast by-design

Exceptions are expensive on many platforms for error-handling that is not rare. In some cases, taking the exceptional path can be 100-1000x slower than the normal return-path.

This can be especially problematic for services that use exceptions for signalling errors which trigger in service-overload situations. If a system is overloaded and starts raising errors but this results in taking code-paths that are significantly more expensive then this can result in a snow-balling effect where a few small errors quickly leads to much larger system failures.

The dynamic nature of exceptions as they are today, which relies on dynamic allocation, run-time type information and dynamic type-matching of handlers during unwind, makes it very difficult to reason about the performance of

code in exceptional situations.

Dynamic allocation can potentially make system-calls to acquire memory from the operating system, or can block acquiring locks on shared data-structures, leading to unpredictable, non-deterministic execution times.

The cost of dynamic type-matching when searching for a handler can depend on what DLLs happen to be loaded into the process - something that is very difficult to reason about locally.

While there have been efforts to reduce the cost of the dynamic exceptions, such as [FastCasting] and [Renwick], they do not solve the whole problem. For example, even with the stack-allocation techniques described in [Renwick], dynamic allocation is still required to transport exceptions across threads - something that will be increasingly common with the upcoming `std::execution` facilities proposed in [P2300].

The cost of exceptions in production implementations have remained roughly the same over the past few decades as improvements, to the extent they would be possible, would result in extremely disruptive ABI breaks.

All of these performance issues with exceptions are well-known and are one of the primary reasons why people avoid using exceptions, or restrict their use to genuinely rare error-conditions.

However, exceptions as a model for error handling generally have a lot of benefits and are well integrated into the language.

If we can find a way to make exceptions fast by-design, of the order of normal return-values and as efficient or better than the alternative error-signalling mechanisms, then it would opens up a number of possibilities.

Exceptions then become possible to use in environments that were traditionally unable to use them (or even enable them) for performance reasons. e.g. freestanding/embedded systems, or real-time systems such as games.

It also opens up opportunities for APIs to use exceptions to report errors that are not rare. This can help make the ergonomics of using these APIs nicer by keeping the success-path of using the APIs clean and free-of error-handling logic.

4.3. Supporting exceptions in freestanding and real-time environments

The current lack of support for exceptions in freestanding environments significantly limits the how much of the standard library can be used in those environments, and also forces those environments to use other error-handling techniques which are potentially more error-prone. This is unfortunate as many safety-critical systems target freestanding environments.

To be able to be used in freestanding environments, the code-size and run-time overhead of using exceptions needs to be small enough that it can be used on small micro-controllers that may only have 10's of KB of memory available.

Dependencies on dynamic allocation, run-time type-information and thread-locals can easily eat up a significant portion of memory available for program text.

For real-time environments, such as games, where high-performance and binary size is a key requirement, the current dynamic nature of exceptions and the common implementations of that design lead to overhead that is undesirable, even if exception paths are not evaluated.

4.4. Reducing bugs caused by failure to handle error-conditions

Use of exceptions in a program can introduce hidden control-flow that makes it hard to know if you have handled all possible error-cases in your function.

The existing proposals for exceptions have not yet sought to address the problem of programs unintentionally failing to handle exceptions as a potential source of bugs in a program. Although it is worth noting that [P0709R4]'s proposed `try` expression syntax at least sought to make the hidden control flow more visible.

If function authors are able to opt-in to having their function's exception specification checked at compile time to make sure they are not violating it by failing to handle some exception types then we can potentially reduce or eliminate two classes of bugs relating to unhandled exceptions.

The first case is where the programmer fails to handle a given exception due to an oversight when initially writing the function.

The second case is where a function was correct when initially written, but as code evolves, a function it calls is later modified to report new error-conditions for which the initial function was not aware it needed to handle.

Currently, the language offers little help to users wanting to catch this class of errors.

4.5. Describing the contract in code instead of in documentation

A function declaration defines the contract between callers of that function and the implementations of that function.

The status quo is that the function declaration currently only describes the types that a caller must provide as arguments, the type that the function will return on success, and whether or not the function might indicate failure by throwing an exception (via the `noexcept` specifier).

We have also added attributes such as `[[noreturn]]` and `[[nodiscard]]` that provide the user more information about whether the function can return normally, or whether the return-value is safe to discard.

With the contracts facility proposed in [P2900R6], we would gain the ability to describe runtime properties of the values that must be passed to a function and returned from it.

The set of exception-types that may be thrown forms an important part of a function's contract - it tells the caller "here are the ways in which I might fail", not just that it might fail.

However, currently, the ability to describe a set of exception-types that a function may throw in the function declaration is noticeably absent.

Prior to C++17, there was the ability to declare constraints on the set of exception types a function could throw, by declaring a function with a throw-specifier. The types listed, however, were not necessarily the complete set of types - the function was permitted to throw types derived from the types listed.

While this information was of some help to users who could know what handlers to write to match all of the exceptions, it was not useful for compilers in determining the potential size of exception objects in order to be able to return them on the stack. Some form of type-erasure and run-time type-information was still required. See [\(Re\)Adding throw\(\) specifiers](#) for more details.

This facility was deprecated in C++11 with the introduction of `noexcept` and removed in C++17.

Both users and compilers would benefit from having more information about the set of exceptions that can be thrown - allowing local reasoning about the potential exception code-paths without requiring global knowledge of the program.

4.5.1. Uses in generic code

When writing generic code that ends up calling into APIs on types provided by the user, we don't necessarily know what exception types they may throw. If that generic code wants to be able to capture an exception, store it and

rethrow it later, the best it can do is store the exception in a `std::exception_ptr` - meaning we need type-erasure and dynamic allocation.

If a program had a way to query the set of concrete exception types that may be thrown from an expression at compile time then the generic code could find more efficient ways to store an exception object, such as in a `std::variant`.

4.6. Reducing boiler-plate when writing functions that are transparent to exceptions

Authors of generic code often desire to write facilities that are transparent to exceptions. e.g. they call onto methods on types provided by the user and want to throw any exceptions that those methods can throw.

This is often expressed in standardese as "is expression equivalent to", or "Throws: exceptions thrown by `func(x)`".

Currently, if generic functions want their `noexcept` specifier to correctly report whether or not the function might throw an exception, the author must write an expression as a parameter to the `noexcept` specifier that computes this result.

To do this correctly, the author must identify and duplicate every expression in the function body (including any implicit conversions) and then query whether or not that expression can potentially throw by passing it as the argument to a `noexcept` expression.

This is highly tedious code to write and often difficult to get right, especially in the presence of constructs like `if constexpr` within the function body.

If you get it wrong by being conservative, resulting in a `noexcept(false)` when a `noexcept(true)` would have been appropriate then you might end up with slower code-generation, or dispatching to a less efficient algorithm (e.g. via `std::move_if_noexcept`).

However, it is much more likely that an author misses an expression and accidentally declares a function as `noexcept(true)` when it should have been `noexcept(false)`. If this is the case and the function throws then your program ends up terminating.

The compiler is much better placed to be able to reliably deduce whether or not a function can potentially throw an exception from the body than the programmer is.

Further, the extra syntactic noise in the signature of the function often obscures the more important parts of the function definition, like the parameters, `const`-qualifiers, etc, reducing readability of the code.

We already have the ability to deduce the return-type of a function from the function body by declaring a return-type of `auto` or `decltype(auto)`. It would simplify a lot of generic code that wants to be transparent to exceptions if we could also let the compiler deduce the exception specification.

5. Proposal

5.1. Overview

The key components of this proposal are as follows:

It proposes (re)adding throw specifiers which can be used to declare a function as having either a static exception specification, `throw(T1, T2...)`, or dynamic exception specification, `throw(...)` or `throw(std::any_exception)`.

It proposes adding the `throw(auto)` syntax for deducing the throw-specification of a function from its definition, which must be visible before use of the function.

It proposes rules for statically checking at compile-time that the bodies of function definitions with throw specifiers do not violate their exception specification. Failure to either handle all such cases or declare that you forward on unhandled exceptions are ill-formed.

It proposes adding the `declthrow(expr)` syntax for querying what set of exception types might be thrown from a particular expression.

It proposes adding the `template catch` syntax to allow catching static exceptions thrown from the associated try-block, allowing a way to use the same handler template to handle different types, without the need for type-erasing the exception.

5.2. (Re)Adding throw specifiers

The original design of exceptions in C++ included a throw-specification that allowed the programmer to declare a list of exception types that a function might exit with, by specifying the `throw(E1, E2, E3)` specifier after the function parameter list.

For example:

```
void Example(const std::string_view& path)
    throw(std::bad_alloc, std::system_error);
```

The throw-specification, as originally designed, had a number of issues that limited its usability and utility, and in time most people came to avoid the feature as its pitfalls outweighed the benefits of using it.

The following is a summarized list of the issues:

- The runtime/code-size overhead cost due to need to dynamically-check for unhandled exception types.
- The `std::unexpected()` notification mechanism did not lend itself to recovery from unhandled exceptions.
- MSVC (at the time) did not enforce the contract - a function with a `throw()` specification could still throw exceptions of types other than those mentioned in the throw-specification, but the compiler would optimise based assumptions that it did not. This made the feature dangerous to use as it would result in undefined behaviour if the programmer failed to adhere to the throw-specification.

In C++11, we introduced `noexcept`, initially as a tool needed to restore the strong exception-safety guarantee to types like `std::vector` after the introduction of move-constructors.

The original throw-specifications were deprecated along with the introduction of `noexcept` and, in C++17, were removed from the C++ language. This frees up the syntax for being reused for a similar purpose, albeit with an improved design that tries to avoid the pitfalls of the original design.

5.3. Static and dynamic exception specifications

A *static exception specification* is an exception specification that lists a finite set of possible exception types that a function may exit with.

A *non-empty static exception specification* is a static exception specification that contains one or more exception types listed in the throw specification.

A *dynamic exception specification* is an exception specification that allows the function to exit with any exception type.

A *throw-specifier* can be used to declare a function with either a static or dynamic exception specification.

For example:

```
void f() throw();    // static-exception-specification with empty exception type list
                    // equivalent to noexcept(true)

void g() throw(...); // dynamic-exception-specification
                    // equivalent to noexcept(false)

void g() throw(std::any_exception); // equivalent to throw(...)
                                   // see section on declthrow for rationale

void h() throw(E1);    // throws only E1 (static-exception-specification)
void i() throw(E1, E2); // throws either E1 or E2
void j() throw(Es...); // throws one of the types in pack Es...

void k() throw(auto);  // set of exceptions it could throw is deduced from body of function
                    // much like using decltype(auto) to deduce the return-type.
```

A declaration signature of `void foo() throw();` is equivalent to `void foo() noexcept;`. However, a function definition with a `throw()` specifier differs from one with `noexcept` specifier in that the definition is ill-formed if an exception can possibly escape the function, whereas `void foo() noexcept` detects such a failure to fulfil its contract at runtime and terminates. i.e. throw-specifications are statically checked/enforced rather than dynamically checked/enforced.

Similarly, `void bar() throw(E1, E2)` is ill-formed if any exception types other than E1 or E2 can possibly escape the body of the function. The aim is to avoid hidden calls to `std::terminate` in response to unhandled exceptions.

It is permitted to declare a function with the specifier `noexcept(true)` and define it with the specifier `throw()`, and vice versa. Doing so allows you to have the compiler statically check that there are no unhandled exceptions exiting the function body that might implicitly result in a call to `std::terminate`.

Similarly, it is permitted to declare a function with the specifier `noexcept(false)` and define it with the specifier `throw(...)`, and vice versa. However, there are no differences in semantics of the definition between these two syntaxes - they are pure aliases for each other.

A forward declaration of a function with a non-empty static exception specification on its definition must have an equivalent static exception specification on the declaration.

5.3.1. Types in a throw-specification form an unordered set

The order of the types in the throw-specification is not significant. The throw-specification declares an unordered set of types that may be thrown, rather than an ordered list of types.

Two throw-specifications are equivalent if they contain the same set of types, regardless of the order in which those types are listed in the source code.

It is valid to list a type multiple times in a throw-specification. Any duplicates are ignored/eliminated by the compiler.

Eliminating duplicates is helpful when composing lists of exception types from multiple `declthrow` expressions that have overlap in the set of exceptions they may throw - see the section "Querying the throw-specification".

For example, the following functions all have the same exception specification:

```
void f() throw(E1, E2);
void g() throw(E2, E1);
void h() throw(E1, E1, E2);
```

The rationale for making the set of exceptions an unordered set rather than an ordered list is to reduce the chance of annoying incompatibilities when casting a function to a function-pointer.

For example:

```
// declared in lib1
void f() throw(E1, E2);

// declared in lib2
void g() throw(E2, E1);

void (*func)() throw(E1, E2) = &f;
if (cond) {
    func = &g; // It would be annoying if this was ill-formed because the throw-specification
}
```

5.3.2. Handling of `std::any_exception` in the throw-specifier

The `std::any_exception` type is a type that is handled specially by throw specifications. See the section "declthrow of a `throw(...)` expression" below for a definition of this type.

If the list of types passed as arguments to the throw specifier contains the type `std::any_exception` then the overall exception-specification is evaluated to be `throw(...)`. i.e. that it can throw any exception type.

For example:

```
void a() throw(std::any_exception); // -> throw(...)
void b() throw(A, B, std::any_exception); // -> throw(...)
```

The use of a type `std::any_exception` allows template metaprogramming libraries to be able to conditionally compute a throw-specification that can evaluate as either a static exception specification or a dynamic exception specification.

For example: Computing a throw-specification to either be `throw(...)` or a static exception specification, depending on a template parameter.

```
template<typename... Ts>
using ...pack = Ts; // P1858R2 pack alias syntax

// Generic case
template<typename T>
struct _compute_foo_throw_types {
    using ...types = pack<std::any_exception>; // P1858R2 pack alias syntax
};

// When T satisfies the Foo concept, we know it will only
// fail with two possible exceptions.
```

```

template<typename T>
    requires Foo<T>
struct _compute_foo_throw_types<T> {
    using ...types = pack<FooError, std::bad_alloc>;
};

template<typename T>
void foo(const T& x) throw(_compute_foo_throw_types<T>:...types...); // P1858R2 pack exp

```

Here, the function `foo<T>` has an exception specification that is either `throw(FooError, std::bad_alloc)` or `throw(...)`, depending on the type, `T`.

5.3.3. The types in the throw specification describe all concrete types that may be thrown

One of the prime motivations behind re-adding throw-specifiers is to provide the compiler with enough static type information for it to be able to allocate storage for exceptions that may be thrown on the stack of the caller, rather than relying on dynamic type-matching or run-time type information.

For this to be possible, the compiler needs to know the size/alignment of all exception types so that it can reserve storage in the stack-frame for any exception-types which cannot be passed back to the caller in registers. Similarly, it needs to know which exception types may be passed back in registers.

This means that we cannot just list an exception base-class in the throw-specifier and then leave the set of possible exception types open to include any type derived from that base-class, as this would not allow callers to reserve space for any such exception on the stack-frame caller.

For example, a declaration with a throw-specifier of `throw(std::exception)` does not declare that the function may throw an exception derived from `std::exception`, it instead states that the function may throw an instance of `std::exception` (e.g. as if via `throw std::exception{};`) and does not exit with any other type of exception.

The implication of this restriction, however, is that any changes to the set of exception-types that may be thrown by a function is a potential ABI break for that function, requiring, at a minimum, recompilation of all callers of that function.

This is no different to changing the return-type of a function. e.g. when adding a new entry to a `std::variant`-returning function.

This places some interesting constraints on the evolution of such functions, which are discussed in detail in a later section.

5.3.4. Exception types may not be references, cv-qualified, or void

Types listed in the throw-specifier may not be references, cv-qualified, or void.

Static-exception types are returned by-value to callers, so it does not make sense to support throw-specifiers that include types that are references or cv-qualified.

5.3.5. Static exception specifications are part of the function type

A static exception specification is part of the function type, much like `noexcept` specifier is part of the function type.

In general, a function-pointer with a non-empty static-exception-specification cannot be cast to a function-pointer type with a different exception-specification. This is because the calling-convention between such functions may be different, as the list of exceptions that may be thrown forms part of the ABI of such a function.

Note that it is possible to cast a function directly to a function-pointer type with a wider exception specification than the function was declared with as the compiler is able to then generate a thunk that can implement the ABI for the wider specification in terms of the function's native ABI.

Once the identity of the function has been erased as a function-pointer, it is no longer possible for the compiler to know how to generate such a thunk.

For example:

```
void f() throw();
void g() throw(E1);
void h() throw(E1, E2);
void i() throw(...);

void(*pf)() throw() = f; // OK
pf = g; // ERROR - can't cast g() to a function-ptr with narrower throw-specification
pf = h; // ERROR - can't cast h() to a function-ptr with narrower throw-specification
pf = i; // ERROR - can't cast i() to a function-ptr with narrower throw-specification

void(*pg)() throw(E1) = g; // OK
pg = f; // OK - points either to f or to thunk that calls f
pg = h; // ERROR - can't cast h() to a function-ptr with narrower throw-specification
pg = i; // ERROR - can't cast i() to a function-ptr with narrower throw-specification

void(*ph)() throw(E1, E2) = h; // OK
ph = f; // OK - ph points to f or to a thunk that calls f
ph = g; // OK - ph points to a thunk that calls g
ph = i; // ERROR - can't cast i() to function-ptr with narrower throw-specification

void(*pi)() throw(...) = i; // OK
pi = f; // OK - ph points to f (same as casting noexcept(true) function-ptr to a noexcept
pi = g; // OK - ph points to a thunk that calls g and translates static-exceptions into c
pi = h; // OK - ph points to a thunk that calls g and translates static-exceptions into c

// The same casts are not all valid when casting function-pointers to other function-poin
// types instead of functions to function-pointer types.
pf = pg; // ERROR: Can't cast function-ptr with static throw specification to another fun
pf = ph; // ERROR: (same)
pf = pi; // ERROR: Can't cast throw(...) function-ptr to throw() function-ptr

pg = pf; // MAYBE?: In some ABIs the calling convention may be compatible.
// Do we want to restrict the options here?
pg = ph; // ERROR: Can't cast to function-ptr with narrower throw-specification
pg = pi; // ERROR: Can't cast to function-ptr with narrower throw-specification

ph = pf; // MAYBE?: In some ABIs the calling convention may be compatible.
ph = pg; // ERROR: Can't cast function-ptr with static exception specification to functio
// different exception specification. Compiler is unable to generate the necessa
ph = pi; // ERROR: Can't cast to function-ptr with narrower throw-specification.

pi = pf; // OK: this is same as casting function-ptr with noexcept(true) to function-ptr
pi = pg; // ERROR: Can't cast function-ptr with static exception specification to functio
// different exception specification. Compiler is unable to generate the necessa
pi = ph; // ERROR: Can't cast function-ptr with static exception specification to functio
// different exception specification.
```

The existing type-conversions from pointers to a function with a `noexcept(true)` exception specification to a pointer to a function with a `noexcept(false)` exception specification are unchanged.

5.3.6. Deducing the throw-specifier from a function signature

It is permitted to allow template arguments to be deduced from the throw-specification in a function-signature.

For example:

```
template<typename Ret, typename... Args, typename... Errors>
void Call(Ret(*func_ptr)(Args...) throw(Errors...));

void a() throw();
void b() throw(int);
void c() throw(std::bad_alloc, std::system_error);
void d() throw(...);

Call(&a); // deduces Errors to be the empty pack.
Call(&b); // deduces Errors to be the pack: int
Call(&c); // deduces Errors to be the pack: std::bad_alloc, std::system_error (in some
Call(&d); // deduces Errors to be the pack: std::any_exception
```

This is similar to the ability to deduce whether a function signature is `noexcept` or not.

5.3.7. `throw(auto)` - Deducing exception-specifications from the body of a function

Often, when writing forwarding functions, or function templates, you just want the function to be transparent to exceptions. i.e. any unhandled exceptions should be propagated to the caller.

In these cases, ideally the function's exception-specification should mirror the set of exceptions that the body of the function may throw.

With the current facilities available with `noexcept`, this typically means that you need to repeat every expression in the body of the function in the `noexcept` specifier for that function.

For simple functions this is manageable, although tedious. However, for more complicated function bodies, or for function-bodies that include conditionally-executed logic guarded by an `if constexpr` branch, the expression needed to compute the `noexcept` specifier argument quickly becomes unwieldy.

1. Prior work on deducing exception specifications

This usability issue was identified as a problem back when `noexcept` was originally proposed for C++11:

- [N3227](#) - Please reconsider `noexcept` (Ottosen, 2010)

There have since been multiple papers exploring the idea of deducing the exception-specification:

- [N3202](#) - To which extent can `noexcept` be deduced? (Stroustrup, 2010)
- [N3207](#) - `noexcept(auto)` (Merrill, 2010)
- [N4473](#) - `noexcept(auto)`, again (Voutilainen, 2015)
- [P0133R0](#) - Putting `noexcept(auto)` on hold, again (Voutilainen, 2015)

It is worth noting that the rationale given in P0133R0 for putting on hold the pursuit of `noexcept(auto)` was mainly because it did not solve the whole problem of having to duplicate the function-body in the

declaration - the expressions of the body still needed to be duplicated in the return-type for SFINAE purposes - and therefore it was not good use of committee time to pursue a partial solution.

Since this paper was written, we have gained support for concepts in C++20, which goes some way to simplifying the code needed to write function-templates that eliminates overloads with SFINAE. However, this only applies when there are existing concepts defined that can be used to constrain the function. For many cases you still need to duplicate the expressions of the function body in a `requires` clause.

Despite this limitation, I feel there is still benefit to enabling deduced exception specifications as there are often cases that are either covered by concepts or that do not require SFINAE, but that do need to compute accurate exception specifications.

2. `throw(auto)`

With the (re)introduction of throw-specifiers, the task of computing a correct throw-specification from a set of sub-expressions becomes even more onerous than for `noexcept`, as you need to compute lists of types, not just a boolean expression.

This paper therefore proposes the addition of the `throw(auto)` specifier on a function declaration, as a way of declaring that the compiler should compute the set of exception types that may exit the function from the definition of the body of the function and use that as the exception-specification for the function.

For example, consider a hypothetical `for_each` function that invokes a function for each element of a range. If we wanted this function to have the same exception-specification as its body, it would need to be written with `noexcept` specifiers, something similar to the following:

```
template<
    std::ranges::range Range,
    typename Func>
requires std::invocable<Func&, std::ranges::range_reference_t<Range>>
void for_each(Range&& range, Func&& func)
    noexcept(noexcept(std::ranges::begin(range)) &&
             noexcept(std::ranges::end(range)) &&
             noexcept(++std::declval<std::ranges::iterator_t<Range>&>()) &&
             noexcept(std::declval<std::ranges::iterator_t<Range>&>() != std::declval<std::ranges::iterator_t<Range>&>()) &&
             noexcept(func(*std::declval<std::ranges::iterator_t<Range>&>()))) {
    auto iterEnd = std::ranges::end(range);
    auto iter = std::ranges::begin(range);
    while (iter != iterEnd) {
        func(*iter);
        ++iter;
    }
}
```

And with the `throw()` specifier proposed by this paper, in conjunction with the `declthrow()` expression (described in detail in the following section), we would need to write:

```
template<std::ranges::range Range, typename Func>
requires std::invocable<Func&, std::ranges::range_reference_t<Range>>
void for_each(Range&& range, Func&& func)
    throw(declthrow(std::ranges::begin(range))...,
          declthrow(std::ranges::end(range))...,
          declthrow(++std::declval<std::ranges::iterator_t<Range>&>())...,
          declthrow(std::declval<std::ranges::iterator_t<Range>&>() != std::declval<std::ranges::iterator_t<Range>&>())...,
          declthrow(func(*std::declval<std::ranges::iterator_t<Range>&>())...)) {
    auto iterEnd = std::ranges::end(range);
    auto iter = std::ranges::begin(range);
    while (iter != iterEnd) {
```

```

    func(*iter);
    ++iter;
}
}

```

Having to repeat the body in a different way in the `noexcept` or `throw` specification like this is tedious and error-prone. It can be easy to miss an expression, or to later modify the body of the function and forget to update the throw-specification.

Instead, if we use the proposed `throw(auto)` syntax, then the function definition simply becomes:

```

template<std::ranges::range Range, typename Func>
requires std::invocable<Func&, std::ranges::range_reference_t<Range>>
void for_each(Range&& range, Func&& func) throw(auto) {
    auto iterEnd = std::ranges::end(range);
    auto iter = std::ranges::begin(range);
    while (iter != iterEnd) {
        func(*iter);
        ++iter;
    }
}

```

This is much more concise, and is now impossible for the throw-specification to be inconsistent with the function body.

This facility will greatly simplify the definition of function-templates, in particular the function-templates that are defined as "expression-equivalent to" some expression or statement sequence.

3. Further motivation for `throw(auto)` from P2300 `std::execution`

One place where having accurate exception specifications (whether `noexcept` or `throw()` specifications) is when using the `std::execution` facility proposed in P2300.

There are generic async algorithms that can potentially have more efficient implementations if they know that a given operation cannot fail with an error.

For example `when_all()` when passed a collection of senders that cannot complete with an error the implementation can avoid introducing expensive stop-token synchronization required for cancelling other child operations if one of them fails. It can also avoid having to reserve storage for a `std::exception_ptr` (or other error type) in the operation-state in order to be able to stash the error while waiting for the other operations to stop.

So throughout the design of P2300, the specification tries to ensure that, as much as possible, the `noexcept`-ness of expressions are passed-through. An unnecessarily conservative `noexcept(false)` can result in additional overhead that the compiler cannot inline away like it can for normal functions.

The `noexcept`-ness of operations on arguments passed to `std::execution` algorithms can influence the return-type of functions, whether particular overloads of template `set_error()` functions are instantiated, etc. and so can influence the ABI and whether a program is well-formed.

For users using the `std::execution` algorithms, using the `throw(auto)` syntax would be beneficial for cases where they are passing lambdas as parameters to these algorithms and they either:

1. Don't care whether or not the expressions could throw, but if they can then just do the right thing by having those expressions transparently propagate exceptions, and if they don't then do the fast thing.

2. The do care, but they are writing generic code which may or may not be noexcept depending on the types it is instantiated with.

For example:

```
template<std::execution::sender S>
auto sender_example(S source) throw(auto) {
    return std::move(source)
        | std::execution::then([](const auto& data) throw(auto) {
            // do something with data that might throw or might not throw depending on
            return some_computed_value;
        })
        | std::execution::let_value([](auto& computed_value) throw(auto) {
            return std::execution::when_all(
                sub_operation_1(computed_value),
                sub_operation_2(computed_value))
            | std::execution::then([&](auto op_1_result, auto op_2_result) throw(auto) {
                // ... combine results
                return some_expr;
            });
        });
}
```

If we want this expression to produce a sender that is no-fail when the lambdas within it are guaranteed not to throw exceptions then currently you'd have to duplicate the body of each of the lambdas in the noexcept/throw-specifier. This greatly affects the readability of this sort of code. Most people are probably not going to bother and so the sender algorithm will have to pessimistically choose a less-efficient implementation to handle the possibility that some of those expressions might throw. If the author of the lambdas had access to `throw(auto)` then users would probably annotate their lambdas as a matter of course so that their sender/receiver code runs faster when appropriate.

5.3.8. Forward declarations of `throw(auto)` functions

The use of `throw(auto)` on a forward-declaration of the function requires that the definition of the function is visible before the use of the function, in the same way that a function declared with a deduced-return-type requires that the function definition is available before it's ODR-used. This is consistent with the behaviour of functions with deduced return-types.

For example:

```
void example() throw(auto);

void caller1() {
    example(); // ill-formed. cannot be ODR-used before the definition is seen
}

auto* example_ptr = &example; // ill-formed. Type of example() is not known until definition

void caller2() throw(declthrow(example())...); // ill-formed. Cannot query the exception
                                                // of example() before it's definition is
void caller3() noexcept(noexcept(example())); // ill-formed. For same reason.

void example() throw(auto) {
    if (foo()) {
        do_thing1();
    } else {
        try {
```

```

        do_thing2();
    } catch (Thing2Failure) {
        do_backup_thing2();
    }
}

// Now that the definition is visible and the exception-specification
// can be deduced, the following usages are well-formed.

void caller4() throw(declthrow(example())) { // OK
    example(); // OK
}

auto* example_ptr2 = &example; // OK

```

The restriction that the function definition with a deduced exception specification needs to be visible before it can be used has implications for recursive functions, however.

5.3.9. Deduced exception-specifications and recursive functions

Supporting deduced exception-specifications for recursive functions is a challenge.

In theory we could define some language rules that would allow some kinds of recursive functions to be able to deduce their exception-specification.

For example:

```

struct Tree {
    Tree* left;
    Tree* right;
    int value;
};

void process_value(int value) throw(InvalidValue);

void process_tree(Tree& tree) throw(auto) {
    if (tree.left != nullptr)
        process_tree(*tree.left);

    process_value(tree.value); // recursive-call

    if (tree.right != nullptr)
        process_tree(*tree.right);
}

```

In this case, the only call that is made that is not recursive is the call to `process_value()` which can throw `InvalidValue`. Therefore, we could in theory deduce that the overall throw specification is `throw(InvalidValue)`.

However, it is relatively easy to construct examples where such rules would not work.

Consider:

```

void contradiction(int arg) throw(auto) {
    if constexpr (noexcept(contradiction(arg))) {
        throw X{};
    }
}

```

```

    } else {
        if (arg > 0)
            return contradiction(arg - 1);
    }
}

```

If the throw-specification is deduced to be `throw()` then it throws an exception, otherwise if it is potentially throwing, it calls itself but no longer contains any statements that might throw an exception except the call to itself, leading to a contradiction.

The key feature of this example that makes it problematic is that it is attempting to query the exception specification before the exception specification has been deduced.

There are also other cases that can directly or indirectly require the exception specification to be known. Including:

- Calling the function within a `try { ... } template catch (auto e) { ... }` block. The template catch block needs to know the types that might be thrown in order to instantiate the catch-block with the correct types.
- Passing a pointer to the function to an algorithm. Constructing the function-pointer type to pass requires knowing the exception specification.
- Forming a call to the function as a sub-expression passed to `declthrow()`.

There are also further challenges with defining mutually-recursive functions that both have deduced exception specifications.

While we may be able to eventually define rules that may allow a subset of recursive function use-cases to have deduced exception specifications, this seems like a relatively niche case and so this paper proposes that it be left ill-formed for now.

5.3.10. Delayed computation of deduced throw specifications

The exception specification of a function or function-template with a `throw(auto)` specifier need only be computed when the function is selected by overload resolution, or is otherwise ODR-used.

This allows the compiler to avoid instantiating function-templates that are part of an overload set but that are never selected for overload resolution in order to compute the throw specification.

Taking the address of a function with a deduced throw-specification will also force the compiler to compute the throw-specification so that the function-pointer type is known.

This behaviour is consistent with functions with computed/deduced `noexcept` specifiers today.

5.3.11. Do we also need `noexcept(auto)`?

We could also consider adding support for the `noexcept(auto)` syntax, in addition to `throw(auto)`.

The primary semantic difference between these two would be that `noexcept(auto)` would only deduce to either `noexcept(true)` or `noexcept(false)`, (equivalent to `throw()` or `throw(...)`, respectively), whereas `throw(auto)` could also deduce to a non-empty static-exception-specification.

While, in most cases, it would be preferable to use `throw(auto)`, as that allows the exception-specification to deduce to the more-efficient static-exception-specification, where possible, there may be some scenarios where deducing to either `noexcept(true)` or `noexcept(false)` could be preferable.

The one use-case I can think of is where you want to have the exception-specification deduce to a function whose signature allows a pointer to that function to be assigned to a function-pointer variable that has a `noexcept(false)` exception-specification.

However, this use-case is somewhat tenuous as it would still be possible to directly cast any function to a signature-compatible function-pointer with a `noexcept(false)` exception-specification, it's just not possible to cast first to a function-pointer with a non-empty static exception specification and then cast that function-pointer to a function-pointer with a `noexcept(false)` exception-specification.

For example:

```
void a() throw(A);
void b() throw(B);

void c() throw(auto) { // deduces to throw(A, B)
    a();
    b();
}

void d() noexcept(auto) { // deduces to noexcept(false)
    a();
    b();
}

void execute(void(*func)());

void example() {
    auto* c_ptr = &c;
    execute(c_ptr); // ill-formed: no conversion from 'void(*)() throw(A,B)' to 'void(*)()'

    auto* d_ptr = &d;
    execute(d_ptr); // OK: 'void(*)() noexcept' implicitly convertible to 'void(*)()'.
}

void workaround() {
    execute(static_cast<void(*)>(&c)); // OK: explicit cast to noexcept(false) function-p
    execute(&d); // OK: Explicit cast not needed
}
```

It is an open question whether adding support for `noexcept(auto)` in addition to `throw(auto)` is worth the extra complexity/specification effort.

However, in the author's opinion, it is probably not necessary to add in the initial version. It can be added later if usage experience shows that it would have sufficient value.

5.4. Querying the throw-specification

Once we have the ability to specify static-exception-specifications on functions, there will inevitably be cases where we want to be able to know what that set of exception types is in library code.

This paper proposes adding `declthrow(expr)` syntax as a way of querying what the list of exceptions that `expr` may exit with.

As the `declthrow(expr)` needs to be able to produce a list of types, it is proposed that this form names a pack of types, which can be expanded as needed using `declthrow(expr)...`

Note that the pack of types produced by `declthrow()` does not contain any duplicate types.

One of the common expected use-cases is in computing a derived throw-specification for a function composing other functions such that if their exception specifications change then so does the exception specification of the function composing them.

For example:

```
// Header file
void PartA() throw(OutOfWidgets);
void PartB() throw(ProtocolError, Timeout);

void ComposedOperation() throw(declthrow(PartA())...,
                               declthrow(PartB())...);

// ... out-of-line definition in .cpp file

void ComposedOperation() throw(declthrow(PartA())...,
                               declthrow(PartB())...) {
    PartA();
    PartB();
    try {
        PartC();
    } catch (...) {
        NothrowFallbackPart();
    }
}
```

5.4.1. declthrow of a call to a throw(...) function

If the expression may exit with a dynamic-exception (i.e. one of the sub-expressions has an exception specification of `noexcept(false)` or `throw(...)`) then the result of this is a compiler-generated type, much like `decltype(nullptr)`.

An alias for this type is made available as `std::any_exception` in the header `<exception>`.

```
namespace std {
    // NOTE: using pack indexing syntax proposed in P2662R2
    using any_exception = declthrow(static_cast<void*>()throw(...)>(nullptr)())...[0];
}
```

The `std::any_exception` type is not constructible or usable as a value. It is only intended for use as a placeholder/marker for throw-specifications to indicate a dynamic exception specification.

An alternative design worth considering is having the special type that indicates a dynamic exception specification to instead be the type `std::exception_ptr`.

This would be useful in cases where you want to store the exception results in a `std::variant`. However, it would mean that you could not have an exception specification that allowed throwing a `std::exception_ptr` object itself (instead of rethrowing the exception object contained within the `std::exception_ptr`).

This is explored in more detail in the design discussion section.

5.4.2. Mixed dynamic and static exception specifications

When the operand to `declthrow()` contains multiple sub-expressions, some of which have non-empty static exception specifications and some of which have dynamic exception specifications, there is the question of what the result of the `declthrow()` query should be.

For example:

```
// Given the following
struct X;
struct Y;
int foo() throw(X, Y);
void bar(int x) throw(...);

// What types are in the following type-list?
using types = type_list<declthrow(bar(foo()))...>;
```

There are two viable options to consider here:

- We say that the overall expression could emit any exception, so the deduced exception specification of a function containing this expression would be `throw(...)`, and so the resulting type list should contain only `std::any_exception`; or
- We list the union of all of the types listed in static exception specifications and also list `std::any_exception` in the result.

This paper proposes to have the result include both `std::any_exception` and the types from any static exception specifications, for the following reasons:

- It can be used to determine what types might be used to instantiate a template catch block (see section on this below) associated with a try-block that contains this expression.
- It is not necessary to reduce the result to `std::any_exception` in the `declthrow()` expression if it is being used as the argument to a throw specifier - the throw specifier will do the reduction for you. Reducing the result early is just throwing away type information.

For example: With this behaviour we can write the following code

```
template<typename T, typename... Ts>
concept one_of = (std::same_as<T, Ts> || ...);

template<typename... Es>
using err_variant = std::variant<std::monostate,
                                std::conditional_t<std::same_as<std::any_exception, Es>,
                                                    std::exception_ptr, Es>...>;

err_variant<declthrow(do_foo())...> error;
try {
    do_foo();
} template catch (auto e) {
    error.emplace<decltype(e)>(std::move(e));
} catch (...) {
    if constexpr (one_of<std::any_exception, declthrow(do_foo())...>) {
        error.emplace<std::exception_ptr>(std::current_exception());
    }
}
```

5.4.3. Order of the exception types

In the section on throw-specifications above it noted that the order of types listed in the throw specification was not significant, and that the types in the throw-specification formed an unordered set for the purposes of function-type-equivalence.

However, when querying the types in the throw-specification, we need to return the types in *some* order, and so we need to specify what the constraints of that order are.

At the very least, the order of the types returned needs to be deterministic and consistent across different queries of the same expression, across all translation-units. This is because code may compute types that have different layouts or ABIs based on the order of the types produced by the `declthrow` expression, and having the same computation produce the results in different orders is a sure-fire way to introduce ODR-violations.

Any unique set of exception types queried via a `declthrow` expression or by deducing the types listed in a static exception specification of a function signature needs to consistently yield a list of types in that set in a canonical order.

There are a few other questions around the ordering of the exception types:

- Should the order be a total ordering of all types? i.e. if E1 appears before E2 in some `declthrow()` query, then E1 appears before E2 in *all* `declthrow()` queries that return those two types.
 - This would effectively provide a built-in facility for sorting types in type-lists.
 - Note that [P2830R1] "Standardized Type Ordering" is also exploring the design space for sorting of types. Should the order that types are returned in be consistent with the order produced by that facility?
- Should the order be specified by the standard? or should it be unspecified/implementation-defined?
 - It might be difficult to specify an ordering of all types in a portable way.
 - Doing so may improve portability/compatibility of code across compilers, however.
 - Standard library implementations do not necessarily define all types with portable canonical names. e.g. some implementations place some `std::` library types inside inline ABI-version namespaces, which would give those types different names to the same types defined in other standard library implementations, which would negate some of the portability benefit.
- Should the order of the exceptions from a `declthrow()` query be consistent with the order of exception types deduced from the throw-specification of a function type? For example:

```
// Given the following.
void foo() throw(A, B);

template<typename T>
struct throw_specifier;

template<typename Ret, typename... Args, typename... Es>
struct throw_specifier<Ret(Args...) throw(Es...)> {
    using ...types = Es;
};

template<typename... Ts>
struct type_list {};

// Should the following static_assert be guaranteed to hold on all conforming imple
static_assert(std::same_as<type_list<declthrow(foo())...>,
               type_list<throw_specifier<decltype(foo)>::types...>>);
```

- Should the `std::any_exception` type appear in a specific location within the types returned by `declthrow()` if it is present? e.g. as the first or last type in the pack.
 - This might make it easier/more compile-time efficient to write metafunctions that want to detect whether there is a dynamic exception that may be thrown. e.g.

```

// If std::any_exception is always first type
template<typename... Es>
concept DynamicException = sizeof...(Es) > 0 && std::same_as<std::any_exceptio

// vs

// If std::any_exception could appear anywhere
template<typename... Es>
concept DynamicException = (std::same_as<std::any_exception, Es> || ...);

```

- Doing so might be inconsistent with rules for sorting types, however, if we decide that the type list produced by a `declthrow()` query must produce types in a sorted order consistent with the sorting order described in P2830.
- Do exception types need to be complete when used in throw specifications and subsequently queried via `declthrow()`?
 - This may be somewhat limiting - preventing use of
 - It would open the possibility of sorting types based on their ABI properties like size/trivial-copyability, etc. e.g. so that all error-types that might be returned by register appear earlier in the list
 - The exception types need to be complete anyway when a function that might throw them is invoked, just like the return-type needs to be complete.
 - The syntax proposed below for filtering exception types would need the exception types to be complete so that it can determine whether they would match a given catch handler.

5.4.4. Exception specifications of defaulted special member functions

See [dcl.fct.def.default].

The following functions may have defaulted definitions

- special member functions
 - default ctor
 - move ctor
 - copy ctor
 - move assignment
 - copy assignment
 - destructor
- comparison operators
 - equality
 - three-way-comparison

For defaulted functions:

- implicitly defaulted functions have an implicit exception specification
- explicitly defaulted functions which are defaulted on first declaration have an implicit exception specification if they don't explicitly specify an exception specification.
- explicitly defaulted functions which are defaulted on first declaration that have an explicit exception specification use that explicit exception specification.

This paper proposes changing the implicit exception specifications of defaulted functions to be equivalent to a throw-specification of `throw(auto)`.

This should have no semantic effect on existing types / existing programs as all existing types will have either a `noexcept(true)` or `noexcept(false)` member function and thus the deduced exception specification will either deduce to `noexcept(false)` or `noexcept(true)`. The rules for deduction of the exception specification via `throw(auto)` are consistent with the pre-existing rules of deduction for defaulted member functions.

However, it would ideally have an effect on types that compose new types that are defined with static exception specifications for these special member functions.

For example: Defining a struct that composes two types with static exception specifications on their special member functions.

```
struct A {
    A() throw(std::bad_alloc);
    A(const A&) throw(std::bad_alloc);
    A(A&&) throw();
    ~A();
};

struct B {
    B() throw(std::system_error);
    B(const B&) throw(std::system_error);
    B(B&&) throw();
    ~B();
};

struct C {
    A a;
    B b;

    // C has implicitly defaulted special member functions.
};

template<typename... Ts>
struct type_list;

template<typename Func>
struct throw_specification;

template<typename Ret, typename... Args, typename... Es>
struct throw_specification<Ret(Args...) throw(Es...)> {
    using types = type_list<Es...>;
};

// Sorts the list of types in the canonical order for a throw-specification
template<typename... Ts>
using throw_specification_t = typename throw_specification<void() throw(Ts...)>::types;

// The following static_asserts will always pass for conforming implementations.
static_assert(std::same_as<throw_specification_t<declthrow(C{})...>,
                    throw_specification_t<std::bad_alloc, std::system_error>>);
static_assert(std::same_as<throw_specification_t<declthrow(C{std::declval<const C&>()})...>,
                    throw_specification_t<std::bad_alloc, std::system_error>>);
static_assert(std::is_nothrow_move_constructible_v<C>);
```

It would also be ideal if the same approach could be applied to special member functions of certain standard library types.

For example: Constructing a `std::tuple` of types with default-constructors with static exception specifications would ideally result in the `std::tuple` type having a static exception specification.

```
// Ideally the following would hold true for all implementations.
// i.e. the throw-specification of the default constructor of std::tuple is the union
// of the throw-specifications for all of the tuple member default constructors.
```

```
static_assert(std::same_as<throw_specification_t<declthrow(std::tuple<A, B>{})...>,
                      throw_specification_t<std::bad_alloc, std::system_error>>);
```

It's worth noting that, as currently specified, the default constructor of `std::pair` or `std::tuple` is not required to be declared `noexcept` if all of its member default constructors are declared `noexcept`, so making this work would require a change to the exception-specification of the default constructors.

The copy/move constructors are, however, declared as either implicitly or explicitly defaulted, which therefore implies that the exception specification for these functions is deduced from the exception specifications of the members.

A more in-depth analysis of standard library types is required to determine where this kind of defaulted exception specifications can be applied.

5.4.5. Introducing a pack outside of a template

The introduction of a `declthrow(expr)` syntax that can introduce a pack of types at an arbitrary point within the program.

It may be problematic for some compilers to support arbitrary use of anonymous packs outside of templates.

If this is a restriction we want to maintain in the language, then it's possible we can restrict, for now, the `declthrow(expr)` syntax to having to be immediately expanded in-place to the list of types. i.e. `declthrow(expr)` must be immediately followed by a `...` to expand the pack.

While this would be somewhat restrictive, it would still allow some basic common usage within `throw()` specifiers, and can be used to expand into the template arguments of variadic class templates, or concepts.

For example:

```
template<typename... Ts> class type_list {};

// Can pass the result as template arguments to a class-template.
using error_types = type_list<declthrow(foo(a,b,c))...>;

template<typename T, typename... Ts>
concept one_of = (std::same_as<T, Ts> || ...);

// Can pass the result as template-arguments to a concept.
constexpr bool throws_bad_alloc =
    one_of<std::bad_alloc, declthrow(foo(a,b,c))...>;

// Can use it to compute the type of a variant that can hold all
// possible exception types that might be thrown.
std::variant<std::monostate, declthrow(foo(a,b,c))...> error;
try {
    foo(a,b,c);
} template catch (auto e) {
    error.template emplace<decltype(e)>(std::move(e));
}

// Can use it in the throw-specification of a function that wants to transparently
// throw whatever exceptions foo() throws, plus errors that it throws itself.
void example(int a, int b, int c) throw(std::system_error, declthrow(foo(a,b,c))...);
```

However, it wouldn't be able to support things like the following:

```

void foo() throw(A, B);

template<typename Nested>
struct BarError {
    Nested nested;
};

void bar(int count) throw(BarError<declthrow(foo())>...) {
    try {
        for (int i = 0; i < count; ++i) {
            foo();
        }
    } template catch(auto e) {
        throw BarError<decltype(e)>{std::move(e)};
    }
}

```

As that requires using the pack in way that is not immediately expanding the pack.

Further, if we do not have the ability to generate a pack in a non-template then we will not be able to take a type-list computed by some meta-programming and then expand that type-list into elements of the `throw()` specification.

```

template<typename... Ts>
struct compute_new_exception_types {
    using type = type_list< /* template magic goes here */>;
};

template<typename T>
void algorithm(const T& obj)
    throw(typename compute_new_exception_types<
        declthrow((obj.foo(), obj.bar()))...>::type /* how to expand this to a pack here */)

```

While additional workarounds could be added to the `throw()` specification to make this work, I think doing this would needlessly complicate the design. I am hopeful that we can instead make progress on improving general pack-manipulation facilities to make some of these cases possible. See P1858R2, P2632R0.

5.4.6. Packs of `declthrow` packs

One common use-case of `declthrow` is to compute throw-specifications for other functions.

For example, say we have a user pass an invocable that we will call with elements of a span, the `throw()` specification might be defined as follows:

```

template<typename T, typename Func>
requires std::invocable<Func&, T&>
void for_each(std::span<T> values, Func&& func) throw(declthrow(func(std::declval<T&>())))

```

However, if we were to, say, try to do something similar with a `std::tuple`, where the function may be evaluated with multiple different argument types, each argument type represented by a pack element, then the throw-

specification effectively needs to become a concatenation of the `declthrow` packs, one pack for each element of the tuple.

Ideally we'd be able to write something like the following:

```
template<typename... Ts, typename Func>
    requires (std::invocable<Func&, Ts&> && ...)
void for_each(std::tuple<Ts...>& values, Func&& func) throw(declthrow(func(std::declval<
```

However, there are known issues with expanding a pack of packs (see P2632R0 - section "Single level of packness").

As a workaround, we could instead write this with a single `declthrow` expression that contains a compound expression using operator, `.`. For example:

```
template<typename... Ts, typename Func>
    requires (std::invocable<Func&, Ts&> && ...)
void for_each(std::tuple<Ts...>& values, Func&& func) throw(declthrow((func(std::declval<
```

This way the `Ts` pack is expanded inside the argument to `declthrow` and it is no longer problematic expanding the `declthrow` expression.

The other alternative for function templates / inline functions that wish to be transparent in the set of exceptions they may throw is to just use `throw(auto)` to deduce the throw-specification from the body, rather than having to duplicate the relevant parts of the body in the `throw()`-specification.

5.4.7. Availability of the `declthrow` keyword

A search of GitHub public repositories yielded no direct matches for the identifier `declthrow`, although it is worth noting that it did yield instances of a macro named `DECLTHROW(X)` which was used to conditionally define throw-specifications if available in the target C++ language/compiler.

A search of <https://codesearch.isocpp.org/> yielded no matches for `declthrow`.

5.4.8. Alternative Syntaxes Considered

Another alternative syntax considered was the reuse of the `throw` keyword in a `throw...(expr)` that would expand to the pack of types that could potentially be thrown by that expression.

However, this syntax would have a potential inconsistency with `sizeof...(pack)` which takes an unexpanded pack and returns a single value. Whereas `throw...(expr)` needs to take a single expression and produce a pack.

The `throw...(expr)` syntax may also be more easily confused with `throw (expr)` which throws an exception instead of querying what exception types it might throw.

The `declthrow` keyword also has the benefit of association/similarity with `decltype` which is used to query the value-type of an expression.

5.4.9. Filtering the set of exceptions

Sometimes we want to build a throw-specification that indicates that we throw any exception that some other expression throws, but that we handle some number of errors within the function and so we want to exclude those from the list. This way if the exception-specification of the other expression changes, then the expression-specification of our function changes to include the new set of exceptions.

While this could, in theory, be done with some template metaprogramming on packs, which would become possible with the introduction of more pack-manipulation facilities described in P2632R0, the resulting code is still onerous, and compile-time expensive compared to not filtering the exceptions.

For example: Using `throw(auto)` and P3115R0 generalized pack facilities, we can define a helper `filter_exceptions`

```
template<typename ErrorType>
[[noreturn]] _throws() throw(ErrorType);

template<typename HandledType, typename ErrorType>
void _handle() throw(auto) {
    if constexpr (not std::same_as<HandledType, std::any_exception>) {
        try { _throws<ErrorType>(); } catch(HandledType) {}
    }
}

// P3115R0 pack alias syntax
template<typename HandledType, typename... Errors>
using ...filter_exceptions = declthrow((_handle<HandledType, Errors>(), ...));
```

Which could then be used as follows:

```
void example() throw(filter_exceptions<CaughtException, declthrow(some_expression)...>...
```

One alternative would be to add a syntax that allowed the programmer to describe the intent to filter the exception list directly in the language.

A strawman syntax for this could be to allow additional arguments to `declthrow()` to list types to exclude from the list of types. i.e. `declthrow(expr, filter-clauses...)`

For example: We could add additional `catch(type)` arguments after the first argument to `declthrow()` to list exception types from the expression that are caught and thus should be removed from the list.

```
// Given.
struct A : std::exception {};
struct FooError : std::exception {};
struct B : FooError {};
struct C : FooError {};

void foo() throw(A, B, C);

void example1() throw(declthrow(foo())...); // -> throw(A, B, C)
void example2() throw(declthrow(foo(), catch(A))...); // -> throw(B, C)
void example3() throw(declthrow(foo(), catch(A), catch(B))...); // -> throw(C)
void example4() throw(declthrow(foo(), catch(FooError))...); // -> throw(A)
void example5() throw(declthrow(foo(), catch(std::exception))...); // -> throw()
```

Note that listing the `catch(FooError)` base class removes both derived types from the list.

Despite the potential syntactic and compile-time benefits that might arise from adding such a syntax, it's not clear whether the added complexity is worthwhile at this point. Usage experience is needed to better understand how often such a feature would be needed.

For a lot of these cases, it is expected that the `throw(auto)` syntax will serve most of the needs in this direction, and assuming that more generalised pack facilities become available, users that really need to do such filtering would still be able to do this in library.

If we can specify the syntax of `declthrow` such that it reserves the ability to be extended in some way such that this capability could be added later, then we can take a wait-and-see approach.

5.5. Checking the throw-specification of a function

A function declaration that includes a *static-exception-specification* must have a definition that ensures that only exceptions of those types may exit the function.

To assist with this, the compiler looks at the body of the function to compute the set of potentially-thrown exception types that may exit the body of the function.

If this set of possible exception types is not a subset of the set of exception types listed in the exception-specification then the program is ill-formed.

For example:

```
int other() throw(A);

// OK: set of potentially-thrown exceptions is {A, B}, all of which are
// listed in the function's throw-specification.
void example1() throw(A, B) {
    int x = other();
    if (x < 0) throw B{};
}

// Ill-formed: call to other() can potentially throw exception A
// which is not listed in example2()'s throw-specification.
void example2() throw(B) {
    int x = other();
    if (x < 0) throw B{};
}
```

Note that for functions with a throw-specifier of `throw(auto)` the check always passes as the compiler computes the exception-specification to be exactly the set of potentially-thrown exception types and thus every exception type is, by-definition, listed in the exception-specification.

For functions with a throw-specifier of `throw(...)` or `noexcept(false)`, the function is permitted to throw an exception of any type and so this check is not required to be performed.

5.6. Computing the set of potentially-thrown exception types

The ability to check the throw-specifier of a function, compute the results of a `declthrow` query, instantiate a template-handler with the appropriate types, or deduce the set of exception types that may be thrown from a function with a `throw(auto)` throw-specification all depend on the ability to compute the set of potentially-thrown exception types for expressions and statements.

When computing the set of exception types that might exit an expression, statement or function, we ideally want a set of rules that can be reliably evaluated in a consistent way across all conforming implementations, and that is

not dependent on inlining, or compiler optimisations. This is because the computation can be important for correctness and well-formedness of a program, and can also affect the ABI of functions with deduced throw specifications.

Computing the set of potentially thrown exception types, therefore, needs to be computable locally for each function, from looking only at the function body and the signatures of any functions called from that function, since we cannot assume that the definitions of called functions will be available.

The following sections describe such a set of rules for computing the *set of potentially-thrown exception types* for each grammar term that may appear within a *function-body*.

The descriptions here are not as precise as they would need to be for wording, but are hopefully descriptive enough to understand the proposed semantics.

5.6.1. Statement Reachability

When computing the set of exceptions that may be thrown from some constructs, there are cases where we need to determine whether execution can potentially flow off the end of a *compound-statement* as these can affect the set of exceptions that can potentially be thrown.

For example:

- If execution flows off the end of a coroutine, it implicitly evaluates `co_return;`. This calls `promise.return_void()` which may be potentially-throwing.
- If execution flows off the end of a handler of a *function-try-block* for a constructor or destructor then the exception is implicitly rethrown as if there was a `throw;` statement inserted at the end of the handler's *compound-statement*.

Therefore, we need to first define some rules around defining the reachability of certain statements. These rules will need to be somewhat conservative as computing an accurate sense of reachability is equivalent to solving the halting problem, and thus intractable.

The rules below use the terminology *potentially reachable statement* to indicate that the computation is conservative.

A *compound-statement* evaluates a sequence of *statements*. There are some statements/expressions for which it is never possible to execute the next statement, however, as they unconditionally divert control-flow elsewhere.

1. Interrupted-flow statements

An *interrupted-flow statement* is a statement for which execution cannot flow to the next statement from this statement.

The following statements are *interrupted-flow statements*:

- A *jump-statement* - i.e. `break;`, `continue;`, `goto;`, `return expr-or-braced-init-list[opt];` or *coroutine-return-statement*.
- A *compound-statement* where execution cannot flow off the end of the block (see below)
- An if or if-else *selection-statement* where either;
 - the *init-statement*, if any, is an *interrupted-flow statement*; or
 - the *condition* is an *interrupted-flow expression*.
- An if-else *selection-statement* where the first and second sub-statements are both *interrupted-flow statements*. Note: this includes `if constexpr` selection-statements.
- A *constexpr if* or if-else *selection-statement* where the condition evaluated to true and the first sub-statement is an *interrupted-flow statement*.

- A *constexpr if-else selection-statement* where the condition evaluated to false and the second sub-statement is an *interrupted-flow statement*.
- A *try-block* where the *compound-statement* is an *interrupted-flow statement* and the *compound-statement* of every reachable handler (see section on *try-block* below) of the *try-block's handler-seq* is an *interrupted-flow statement*.
- A *switch selection-statement* where either;
 - the *init-statement*, if any, is an *interrupted-flow statement*; or
 - the *condition* is an *interrupted-flow expression*; or
 - all of the following are true;
 - the *body statement* is an *interrupted-flow statement*; and
 - the *body statement* has a `default:` label associated with the switch; and
 - there is no potentially-reachable `break`; statement associated with the switch.
- A *do-while iteration-statement* where both the following are true;
 - the loop *body statement* does not enclose any potentially-reachable `break`; statements associated with the loop; and
 - either;
 - both of the following are true;
 - the loop *body statement* does not enclose any potentially-reachable `continue`; statements associated with the loop; and
 - the loop *body statement* is an *interrupted-flow statement*; or
 - the loop *expression* is an *interrupted-flow expression*
- A *for or while iteration-statement* where either;
 - the *init-statement*, if present, is an *interrupted-flow-statement*; or
 - the *condition* expression is an *interrupted-flow expression*;
- A *range-based for iteration-statement* where either;
 - the *init-statement* is an *interrupted-flow statement*; or
 - the *for-range-initializer* expression is an *interrupted-flow expression*; or
 - the *begin-expr* is an *interrupted-flow expression*; or
 - the *end-expr* is an *interrupted-flow expression*.
- An *expression-statement* where the expression is an *interrupted-flow expression*.
- A *declaration-statement* that is an object declaration where the initializer expression is an *interrupted-flow expression*.

2. Interrupted-flow expressions

An *interrupted-flow-expression* is a potentially evaluated expression that is one of the following:

- A *throw-expression*
- A *postfix-expression* that evaluates a call to a function marked `[[noreturn]]`.
- A *conditional-expression* (ternary `?:` operator) where either;
 - the first sub-expression is an *interrupted-flow-expression*; or
 - the second and third sub-expressions are both *interrupted-flow-expressions*.
- A built-in logical AND or logical OR expression where the first sub-expression is an *interrupted-flow expression*.
- A *prvalue* expression of class type whose destructor is marked `[[noreturn]]`.
- Any other compound expression that has an immediate sub-expression that is an *interrupted-flow-expression*.

3. Potentially-reachable statements

A *potentially-reachable statement* is a statement of a function that the compiler determines can potentially be executed based on a local analysis of the control-flow of the function. It does not consider the values of any expressions which are semantically computed at runtime.

1. Reachability of compound-statements

A sub-statement of a *compound-statement* is a *potentially-reachable statement* if:

- it is the first sub-statement of the *compound-statement* and the *compound-statement* is reachable; or
- the immediately preceding statement is a *potentially-reachable statement* and was not an *interrupted-flow-statement*; or
- the statement was immediately preceded by a label (Note: this does not include the implicit labels mentioned in the definition of a while statement)

Otherwise a sub-statement of a *compound-statement* is considered an *unreachable-statement*.

A *compound-statement* that is the top-level *compound-statement* of a function body or lambda body is a *potentially reachable statement*.

2. Reachability of components of an if-statement

In an if-statement of the form `if ( /condition/ ) /statement/` or `if ( /init-statement/ /condition/ ) /statement/` with or without the `else /statement/ then;`

- The *init-statement*, if present, is a potentially-reachable statement if the if-statement is a potentially-reachable statement.
- The *condition* expression is a potentially reachable statement if;
 - The if-statement is potentially reachable; and
 - The *init-statement* is either not present, or if present, is not an *interrupted-flow statement*.
- The first and second (if present) *statement* is a potentially reachable statement if the *condition* expression is a potentially-reachable expression and the *condition* expression is not an *interrupted-flow expression*.

In a `constexpr if` statement;

- the first substatement is potentially reachable if and only if the if-statement is potentially reachable and the *condition* evaluates to true;
- the second substatement, if present, is potentially reachable if and only if the if-statement is potentially reachable and the *condition* evaluates to false.

3. Reachability of components of a switch statement

In a switch-statement of the form `switch ( /condition/ ) /statement/:`

- the *condition* expression is potentially reachable if the switch-statement is potentially reachable

And, in a switch-statement of the form `switch ( /init-statement/ /condition/ ) /statement/:`

- the *init-statement* is potentially reachable if the switch-statement is potentially reachable
- the *condition* expression is potentially reachable if the switch-statement is potentially reachable; and the *init-statement* was not an interrupted-flow statement.

In both cases, the *statement* is not potentially-reachable. Execution can only enter *statement* via a jump to a label enclosed by *statement*.

Any case and default labels associated with the switch statement are potentially reachable if and only if the *condition* expression is potentially reachable and is not an *interrupted-flow expression*.

For example:

```
void f(int x) {
    switch (x) {
        a; // not-reachable
    case 0:
        b; // reachable - appears after a label
        break;
```

```

    c; // not reachable – appears after a jump-statement
default:
    d; // reachable – appears after a label
}
}

```

4. Reachability of components of an iteration-statement

In an iteration-statement of the form `while ( /condition/ ) /statement/`

- The *condition* is a potentially reachable expression if the while-statement is a potentially reachable statement
- The *statement* is a potentially reachable statement if the *condition* expression is potentially reachable and the *condition* expression is not a *flow-interrupted expression*.

In an iteration-statement of the form `do /statement/ while ( /expression/ ) ;`

- The *statement* is potentially reachable statement if the do-statement is potentially reachable
- The *expression* is a potentially reachable expression if do-statement is potentially reachable and either;
 - the *statement* is not a *flow-interrupted statement*; or
 - the *statement* encloses a potentially reachable `continue`; statement associated with the do-statement

In an iteration-statement of the form `for ( /init-statement/ /condition/ ; /expression/ ) /statement/`

- The *init-statement* is potentially-reachable if the for-statement is potentially-reachable
- The *condition* expression (if present) is potentially-reachable if the for-statement is potentially-reachable and the *init-statement* is not an *interrupted-flow statement*
- The *statement* is a potentially-reachable statement if the *init-statement* is a potentially-reachable statement and is not an *interrupted-flow statement* and either the *condition* expression is not present or the *condition* expression is not an *interrupted-flow expression*.
- The *expression* is a potentially-reachable expression if either;
 - The *statement* is a potentially-reachable statement and is not an *interrupted-flow statement*; or
 - There is a potentially-reachable `continue`; statement enclosed by *statement* that is associated with the for-loop.

5. Reachability of identifier labels

These rules treat all identifier labels as potentially-reachable and does not do any analysis to determine whether there is any jump-statement that could potentially jump to that label.

For example, the rules could potentially require looking elsewhere in the function to determine whether there are any `goto` statements that target a particular label.

However, requiring this prevents doing analysis of reachability in a single pass as you may need to look later in the function in order find a `goto` statement that targets a label earlier in the function.

For example: When the compiler reaches the `retry: label` it has not yet seen the `goto retry;` statement and so does not yet know whether `retry: label` is reachable.

```

int foo(int x) {
{
    auto result = try_fast(x);
    if (!result) {
        goto slow;
    }
}
}

```

```

    return result.value();
}

retry:
    reset_slow();

slow:
    auto result = try_slow(x);
    if (!result) {
        goto retry;    // only know that 'retry:' label is reachable after processing
    }

    return result.value();
}

```

An even more sophisticated approach could consider the potential reachability of the goto statement targeting a label itself.

Th
fui

For example: In the following function there is a goto statement targeting each of the labels in this function, but none of those goto statements are themselves reachable from the function entry-point.

```

void foo(int x) {
    if constexpr (false) {
        goto foo;
    }

    return x;

foo:
    if (x < 0)
        throw negative_error{};
    goto baz;

bar:
    --x;
    goto foo;

baz:
    goto bar;
}

```

It is not difficult to imagine such code occurring in practice in function templates where there are goto statements in if constexpr branches that are either discarded or not discarded, depending on the types the function template was instantiated with.

The rules could potentially be extended to consider a label as potentially reachable only if there is a potentially reachable goto statement that targets the label.

Computing the reachability in this case would basically require the compiler to hold the control-flow graph of the entire function in memory and then walk that graph, marking statements as reachable or not. This may be incompatible with the architecture of some compiler implementations.

The proposed design chooses a more conservative algorithm that treats all labels as reachable in order to permit implementations that can compute a more conservative concept of reachability in a single

pass.

It is not clear whether or not handling such cases in a more accurate way would be worth the additional complexity it would place on implementations.

4. Flowing off the end of a compound-statement

Execution may flow off the end of a *compound-statement* if either;

- the *compound-statement* is a *potentially-reachable statement* and has an empty sequence of sub-statements; or
- both;
 - the last sub-statement of the *compound-statement* is potentially-reachable and is not an *interrupted-flow-statement* (Note: This includes any null sub-statement implicitly inserted after a trailing label immediately before the closing brace); and
 - There are no object declarations declared in the scope of the *compound-statement* that have destructors that have the attribute `[[noreturn]]`.

5. Flowing off the end of a switch statement

The rules for determining that a switch statement is an *interrupted-flow statement* require that the body of the switch statement has a `default:` label associated with the switch.

This approach is somewhat conservative, as it may be possible that all of the potential cases are already covered by case labels and that, therefore, it is not possible for the switch statement to jump over the statement body and flow onto the next statement.

For example: The rules above result in the following

```
void example(bool x) {
    // Not an interrupted-flow statement - no default: case
    switch (x) {
        case true: throw X{};
        case false: throw Y{};
    }
    // The following statement is considered potentially-reachable.

    // An interrupted-flow statement - has a default: case
    switch (x) {
        case true: throw X{};
        default: throw Y{};
    }

    // Not potentially-reachable.
    // Prior statement is an interrupted-flow statement.
    // Control cannot flow off the end of the function's compound-statement.
}
```

The rationale here is that trying to determine whether every possible value for the switch expression is covered by a case label is non-trivial and/or probably doesn't do what you want.

For example: Consider switching on an enum where all enum members have case labels.

```
enum class state_t { stopped = 0, starting = 1, running = 2 };

int example(state_t state) {
    switch (state) {
        case state_t::stopped: return 0;
        case state_t::starting: return 1;
        case state_t::running: return 2;
    }
}
```



```

    }

    foo(); // should this statement be considered "potentially-reachable"?
}

// Consider the following call.
example(static_cast<state_t>(3));

```

If, instead, we just look for a default: label then we know that every possible case is handled.

If we are willing to define rules for determining whether all possible cases are listed as case labels then we could potentially relax the rule requiring the use of a default: label here.

6. Use of `[[noreturn]]` for normative semantics

The rules above treat calls to functions marked as `[[noreturn]]` as being interrupted-flow expressions and the interpretation as such can potentially affect the computation of the set of potentially-thrown exceptions, which in turn can affect the semantics and well-formedness of a program.

The use of an attribute in this way is novel and would no longer have optional semantics, which would go against the intent of the following note in `[dcl.attr.grammar]` p6

[Note : The attributes specified in `[dcl.attr]` have optional semantics: given a well-formed program, removing all instances of any one of those attributes results in a program whose set of possible executions (`[intro.abstract]`) for a given input is a subset of those of the original program for the same input, absent implementation-defined guarantees with respect to that attribute. — end note]

The statement reachability computation depends on the ability to determine whether a function can return normally and flow to the next statement or not. For example, programs may insert calls to `std::terminate()` or `std::unreachable()` before the end of a compound-statement to indicate that control should not flow off the end (e.g. after a loop that is never expected to exit except by return).

If we do not wish to give the `[[noreturn]]` attribute normative semantics, then perhaps we should explore defining an alternative normative mechanism for annotating functions as never returning normally.

5.6.2. *function-body*

The computation of the set of exception types of a *function-body* is used for two main purposes:

- checking that exception types that can potentially exit the function are listed in a function's *throw-specifier*.
- deducing the throw-specification for a function with a `throw(auto)` specifier.

The following steps are used to compute the set of potentially-thrown exception types for a function body.

Let A be the set of potentially-thrown exception types for the function body's *compound-statement*.

If the function is a coroutine and `return_void` is found in the scope of the coroutine's `promise_type` then flowing off the end of the coroutine is equivalent to evaluating `co_return;`. If this implicit `co_return;` statement is potentially reachable (see above definition), then the computation of A takes into account any potentially-thrown exceptions that may result from the evaluation of the `co_return;` statement.

If the function is a constructor, then

- Let B be the set of potentially-thrown exception types of the function call expressions of the constructors of the base-classes and non-static data-members.

Otherwise, if the function is a destructor, then

- Let B be the set of potentially-thrown exception types of the function call expressions of the destructors of the base-classes and non-static data-members.

Otherwise,

- Let B be the empty set.

Let C be the union of the sets A and B .

If the *function-body* has a *function-try-block*, then;

- let D be the subset of types in C that would be caught by the handlers of the *function-try-block*. (see the *try-block* description for more details about this); and
- for each potentially reachable handler, H_i , of the try-block, let E_i be the set of potentially-thrown exception types corresponding to the *compound-statement* of that handler. For the purposes of computing the set of potentially-thrown exception types, if the function-body is of a constructor or destructor then the *compound-statement* of H_i should be considered to have an implicit `throw;` statement inserted immediately prior to the closing brace. Note: This implicit `throw;` statement may or may not be potentially-reachable and therefore may or may not contribute to the set of potentially-thrown exception types computed for E_i .
- Let E be the union of the sets E_i .

Otherwise, let D and E both be the empty set.

Then the set of potentially-thrown exception types of the *function-body* is the set of types described by $(C - D) \cup E$.

5.6.3. *statement*

A statement is one of the following cases:

- *expression-statement*
- *compound-statement*
- *selection-statement*
- *iteration-statement*
- *jump-statement*
- *declaration-statement*
- *try-block*

See the relevant section for a description of each.

5.6.4. *expression-statement*

An *expression-statement* has a set of potentially-thrown exception types equal to the set of potentially-thrown exception types of the *expression*.

See section on *expression* handling below.

5.6.5. *compound-statement*

The set of potentially thrown exception types for a *compound-statement* is the union of the set of potentially-thrown exception types for each of the *potentially-reachable statements* in the *statement-seq* of the *compound-statement*.

Note that this takes into account some basic control-flow analysis to eliminate potentially-thrown exceptions from statements in the *statement-seq* that are determined to be unreachable. e.g. ignoring a statement because a preceding statement branched unconditionally to some other code-path via `return`, `break`, `continue`, `goto`, `throw` or calling a `[[noreturn]]` function.

For example: Assuming the following declarations:

```
void foo() throw(A);
void bar() throw(B, C);
void baz() throw(D);
```

The set of potentially-thrown-exceptions from the following compound-statement is { A, D }

```
{
    foo(); // might throw A
    goto label;
    bar(); // might throw B or C (note this is an unreachable-statement)
label:
    baz(); // might throw D
}
```

5.6.6. *selection-statement*

Selection statements include `if`, `if constexpr`, `if constexpr` and `switch` statements.

1. `if` statements

The set of potentially-thrown exception types of an `if` statement is the union of the potentially-thrown exception types of the:

- *init-statement* - if present
- *condition* expression
- *statement* - the first substatement
- *statement* - the second substatement (if the `else` part is present)

Note that the computation of potentially-thrown exception types does not consider whether or not the *condition* is a constant expression or not - both branches of sub-statements are always considered when computing the set of potentially-thrown exception-types.

For example: The following `if`-statement has a set of potentially-thrown exception types equal to { X }, despite the condition being a constant

```
if (false) {
    throw X{};
}
```

If you want to force the branching decision to be performed at compile-time then use the `if constexpr` form of selection-statement (see below).

1. Reachability of `if`-statement components

With `if`-statements there is the question of whether we should consider the set of potentially-thrown exceptions of the first or second sub-statements if either the *init-statement* is a *interrupted-flow statement*, or if the *condition* is an *interrupted-flow expression*.

For example: Should the following function, `f()` deduce to a throw-specification of `throw(X)` or to `throw(X, Y, Z)`?

```
[[noreturn]] bool throws_something() throw(X);

void f() throw(auto) {
    if (throws_something()) {
        throw Y{};
    } else {
        throw Z{};
    }
}
```

The rules above do not try to compute the individual reachability of the substatements and applying the rules as written would result in a deduced exception specification for `f()` of `throw(X, Y, Z)`.

While it would be relatively straight-forward to extend the rules to, instead, compute a deduced exception specification of `throw(X)`, it is not clear that this would bring significant value, as this kind of code is expected to be relatively rare, and could be straight-forwardly rewritten in a form that separates the *interrupted-flow expression* into a separate statement.

For example: The following code is equivalent but deduces the throw-specification to `throw(X)` according to the above rules since the if-statement is not a reachable statement.

```
void f() throw(auto) {
    bool cond = throws_something();
    // The if-statement is unreachable as prior statement was an interrupted-flow
    if (cond) {
        throw Y{};
    } else {
        throw Z{};
    }
}
```

2. if constexpr statements

As the condition of a `constexpr if` statement is evaluated as part of constant evaluation and a constant evaluation is not permitted to result in a thrown exception, the *condition* part does not contribute to the set of potentially-thrown exception types.

If the *selection-statement* contains an *init-statement* part, then let *I* be the set of potentially-thrown exception types of the *init-statement*, otherwise let *I* be the empty set.

If the value of the converted *condition* expression is `true` then the set of potentially-thrown exception types of the *selection-statement* is the union of *I* and the set of potentially-thrown exception types of the first substatement. i.e. the body of the *if statement*.

Otherwise, if the `else` part of the selection statement is present, then the set of potentially-thrown exception types of the *selection-statement* is the union of *I* and the set of potentially-thrown exception types of the second substatement. i.e. the body of the *else statement*.

Otherwise, the set of potentially-thrown exception types of the *selection-statement* is *I*.

For example: The following statement has a set of potentially-thrown exceptions equal to `{ X }`.

```

if constexpr (true) {
    throw X{};
} else {
    throw Y{};
}

```

3. if constexpr

An if-statement of the form `if constexpr /compound-statement/` has a set of potentially-thrown exception types that is the set of potentially-thrown exception types of the *compound-statement*.

An if-statement of the form `if constexpr /compound-statement/ else /statement/` has a set of potentially-thrown exception types that is equal to the union of the sets of potentially thrown exception types of *compound-statement* and *statement*, respectively.

Note that the *compound-statement* is manifestly constant-evaluated and so is not currently permitted to throw exceptions and so could potentially be considered as not contributing to the set of potentially-thrown exception types.

However, it is possible that we may want to support the ability to throw exceptions during constant-evaluation in future. See [P3068R0](#) "Allowing exception throwing in constant-evaluation" for such a proposal.

If we were to initially treat code within the manifestly-constant-evaluated branch as non-throwing then later changing it to be potentially-throwing would be a breaking change.

4. switch

A switch statement of the form `switch ( /init-statement/ /condition/ ) /statement/` or `switch ( /condition/ ) /statement/` has a set of potentially thrown exception-types equal to union of the sets of potentially thrown exception types of the following parts:

- *init-statement* (if present)
- *condition* expression
- *statement*

Note that similarly to the if-statement we could modify this to consider the *condition* only if it is a potentially-reachable expression (i.e. the *init-statement* is absent or is not a flow-interrupted statement).

Further, if the *condition* is either not reachable or is a flow-interrupted expression then we could also consider any case or default labels associated with the switch-statement as unreachable labels.

The *statement* of a switch is not itself a potentially-reachable statement, although it may contain potentially-reachable sub-statements if there are labelled sub-statements. The set of potentially-thrown exception types of the *statement* only considers the potentially-reachable sub-statements.

5.6.7. iteration-statement

The following kinds of iteration-statement are possible:

- `while ( /condition/ ) /statement/`
- `do /statement/ while ( /expression/ ) ;`
- `for ( /init-statement/ /condition/ ; /expression/ ) /statement/`
- `for ( /init-statement/ /for-range-declaration/ : /for-range-initializer/ ) /statement/`

For all of these forms of iteration statement, the set of potentially-thrown exception types of the iteration statement is the union of the sets of potentially-thrown exception types from each of the relevant subexpressions or substatements:

- *condition*
- *statement*
- *expression*
- *init-statement*
- *for-range-declaration*
- *for-range-initializer*

Note that this includes exceptions that may be thrown from the body of the iteration statement, even if the statement's *condition* is such that the body will never be executed.

Note that if [P2809R2] "Trivial infinite loops are not Undefined Behavior" is adopted, then trivial infinite loops could potentially be required to be implemented as a call to the proposed function `std::this_thread::yield_forever()`, which is marked `[[noreturn]]`. This means that such a trivial infinite loop would be considered a flow-interrupted statement. If P2809 is adopted after this paper then this could potentially be a breaking change as it could change the deduced set of potentially-thrown exception types of a function with a `throw(auto)` specifier.

For example: Without P2809 the following function would deduce its exception specification to `throw(X)` but with P2809 it would deduce to `throw().\`

```
void example() throw(auto) {  
    while (true) ;  
    throw X{};  
}
```

5.6.8. *jump-statement*

Jump statements include:

- `break;`
- `continue;`
- `return expr-or-braced-init-list ;`
- *coroutine-return-statement*
- `goto identifier ;`

Only the `return` and `co_return` statements can potentially affect the set of potentially-thrown exception types here. The others are pure control flow, and while they can potentially trigger exceptions to be thrown when exiting scopes via that control-flow (if destructors are potentially-throwing), those exceptions should be covered by the declaration statement for that variable.

1. `return` statements

A `return` statement has a set of potentially-thrown exception types equal to the union of the set of potentially-thrown exception types of the operand expression, and the set of potentially-thrown exception types from any implicit conversion or constructor call required to initialize the return-value.

Note that there is an edge-case here that needs to be considered, where the operand to the `return` statement is a prvalue which is returned with guaranteed copy-elision and where the object has a potentially-throwing destructor. Normally, an expression that creates a pr-value includes the potentially-throwing types of both the call to the constructor, and the call to the destructor, as a statement containing that expression will also call the destructor at the end of the full-expression. However, for a return-value that is initialized with

guaranteed copy-elision, the destructor will be invoked by the caller of the function, rather than the local function, and so the exception-specification of the return-value type should not be included in the calculation of the set of potentially-thrown exception types for the return statement.

For example:

```
struct Foo {
    Foo() throw(A);
    Foo(Foo&&) throw(B);
    ~Foo() throw(C);
};

Foo f() throw(auto) { // deduces to throw(A)
    return Foo{}; // constructor called here but not destructor
}

Foo g() throw(auto) { // deduces to throw(A, B, C)
    Foo f; // declaration statement potentially throws A (from constructor) and C (from destructor)
    return f; // move-constructor potentially called here, even if copy is elided due to guaranteed copy-elision
}
```

We also need to consider the case where the returned object is initialized using aggregate initialization, where there may be a whole tree of sub-expressions that potentially initialize sub-objects of the returned object.

Any expression in such a return-statement that directly initializes an object or sub-object of the return-value that will be destroyed by the caller should not consider the exception-specification of that sub-object's type's destructor when computing the set of potentially-thrown exceptions of the return statement.

Note that, while it may be possible that the return-statement may execute the destructor of some of these sub-objects in the case that initialization of a subsequent sub-object exits with an exception, we do not need to consider this case for the purposes of computing the potentially-thrown exceptions as these destructors will only be called in case there is an unwind due to another exception - if these destructors then throw their own exceptions during unwind then this results in an immediate call to `std::terminate`.

For example:

```
struct X {
    X() throw(A);
    ~X() throw(B);
};

struct Y {
    Y() throw(C);
    ~Y() throw(D);
};

struct Z {
    X x;
    Y y;
};

Z h() throw(auto) { // deduces to throw(A, C)
    return Z{X{}, Y{}};
}
```

In this example, if `Z::x` is initialized and then the call to `Z::y`'s constructor throws then the `Z::x` destructor will be called, which could theoretically exit with an exception of type, B. However, since this can only

happen while unwinding with an exception of type C, if $Z : x. \sim X()$ exits with an exception during unwind then `std::terminate()` is called. So it is not possible for the function `h()` to exit with an exception of type B.

As an aside, it would be useful to be able to make ill-formed any cases that could result in potential calls to `std::terminate()` due to an exception being thrown during unwind - at least in pursuit of the goal of embedded systems having no hidden calls to `std::terminate()`. How, and whether, to do this is an open-question.

2. `co_return` statements

A `co_return` statement of the form `co_return;` has a set of potentially-thrown exception types equal to the set of potentially-thrown exception types of the statement `promise.return_void()`;, where *promise* is the current coroutine's promise object.

A `co_return` statement of the form `co_return /expr/ ;`, where *expr* has type `void` has a set of potentially-thrown exception types equal to the union of the set of potentially-thrown exception types of *expr* and the set of potentially-thrown exception types of the statement `promise.return_void()`, where *promise* is the current coroutine's promise object.

A `co_return` statement of the form `co_return /expr-or-braced-init-list/ ;`, where the operand is either an *expression* of non-void type or is a *braced-init-list*, has a set of potentially-thrown exception types equal to the set of potentially thrown exception types of the statement `promise.return_value( /expr-or-braced-init-list/ );`.

5.6.9. *declaration-statement*

A declaration statement consists of a *block-declaration*, which in turn consists of one of the following:

- *simple-declaration*
- *asm-declaration*
- *namespace-alias-definition*
- *using-declaration*
- *using-enum-declaration*
- *using-directive*
- *static_{assert}-declaration*
- *alias-declaration*
- *opaque-enum-declaration*

Other than the first two cases, the rest of the declarations do not introduce any executable code that might throw exceptions.

Note that, while the *static_{assert}-declaration* has a child expression, this expression is evaluated as a manifestly constant expression, and therefore the program is ill-formed if that expression exits with an exception.

1. *simple-declaration*

A declaration statement that declares one or more block variables with automatic storage duration has a set of potentially-thrown exception types equal to the union of the sets of potentially thrown exception types of any *initializer* expressions and any calls to constructors or conversion operators required to initialize the local variables, and any potentially thrown exception types of calls to the destructors of the declared variables.

A declaration statement that declares one or more block variables with either static storage duration or thread storage duration has a set of potentially-thrown exception types equal to the union of the sets of potentially-thrown exception types of any *initializer* expressions and any calls to constructors of conversion

operators required to initialize the local variables, but does not include exception types of calls to the destructors of the declared variables.

We do not include the exceptions thrown by destructors of static/thread_{local} variables because these destructors are not called from within the scope of a function that initializes these variables.

Q. Initialization of variables with static storage duration needs to perform synchronization in multi-threaded environments in order to guard against data-races initializing the variable. Is it possible that operations on the synchronization primitives might fail with an implementation-defined or unspecified exception (in which case we would need to include this in the set of exception types) or can we assume that the synchronization will always succeed?

A declaration statement that declares a constant expression (i.e. is declared with the `constexpr` specifier) has an empty set of potentially-thrown exception types, assuming that the program is ill-formed if a constant-evaluation exits with an exception.

2. *asm-declaration*

The *asm-declaration* has implementation defined behaviour and, while in theory, on some implementations, an assembly declaration might be able to throw an exception, we cannot, in general, deduce anything about the set of exception types that might be thrown in a portable way.

There are three possible options we take here:

- the *asm-declaration* has implementation-defined behaviour, so the set of potentially-thrown exceptions from such a declaration should be implementation-defined.
- the *asm-declaration* could potentially do anything (invoke a potentially-throwing function, implement some exception-throwing mechanics, etc.) so we should treat this as potentially throwing any type of exception.
- the vast majority of *asm-declaration* usage is for implementing optimized, inline code, which won't throw any exceptions, so we could define it to be non-throwing.
- we could make it ill-formed to use an *asm-declaration* in any context in which the exception-specification needs to be deduced.

This paper suggests treating an *asm-declaration* statement as having an implementation-defined set of potentially-thrown exceptions as this at least allows the possibility of the implementation being able to analyse a declaration and deduce what exceptions might be thrown in an implementation-specific way.

However, it would be worth a more detailed discussion within the Evolution sub-group about what the desired semantics are here.

5.6.10. *try-block*

A *try-block* statement has the form `try /compound-statement/` followed by one or more handlers of the form `catch ( /exception-declaration/ ) /compound-statement/`.

In addition, a *function-try-block* statement for a constructor can also have the form `try /ctor-initializer/ /compound-statement/` followed by one or more handlers of the form `catch ( /exception-declaration/ ) /compound-statement/`.

Let A be the set of potentially-thrown exception types of the try block's *compound-statement*.

If this is a function try block for a constructor then let B be the union of A and the sets of potentially-thrown exceptions of initializer expressions and calls to constructors of any base classes and non-static data-members.

Otherwise, let B be the set A .

Let E initially be the empty set.

For each type, x , in the set B

- if x is `std::any_exception` then
 - add the set of potentially-thrown exception types of the *compound-statement* of every reachable handler to E .
 - NOTE: a handler is not reachable if the *exception-declaration* for that handler names a type unambiguously derived from a base class type that is listed in an earlier handler's *exception-declaration*.
 - NOTE: this does not include template handlers as those are only invoked from exceptions thrown from expressions that have a static exception specification.
 - if the list of handlers associated with this try-block does not include a handler with an *exception-declaration* of `...` then add `std::any_exception` to E .
- otherwise, if any handler is a match for an exception object of type x then
 - The selected handler for this exception type is the first handler, h , that matches an exception object of type x . h is a potentially-reachable handler.
 - Add the set of potentially-thrown exception-types of h 's *compound-statement* to E .
 - Add the set of potentially-thrown exception-types of a function call expression that invokes the destructor of an object of type, x .
 - NOTE: This is because exiting the handler's *compound-statement* will potentially call the destructor of the exception object of type x .
 - NOTE: There is a potential edge-case here, where the *compound-statement* unconditionally rethrows the exception (e.g. with a `throw;` statement). In this case, the destructor of the exception object is never called upon exiting the *compound-statement* and so we could avoid adding the destructor's set of potentially-thrown exception types to the set of potentially-thrown exception types for this handler. This is unlikely to make much difference in practice, as most exception types used in practice have a non-throwing destructor, but still needs to be defined.
- otherwise,
 - add x to E

If the *try-block* is part of a *function-try-block* of a constructor or destructor then a handler associated with that *try-block* should be treated as if the statement `throw;` was inserted immediately prior to the closing brace of its *compound-statement* for the purposes of computing its set of potentially-thrown exception types.

The set of potentially-thrown exception types of the *try-block* is the resulting set, E .

5.6.11. *init-statement*

An *init-statement* is either a:

- *simple-declaration*
- *expression-statement* - Expression statements are already described above.
- *alias-declaration* - These do not contain any executable code and thus do not contribute to the set of potentially-thrown exception types.

If the statement is a *simple-declaration* that is an object declaration, the set of potentially-thrown exception types of that statement is the union of the sets of potentially-thrown exception types of the *initializer* expression, the function call expression of the objects selected constructor and the function call expression of the object's destructor.

5.6.12. *expression*

There are many different types of expressions that need to be considered.

This paper does not attempt to list all of them here, but instead gives some general rules that apply to most expressions and then describe separately the rules for any expressions that do not follow the general rules.

1. General rules for expressions

Expressions that have sub-expressions in general have a set of potentially-thrown exception types that includes the union of the sets of potentially-thrown exception types of the immediate-sub-expressions of that expression.

Operator expressions or conversions that resolve to calls to user-defined operator functions have a set of potentially-thrown exception types of a function call expression to that user-defined operator function. Built-in implicit conversions and operators generally have an empty set of potentially-thrown exceptions.

2. Function call expressions

Function call expressions have a set of potentially-thrown exception types equal to the union of the sets of potentially-thrown exception types of the following expressions:

- the expressions provided as arguments to the function, including any default argument expressions.
- any implicit conversion expression required to convert the argument to the corresponding parameter type, and
- the *postfix-expression* immediately preceding the parenthesised argument-list

unioned with the set of exception types listed in the *postfix-expression*'s function type's or function-pointer type's exception-specification.

If the *postfix-expression* has function type then the set of potentially-thrown exception types is taken from the exception-specification of the function declaration.

If the *postfix-expression* has function-pointer or member-function-pointer type then the set of potentially-thrown exception types is taken from the exception-specification of the pointed-to function type. Note that this may be a superset of the set of exception types listed in the pointed-to function's exception specification.

If the function or function-pointer has an dynamic exception-specification then the set of potentially-thrown exception types is the set { `std::any_exception` }, otherwise, if it has a static exception-specification then the set of potentially thrown exception types is the set of types listed in the exception specification.

Note that if any parameter types of the function or function-pointer being called have a value category of prvalue then the set of potentially-thrown exception types will also include the exception types listed in the type's destructor's exception-specification, as per the next section.

3. prvalue expressions

If an expression has a prvalue value category then the set of potentially thrown exception types of that expression includes the set of potentially thrown exception types of a function call expression to that object's destructor, unless that expression is the operand of a return statement of a function returning a prvalue of the same type as the operand and the operand is used to initialize the result object via copy-initialization.

See the section on return statements for more details (in particular regarding aggregate initialization of return values).

4. Standard conversions

The set of potentially-thrown exceptions from all standard conversions listed under [conv.general] is the empty set.

5. Constant expressions

In C++23 it is not permitted for an exception to be thrown within the evaluation of a constant expression. If this is to remain the case, then we could assume that any manifestly constant expression has an empty set of potentially-thrown exceptions.

However, it's possible that in the future we may decide to allow exceptions to be thrown during the evaluation of a constant expression. Although this may be limited to the cases where any thrown exception is caught within the same constant evaluation - similar to how dynamic memory allocation is allowed, as long as the memory is freed within the same constant evaluation. The paper [P3068](#) "Allowing exception throwing in constant-evaluation" proposes such a change to the language.

In order to allow for this possibility, and to avoid having adding this capability later be a breaking change, manifestly constant expressions should be treated the same as runtime-evaluated expressions for the purposes of computing the set of potentially-thrown exception types of that expression.

However, the top-level expression that begins a new constant evaluation, such as initialization of a `constexpr` variable or the *condition* of an `if constexpr` statement, should be assumed to be non-throwing. An exception propagating out of such an expression would make the program ill-formed.

For example:

```
constexpr int parse_integer(const char* s) throw(parse_error) {
    int result = 0;
    do {
        if (!std::isdigit(*s)) throw parse_error{};
        result = 10 * result + (*s - '0');
        ++s;
    } while (*s != '\0');
    return result;
}

int example_1() throw(auto) { // deduces to throw()
    constexpr int i = parse_integer("1234");
    return i;
}

int example_2() throw(auto) { // deduces to throw(parse_error)
    const int i = parse_integer("1234");
    return i;
}

int example_3() throw(auto) { // deduces to throw()
    if constexpr (parse_integer("1234") >= 10) {
        return 10;
    }
    return 0;
}

constexpr int example_4() throw(auto) { // deduces to throw(parse_error)
    if consteval {
        return parse_integer("1234");
    } else {
        return 0;
    }
}
```

In `example_2()`, even though a compiler could potentially evaluate the call to `parse_integer()` as a constant-expression, it is not required to do so and so could potentially have a runtime call to a function that is potentially-throwing.

Also, if the function does happen to throw during a speculative constant execution of the function then the compiler falls back to inserting a runtime call to the function instead, rather than making the program ill-formed, like it would be if the invocation in `example_1()` or `example_3()` were to throw.

For example, consider what the semantics should be if `example_2()` replaced the argument to `parse_integer` with `"not-a-number"`. In this case, we would expect that calling `example_2` would throw `parse_error` at runtime. Simply changing the value of the string literal passed to `parse_integer` should not change the deduced exception-specification of the function.

In `example_4()`, despite the call to the `parse_integer()` function being evaluated as a manifestly constant expression (inside the first-substatement of `if constexpr`) this context is not being evaluated as a top-level constant evaluation and so might (one day) propagate the exception up to some caller that then handles the exception.

6. Throw expressions

We need to consider both:

- `throw <expr>` expressions that throw new exception objects, and
 - `throw` expressions that rethrow an existing exception.
1. `throw <expr>`

A throw-expression constructs and throws a new exception object of type equal to the decayed type of the operand expression.

Some throw-expressions may require dynamic allocation of storage for the exception object, which might fail due to resource exhaustion. In this case, the throw expression may instead result in throwing an exception of type `std::bad_alloc`.

Note that currently, the behaviour of a program that fails to allocate storage for a newly thrown exception is unspecified. However, the behaviour for failure to allocate/rethrow an `exception_ptr` is well-defined and so I would expect most implementations to do something similar - i.e. throw `std::bad_alloc`.

This was discussed in the mailing list thread: <https://lists.isocpp.org/core/2023/10/15012.php>

This paper proposes defining some forms of throw expressions as not requiring dynamic memory allocation, but instead requiring that implementations allocate the exception objects as automatic storage duration objects. This is necessary to be able to provide the guarantee to programs that particular throw expressions actually throw an exception of that type and do not throw an expression of some other type (like `std::bad_alloc`) and therefore that the set of potentially-thrown exception types is limited to the types of the actual thrown exceptions.

1. Dynamic and Static throw expressions

The function that contained the throw expression that created the exception object is called the "originating function".

If the exception thrown by a throw-expression is either handled within the originating function or if the exception escapes the originating function (possibly after a number of catch/rethrow steps) and the function definition has a throw-specifier with a static exception specification (note this implies the exception type is listed in the throw-specification as otherwise the program would be ill-formed), then the throw expression is considered a *static throw expression*. All other throw-expressions are *dynamic throw expressions*.

In general, this means that all throw expressions in functions with a throw-specifier on the definition that have a static exception specifications will be *static throw expressions* and throw-

expressions in a function with a dynamic exception specification will be dynamic throw expressions unless they are caught locally within the originating function.

For example: Both of the throw-expressions and the rethrow-expressions are all *static throw expressions*.

```
void example1(int i) throw(Y, Z) {
    try {
        switch (i) {
            case 0:
                throw X{}; // static throw-expression: doesn't escape the function
            case 1:
                throw Y{}; // static throw-expression: can escape the function (wit
            case 2:
                throw Z{}; // static throw-expression: escapes the function (not ha
        }
    } catch (X) {
        // handled, not rethrown
    } catch (Y) {
        // handled, rethrown
        throw;
    }
}
```

Another example: this time with some dynamic exception specifications

```
void example2(int i) throw(...) {
    try {
        switch (i) {
            case 0:
                throw X{}; // static throw-expression: doesn't escape the function
            case 1:
                throw Y{}; // dynamic throw-expression: can escape the function (wi
            case 2:
                throw Z{}; // dynamic throw-expression: escapes the function (not h
        }
    } catch (X) {
        // handled, not rethrown
    } catch (Y) {
        // handled, rethrown
        throw;
    }
}
```

A *dynamic throw expression* allocates the storage in an unspecified way controlled by the implementation in the same way that storage for thrown exception objects are currently allocated. The exception object created by a *dynamic throw expression* is called a *dynamic exception object*.

A *static throw expression* allocates the storage for the exception object with automatic storage duration. The exception object created by a *static throw expression* is called a *static exception object*. The scope of the storage ends after the end of the lifetime of the static exception object.

2. Lifetime of exception objects created by static throw expressions

According to [except.throw] p4

The points of potential destruction for the exception object are:

- when an active handler for the exception exits by any means other than rethrowing, immediately after the destruction of the object (if any) declared in the exception-declaration in the handler.
- when an object of type `std::exception_ptr` that refers to the exception object is destroyed, before the destructor of `std::exception_ptr` returns.

Among all points of potential destruction for the exception object, there is an unspecified last one where the exception object is destroyed. All other points *happen before* the last one.

For exception objects created by *static throw expressions*, we need to be able to deterministically compute their latest-possible point of destruction so that we know where to allocate the automatic storage-duration storage for that exception object.

To allow the compiler to compute this point, we need to eliminate the possibility of the exception object's lifetime being dynamically extended by calling `std::current_exception()` and holding on to the `std::exception_ptr`. However, we still want to permit users to call `std::current_exception()` if desired.

If the exception object being handled by the most-recently entered handler is a *static exception object* then a call to `std::current_exception()` returns an `exception_ptr` that references a *dynamic exception object* that is copied from the *static exception object*.

Note that the current semantics of capturing the exception in a `std::exception_ptr` by calling `std::current_exception()` and throwing the captured exception by `std::rethrow_exception()` does not guarantee to throw the same exception object. An implementation is allowed to copy the current exception object into storage referenced by the returned `std::exception_ptr`, so this behaviour is consistent with existing semantics.

Similarly, if a *static exception object* is rethrown by a `throw;` expression that is not a *lexically associated rethrow* of the active handler then the compiler may not be able to deduce that the current exception object escapes the current handler and should not be destroyed when the current handler exits.

A *lexically associated rethrow* is a `throw;` expression that is a sub-statement of a handler's *compound-statement* and that is;

- not inside the handler of a try-block nested within this handler; and
- not inside the body of a lambda expression nested within this handler; and
- not inside the body of a member function of a local class

A `throw;` expression that is not a *lexically associated rethrow* throws a *dynamic exception object*. If the current exception is a *static exception object* then this will allocate a new *dynamic exception object* copied from the current *static exception object*.

With those cases handled, we can now define that the lifetime of a *static exception object* ends deterministically upon exiting a handler for that exception through any means other than a *lexically associated rethrow*.

Storage for a *static exception object* has automatic storage-duration.

Generally, the storage for a *static exception object* will be allocated in the scope of the function that has the outer-most dynamic scope for a handler that can match the exception object's type, taking into account possible chains of catch/rethrow of the exception object.

However, there are some cases where rethrowing an exception object may result in copying a *static exception object* from automatic storage duration storage allocated in an inner dynamic-scope to automatic storage duration allocated by an outer dynamic-scope.

A caller of a function with a static exception specification will generally provide storage to the called function so that the called function can allocate the exception object in storage owned by the caller before it exits. If the caller is not the final handler of an exception (i.e. the lifetime of the exception object may escape the calling function), the caller may choose to reuse the storage provided to it by its caller as the storage it provides to the function it calls.

1. Examples of static exception object lifetime and storage duration

The following examples walk through some simple cases and describe how the exception object lifetime of static exception objects is intended to work.

Given the following exception type, X, and function f(), which throws an instance of X:

```
struct X {
    explicit X(int) noexcept;
    X(const X&) noexcept;
    ~X();
    int value;
};

extern int some_int;

void f() throw(X) { throw X{some_int}; }
```

Example 0: Consider the following case:

```
void g0() throw(X) {
    f();
}

void h0() throw() {
    try { g0(); } catch (const X&) {}
}
```

In this case:

- h0() allocates automatic-storage duration storage for an exception object of type X and provides the address of this storage to g0().
- then g0() calls f() and provides the address provided by h0() to f()
- if f() throws an exception, then it constructs the object directly into the storage provided by h0().
- the exception unwinds through g0() and the handler in h0() is activated
- the exception object is destroyed and the lifetime of the storage for the exception ends when the handler in h0() exits.

Example 1: Consider the following case:

```
void g1() throw(X) {
    try { f(); }
```



```

    catch (const X&) { throw; }
}

void h1() throw() {
    try { g1(); } catch (const X&) {}
}

```

In this case:

- `h1()` allocates automatic-storage duration storage for an exception object of type `X` and provides the address of this storage to `g1()`.
- when `g1()` calls `f()` it forwards the address provided by `h1()` to `f()` and `f()` constructs the exception object directly into the storage provided by `h1()`.
- execution returns from `f()` and the handler in `g1()` is activated.
- this handler then rethrows the exception - as the exception object is already constructed in the final location there is no need to copy the object.
- execution then returns to `h1()` and its handler is activated
- the exception object is destroyed when the handler in `h1()` exits

Note that `g1()` in this case was able to pass the address of the storage for the exception object provided by `h1()` to `f()` because there was no other exception object being thrown by `g1()` whose lifetime overlaps with the lifetime of the exception object thrown by `f()`.

Example 2: Consider the following case:

```

void g2() throw(X) {
    try { f(); }
    catch (const X& x) { throw x; }
}

void h2() throw() {
    try { g2(); } catch (const X&) {}
}

```

In this case:

- The handler in `g2()` throws a new exception object instead of rethrowing the existing one
- This means that the lifetime of the new exception object thrown by `g2()` overlaps the lifetime of the exception object thrown by `f()`, which is alive until the handler is exited.
- This means that `g2()` cannot reuse the storage for the exception object provided by `h2()` to pass to `f()` and it needs to allocate its own automatic storage duration storage to provide to `f()` for any exception object
- When `f()` throws an exception it constructs into the storage in `g2()` and then unwinds and activates the handler in `g2()`.
- The `throw x;` statement in `g2()`'s handler then constructs a new exception object in the storage provided by `h2()`.
- When the handler in `g2()` exits, the exception object thrown by `f()` is destroyed and its storage lifetime ends.
- Control then unwinds to `h2()` and activates its handler.
- When the handler in `h2()` exits, the exception object thrown by `g2()` is destroyed and its storage lifetime ends.

Example 3: Consider the following case:

```

void g3() throw(X) {
    try { f(); }
    catch (const X& x) {
        if (x.value < 0) {
            throw X{0}; // throw a new object
        }
        throw; // rethrow
    }

    void h3() throw() {
        try { g3(); } catch (const X&) {}
    }
}

```

In this case:

- The handler in `g3()` conditionally either rethrows the current exception object or throws a new exception object.
- In the case that the handler throws a new object, we have a similar case to `g2()` where there is overlap in lifetime between the exception object thrown by `f()` (whose lifetime ends at the end of the current handler) and the new exception object created by the `throw X{0};` statement.
- This means that we cannot reuse the storage that `h3()` provided to `g3()` for returning the exception as the storage that `g3()` provides to `f()` for returning its exception, even though there is another code-path in `g3()`'s handler that rethrows the current exception to `h3()`.
- Instead, `g3()` will need to allocate its own local storage to provide to `f()`, and then in both of the throw and rethrow-expressions, construct a new exception object in the storage provided by `h3()` to `g3()`.
- Note that in the rethrow-expression case, since the original exception object is about to be destroyed, we may be able to move-construct the new exception object from the original object instead of copy it as the `throw;` statement will exit the handler. Note: This may not be the case for all rethrow expressions.

Example 4: Consider the following case:

```

void g4() throw(X) {
    try { f(); }
    catch (const X& x) { if (x.value >= 0) throw; }
    throw X{0};
}

void h4() {
    try { g4(); } catch (const X&) {}
}

```

In this case:

- It is similar to the above `g3()` example, except that this time, the new exception object is thrown from outside of the handler.
- This means that the new exception object's lifetime no longer overlaps the lifetime of the exception object thrown by `f()`. If the exception object is rethrown by `g4()`'s handler then the exception object is already constructed in the right location. Otherwise, the handler exits and the exception object thrown by `f()` is destroyed before the new exception object is created by `throw X{0};`.
- It is therefore safe for `g4()` to pass through the storage provided to it for the exception object to the call to `f()`.

3. Rules for throw expressions

The set of potentially-thrown exception types of a throw expression is the union of

- the set of potentially-thrown exception types of the operand expression
- the type of exception thrown. i.e. `std::decay_t<decltype(<expr>)>`
- the set of potentially-thrown exception type of copy-initialization of the exception-object

Additionally, if the throw-expression is a dynamic-throw-expression then the set of potentially-thrown exception types also includes `std::bad_alloc`, which may be thrown if the implementation fails to allocate storage for the exception object.

2. throw (rethrowing the currently handled exception)

If the rethrow expression occurs lexically within the body of a handler and is not nested within a locally defined function body or a lambda expression body defined in the scope of that handler, then we say that the rethrow expression is associated with the inner-most enclosing handler.

Otherwise, we say that the rethrow expression is unassociated with a handler.

For example:

```
void example() {
    throw; // unassociated - not inside handler

    try {
        throw; // unassociated - not inside handler
    } catch (A a) { // #1
        if (!can_handle(a)) {
            throw; // associated with #1
        }

        auto rethrow = [] {
            throw; // unassociated - inside lambda
        };

        struct LocalClass {
            static void Rethrow() {
                throw; // unassociated - inside nested function
            }
        };
    } catch (...) { // #2
        try {
            throw; // associated with #2
        } catch (B b) { // #3
            throw; // associated with #3
        }
    }
}
```

If a rethrow expression is associated with a handler then we potentially have more static information about the set of possible exception types that might be thrown by that expression than a rethrow expression that is unassociated with a handler.

We can compute the set of exception types that might be thrown by such a rethrow exception based on the set of exception-types that may propagate out of the try-block and based on the exception types that may be handled by handlers earlier in the sequence of handlers for that try-block.

1. Unreachable handlers

One of the interesting cases to handle is when we can statically determine whether the handler is unreachable. This can happen in one of two cases:

- When an earlier handler matches an unambiguous base-class of the later handler's exception-declaration type and will therefore preferentially match any exception types that would be handled by the later handler.
- When the set of potentially thrown exception types of the try-block's *compound-statement* is a finite set (i.e. does not contain `std::any_exception`) and there are no types in that finite set that match this handler and that do not match any earlier handler.

In this case, there is the question of what the set of potentially-thrown exception types should be for a rethrow expression that is associated with such an unreachable handler.

As the handler itself is statically determined to be unreachable, the body of the handler's compound-statement does not contribute to the overall set of potentially-thrown exception types of the try-block, so we might say "it doesn't matter".

However, if we consider the ability to query the set of potentially-thrown exception types using `declthrow()` then a program might want to query within the context of the handler, what types might be thrown by a rethrow expression. i.e. `declthrow(throw)...`

The ability to query whether or not a given handler is reachable can be useful in eliminating code that would otherwise be ill-formed, or that we want to avoid instantiating to reduce compile-times.

For example: Code for implementing the P2300 `std::execution::then()` algorithm needs to determine whether invoking the transformation function might throw and only if it does then invoke the receiver with `set_error()`, otherwise it should not form a call to `set_error()`.

For example:

```
template<typename... Args>
void then_receiver::set_value(Args&&... args) noexcept {
    try {
        if constexpr (std::is_void_v<decltype(this->func(std::forward<Args>(a
            this->func(std::forward<Args>(args)...));
            std::execution::set_value(this->receiver);
        } else {
            std::execution::set_value(this->receiver, this->func(std::forward<A
        }
    } catch (...) {
        // Only want to instantiate the call to set_error() if there are actu
        // any errors possible.
        if constexpr (noexcept(this->func(std::forward<Args>(args)...)) {
            std::execution::set_error(this->receiver, std::current_exception())
        }
    }
}
```

Note that here we need to repeat the essential parts of the body of the try-block in a `constexpr` if condition `noexcept` expression to determine whether or not the `catch(...)` block was reachable. And while, for this example, the body only has one such expression, it is relatively easy to conceive of try-block logic that could be much more involved.

If, instead, we were able to query whether or not the handler was reachable by querying the set of exception types that might be thrown by `throw`; within that handler, then we could avoid having to duplicate the body expressions in the `constexpr` if condition.

For example:

```
template<typename... Args>
void then_receiver::set_value(Args&&... args) noexcept {
    try {
        if constexpr (std::is_void_v<decltype(this->func(std::forward<Args>(a
            this->func(std::forward<Args>(args)...));
            std::execution::set_value(this->get_receiver());
        } else {
            std::execution::set_value(this->get_receiver(), this->func(std::for
        }
    } catch (...) {
        // Only want to instantiate the call to set_error() if there are actu
        // any errors possible.
        //
        // declthrow(throw)... will produce the empty pack if the try-block b
        // is not potentially-throwing.
        if constexpr (sizeof...(declthrow(throw)) != 0) {
            std::execution::set_error(this->get_receiver(), std::current_except
        }
    }
}
```

Another design direction that could be taken here is to allow writing the catch-all handler as a template catch that is instantiated with an exception declaration of `std::exception_ptr`. This could build upon the alternative suggested earlier for using `std::exception_ptr` in place of `std::any_exception` for indicating the set of potentially-thrown exception types for a call to a function with a dynamic exception specification.

If the body of the try-block had a set of potentially-thrown exception types that was empty then the template catch handler would not be instantiated.

For example:

```
template<typename... Args>
void then_receiver::set_value(Args&&... args) noexcept {
    try {
        if constexpr (std::is_void_v<decltype(this->func(std::forward<Args>(a
            this->func(std::forward<Args>(args)...));
            std::execution::set_value(this->get_receiver());
        } else {
            std::execution::set_value(this->get_receiver(), this->func(std::for
        }
    } template catch (auto& error) {
        // Catch-block is only instantiated if there are any exceptions that
        // If an exception is thrown by a 'throw(...)' function then 'error'
        std::execution::set_error(this->get_receiver(), std::move(error));
    }
}
```

2. Rethrow expression rules

If the rethrow expression is unassociated with a handler then the set of potentially-thrown exception types for that rethrow expression is equal to the set `{ std::any_exception }`. This is because the expression could potentially be executed within the dynamic scope of any handler and thus could rethrow any caught exception type.

Otherwise, if the rethrow expression is associated with a handler, *H*, then;

Let *E* be the set of potentially-thrown exception types of the *compound-statement* of the try-block associated with *H*.

Let *H-pre* be the set of handlers associated with the same try-block as *H* that precede *H* in the try-block's *handler-seq*.

Let *X* be initially the empty set.

For each type, *e*, in the set *E*

- if *e* is `std::any_exception` then
 - if the *exception-declaration* of *H* is ... then add `std::any_exception` to the set *X*
 - otherwise, let *h* be the type named in *H*'s *exception-declaration*
 - if any handler in *H-pre* matches exceptions of type *h* then, do nothing (this handler is unreachable)
 - otherwise, if *h* is of non-class type or is a final class then add *h* to *X*
 - otherwise add `std::any_exception` to *X* (it might handle an unbounded set of potential exceptions derived from *h*)
- otherwise,
 - if any handler in *H-pre* matches exceptions of type *e*, then do-nothing
 - otherwise, if *H* matches exceptions of type *e*, then add *e* to *X*
 - otherwise, do nothing

The set of potentially-thrown exception types for a rethrow expression associated with *H* is *X*.

Note that this algorithm can produce a set of potentially-thrown exception types that includes a list of concrete exception types as well as `std::any_exception`.

Note that these rules make the result of a `declthrow(throw)` expression context dependent.

Note that each instantiation of a template handler has a separate set, *X*, of potentially-thrown exception types that it handles. Thus a lexically-associated rethrow expression within a template handler will have different sets of potentially-rewritten exception types for different instantiations. The sets of potentially-rewritten exception types for each instantiation of a template handler will be disjoint.

3. Examples

Given the following declarations:

```
struct A {};  
struct B : A {};  
struct C : A {};  
struct D : B, C {};  
struct E final {};  
  
template<typename... Ts>  
void throws() throw(Ts...);
```

Example 1: static exception list, `catch(...)`

```
try {
    throws<A>();
} catch (...) {
    throw; // declthrow -> { A }
}
```

Example 2: static exception list, typed handler, unreachable catch (...)

```
try {
    throws<A>();
} catch (A) {
    throw; // declthrow -> { A }
} catch (...) {
    throw; // declthrow -> { }
}
```

Example 3: multiple exceptions caught by single handler

```
try {
    throws<A, B>();
} catch (A) {
    throw; // declthrow -> { A, B }
} catch (...) {
    throw; // declthrow -> { }
}
```

Example 4: multiple exceptions caught by multiple handlers

```
try {
    throws<A, B>();
} catch (B) {
    throw; // declthrow -> { B }
} catch (A) {
    throw; // declthrow -> { A }
} catch (...) {
    throw; // declthrow -> { }
}
```

Example 5: ambiguous bases

```
try {
    throws<D>();
} catch (A) {
    throw; // declthrow -> { } - empty because A is ambiguous base of D and
} catch (B) {
    throw; // declthrow -> { D } - catch (B) unambiguously handles exceptio
} catch (C) {
    throw; // declthrow -> { } - already handled by catch (B)
} catch (D) {
    throw; // declthrow -> { } - already handled by catch (B)
}
```

Example 6: mixed static/dynamic

```

try {
    throws<A>();
    throws<std::any_exception>();
} catch (A) {
    throw; // declthrow -> { std::any_exception, A }
} catch (...) {
    throw; // declthrow -> { std::any_exception }
}

```

Example 7: dynamic throw with final class

```

try {
    throws<std::any_exception>();
} catch (E) {
    throw; // declthrow -> { E }
} catch (...) {
    throw; // declthrow -> { std::any_exception }
}

```

Example 8: mixed static/dynamic with final class

```

try {
    throws<E>();
    throws<std::any_exception>();
} catch (E) {
    throw; // declthrow -> { E }
} catch (...) {
    throw; // declthrow -> { std::any_exception }
}

```

Example 9: template handler, single type

```

try {
    throws<A>();
} template catch (auto e) { // Instantiated for types { A }
    throw; // declthrow -> { decltype(e) }
}

```

Example 10: template handler, multiple types

```

try {
    throws<A, B, C, D>();
} template catch (auto e) { // Instantiated for following types { D, B, C }
    throw; // declthrow -> { decltype(e) }
}

```

Example 11: template handler + non-template handler

```

try {
    throws<A, B, C, D>();
} catch (B) {
    throw; // declthrow -> { B, D }
} template catch (auto e) { // Instantiated for { C, A }

```



```
    throw; // declthrow -> { decltype(e) } - either { A } or { C }
}
```

Example 12: template handler + non-template handler + dynamic throw

```
try {
    throws<A, B, C, D>();
    throws<std::any_exception>();
} catch (B) {
    throw; // declthrow -> { B, D, std::any_exception }
} template catch (auto e) { // Instantiated for { C, A }
    throw; // declthrow -> { decltype(e) }
} catch (...) {
    throw; // declthrow -> { std::any_exception }
}
```

Note that for example 12 the template handler is instantiated for types A and C and rethrowing within the template handler only rethrows those concrete types, despite the possibility of the `throws<std::any_exception>()` potentially being able to throw exceptions of type derived from either C or A.

Example 13: constrained template handler + non-template handler + dynamic throw

```
try {
    throws<A, B, C, D>();
    throws<std::any_exception>();
} template catch (std::one_of<B, C> auto e) { // instantiated for { B, C
    throw; // declthrow -> { decltype(e) }
} catch (A) {
    throw; // declthrow -> { A, D, std::any_exception }
} catch (...) {
    throw; // declthrow -> { std::any_exception }
}
```

Note that with this example, the compiler tries to instantiate the template handler for each of the static exception types, but substitution fails for types A and D, succeeding only for types B and C.

The handler for A matches A and D and potentially also an unbounded set of other types derived from A.

The final handler matches an unbounded set of types not derived from A.

7. *await-expression*

An expression of the form `co_await /expression/` has a set of potentially-thrown exception types equal to union of the sets of potentially-thrown exception types of each of the sub-expressions that the *await-expression* de-composes into.

Including:

- The operand *expression*
- The call to `p.await_transform(expr)`, if applicable.
- The call to member or non-member operator `co_await`, if applicable.
- The calls to `await_ready()`, `await_suspend()` and `await_resume()`.

- A call to the destructors of any temporary objects created as a result of one of the above sub-expressions

Note that if one of the sub-expressions is an *interrupted-flow expression* then we still consider the set of potentially-thrown exception types of subsequently evaluated sub-expressions even though they may be unreachable. If we want to consider ignoring unreachable sub-expressions then this should be applied more generally across all expression evaluation.

8. *yield-expression*

A `co_yield /assignment-expression/` or `co_yield /braced-init-list/` expression has a set of potentially-thrown exception types equal to the union of the sets of the potentially thrown exception types of the expression `co_await promise.yield_value(expr)`, where `promise` is an lvalue reference that refers to the current coroutine's promise object.

Note that the call to `promise.await_transform()` is not applied as part of a `co_yield` expression, so does not contribute to the set of exception types.

9. `dynamic_cast`

Basically, any `dynamic_cast` expression to reference-type that could fail the runtime check has a set of potentially-thrown exception types equal to the set `{ std::bad_cast }`.

A `dynamic_cast` expression that is a cast to pointer-type or that was a cast to a reference type that would not fail (e.g. because it was a cast to an unambiguous base-class of the operand's class type) has an empty set of potentially-throw exception types.

Note that the wording of `dynamic_cast` currently permits implementations to throw some type that would match a handler of type `std::bad_cast`, which currently permits throwing types derived from `std::bad_cast`. This change would require that the exception thrown was exactly the type `std::bad_cast`.

10. `typeid`

A `typeid(/type-id/)` expression has an empty set of potentially-thrown exception types.

A `typeid(/expression/)` expression has a set of potentially-thrown exception types equal to `{ std::bad_typeid }`. This exception may be thrown if the operand of the `typeid` expression is the result of dereferencing a null pointer of non-cv type.

While this behaviour matches the current specification of `typeid` expressions, supporting the ability to ask for the type of a dereferenced null pointer seems to be of questionable value. The wording for the builtin unary `*` operator in `[expr.unary.op]` says that the behaviour of performing an indirection on a pointer that does not point to a function or object is undefined *except as specified in `[expr.typeid]`*.

So, literally the only thing you can do with an lvalue produced by dereferencing a null pointer is pass it to a `typeid` expression.

It may be worth exploring whether we can either deprecate/remove this behaviour from `typeid` (which would be a breaking change) or whether we can add a new form of `typeid(/expression/)` that has a precondition that the *expression* does result in an glvalue that refers to an object or function. i.e. such that passing a dereferenced null pointer is undefined-behaviour.

This would allow the set of potentially-thrown exception types for a `typeid` expression to be empty.

11. Name expressions

Expressions that simply name an object

These expressions include:

- *id-expression*
- *this*

These expressions have an empty set of potentially thrown exception types.

12. Lambda expressions

The set of potentially-thrown exception types of a lambda expression are the union of the set of potentially thrown exception types of the initializers of the lambda captures.

5.7. Template handlers

Template handlers are the static-exception equivalent of a `catch(...)`.

They allow a generic handler to be written that can be instantiated to handle any number of exception types statically. The compiler instantiates the handler based on the set of potentially-thrown exception types computed for the body of the corresponding `try` block which were not caught by any preceding handlers.

The short-hand syntax allows the user to prefix the `catch` keyword with the `template` keyword and then use a *placeholder-type-specifier* as the *exception-declaration*.

For example:

```
void do_something() throw(A, B, C);

try {
    do_something();
} template catch (auto e) {
    // catch-block instantiated for each unique type in the static-exception specification
    // In this case where 'e' has type A, B or C.
}
```

The `template` keyword is technically redundant in this case as we could deduce that the handler is a template from the use of a *placeholder-type-specifier*. However, the use of the keyword `template` here can also serve as an indication in code that something a bit different is going on.

The requirement for the `template` keyword could potentially be dropped in this case if desired to match consistency with the syntax for function declarations.

The short-hand syntax also allows a trailing *requires-clause* to be added after the *exception-declaration* to allow introducing type-constraints on the instantiation of the template handler.

For example:

```
void do_something() throw(A, B, C);

try {
    do_something();
} template catch (auto e) requires std::derived_from<decltype(e), std::exception> {
    // catch block only instantiated for types that inherit from ~std::exception~.
    LOG("error: %s", ex.what());
} template catch (auto e) {
    // handle types not inherited from std::exception
}
```

The more general syntax for a template catch-block allows the use of a *template-head* before the catch keyword and then refer to a dependent type that allows any template parameters declared in the *template-head* to be deduced from the type of the exception.

For example: Alternative to `template catch (auto e)` using the *template-head* syntax

```
try {
    do_something();
}
template<typename T>
catch (T e) {
    //... do something with 'e'
}
```

For example: Using constraints and deduced template arguments for a class template

```
template<typename Inner>
struct MyException : std::exception {
    const char* what() noexcept override { return "MyException"; }
    Inner inner;
};

try {
    do_something();
}
template<std::derived_from<std::exception> Inner>
requires (!std::derived_from<Inner, std::runtime_exception>)
catch (const MyException<Inner>& err) {
    LOG("Failed with MyException because: {}", err.inner.what());
}
```

Template handlers can also then be used with `if constexpr` to handle a given exception based on static properties of the exception type.

For example:

```
template<typename T, typename Inner>
struct NestedException : public T {
    Inner inner;
};

template<typename T>
constexpr bool is_nested_v = false;
template<typename T, typename Inner>
constexpr bool is_nested_v<NestedException<T, Inner>> = true;

void foo() throw(FooError,
                NestedException<FooError, std::bad_alloc>,
                NestedException<FooError, std::system_error>);

try {
    foo();
} template catch (std::derived_from<FooError> auto err) {
    // generic FooError handling
    if constexpr (is_nested_v<decltype(err)>) {
        std::print("error: Failed with FooError because {}", err.inner);
    } else {
        std::print("error: Failed with FooError");
    }
}
```

```
}  
}
```

5.7.1. Template argument deduction

When the compiler is trying to determine which handler, if any, will match a static exception object thrown from the body of a *try-block*, it proceeds to try each handler in order. If a handler matches the exception type then the compiler stops searching and does not look at any subsequent handlers.

If, during the search for a matching handler, the compiler encounters a template handler, then the compiler performs the following to determine whether the handler is a match:

- Attempts to deduce template arguments of the template handler by treating the handler as if it were a function template with a single argument and then forming a call to that function-template, passing a single argument which is a non-const lvalue reference to the static exception object. However, this is done with the restriction that the only conversions allowed are implicit conversion to an unambiguous base-class.
- If template argument deduction fails then the handler is not a match.
- Otherwise, if template argument deduction succeeds, then any constraints are evaluated using those deduced template arguments.
- If those constraints are not satisfied then the handler is not a match.
- Otherwise, the handler is a match and the body of the handler is instantiated with the deduced template arguments, substituting those template arguments into the body of the handler. The resulting handler instantiation becomes the handler for static exception objects of that type.

If a handler template is never instantiated (i.e. because it did not match any thrown static exception object types) then the *compound-statement* of that handler does not contribute any statements to the body of the enclosing function.

This can potentially affect the return-type of a function with a deduced return-type.

For example:

```
auto example(auto f) throw(X) {  
    try {  
        f();  
    }  
    template catch (auto err) {  
        return err;  
    }  
  
    throw X{};  
}  
  
example([] throw() {}); // deduces return type to be 'void' as return statement never ins  
example([] throw(int) {}); // deduces return type to be 'int' – single return statement  
example([] throw(int, long) {}); // ill-formed: multiple return statements with different
```

5.7.2. Matching dynamic exception objects

The proposed design is such that a template handler does not match a thrown dynamic exception object, so if the body contains any `throw(...)` expressions then the handler sequence will still require a `catch(...)` handler as the last handler to handle those.

However, one design direction being explored is to allow a dynamic-exception-object to match template and non-template handlers as if the exception object had type `std::exception_ptr`. In this case, a template catch (`auto e`) would be instantiated with `decltype(e)` deduced to `std::exception_ptr`.

5.7.3. Ordering of instantiated handlers

For the purposes of determining which handler is a match, instantiations of a given handler template are unordered. If multiple instantiations would potentially match a given handler when written out then which one is chosen is determined by deducing template arguments and choosing that instantiation rather than looking at which one appears earlier in a list of instantiations.

5.8. Virtual Functions

Virtual functions declared on a base-class can potentially have a different (wider) exception specification than an override of that virtual function declared in a derived class.

For example: A base class might declare a virtual function as `noexcept(false)` whereas a derived class might declare its override as `noexcept(true)`.

However, a derived class cannot have an exception-specification that is wider than the base class method. Callers of a base-class virtual method marked `noexcept` can rightly expect that derived classes cannot override this method to start throwing exceptions from the call.

If we want to extend this idea to throw-specifications, then we need to require that an override of a virtual function does not widen the set of potentially-thrown exception types.

If a base class has a virtual function with an exception specification that is `noexcept(false)` or `throw(...)` then an override of this function in a derived class can have any exception-specifier.

Example 1

```
struct base {
    virtual void f() throw(...);
};

struct derived1 : base {
    void f() throw() override; // OK
};
struct derived2 : base {
    void f() throw(A, B, C) override; // OK
};
struct derived3 : base {
    void f() throw(...) override; // OK
};
```

If a base-class has a virtual function with an exception specification that is `noexcept(true)` or `throw()` then an override of this function must also have an exception specification that is either `noexcept(true)` or `throw()`.

Example 2

```
struct base {
    virtual void f() throw();
};

struct derived1 : base {
    void f() throw() override; // OK
};
```

```

struct derived2 : base {
    void f() throw(A, B, C) override; // ERROR: f() has wider exception specification than
};
struct derived3 : base {
    void f() throw(...) override; // ERROR: f() has wider exception specification than base
};

```

If a base-class has a virtual function with a non-empty static exception specification then an override of this function must have a static exception specification that is a subset of the set of exception types listed in the base class exception specification.

Ex

```

struct base {
    virtual void f() throw(A, B);
};

struct derived1 : base {
    void f() throw(A, B) override; // OK
};
struct derived2 : base {
    void f() throw(A) override; // OK
};
struct derived3 : base {
    void f() throw(B) override; // OK
};
struct derived4 : base {
    void f() throw() override; // OK
};
struct derived5 : base {
    void f() throw(A, C) override; // ERROR: Throws C which is not allowed by base::f() th
};
struct derived6 : base {
    void f() throw(...) override; // ERROR: f() has wider exception specification than base
};

```

It is worth noting here that virtual function overrides are only permitted to throw a subset of the set of types listed in the throw-specification of the base class declaration. They are not permitted to throw types not listed in the throw-specification that are derived from the types listed in the base class declaration.

Virtual functions that want to allow overrides to throw types not listed in the base class function's throw-specification should declare the base-class virtual function with the `noexcept(false)` or `throw(...)` specifier to indicate that a dynamic exception type may be thrown.

Alternatively, the base class function declaration can list an exception type that can allow derived types to extend with different values it can hold. e.g. `std::error_code`

5.8.1. ABI of virtual functions with differing exception specifications

While the language semantics described above are a natural extension of the current rules around `noexcept`, they do have some impacts on what implementations need to do to provide these semantics.

A virtual function with a non-empty static exception-specification can have a different calling-convention to a function with a different exception-specification.

This means that in cases where an override chooses to narrow the exception specification that we may not be able to directly use a pointer to the overridden function in the vtable slot used for a call that dispatches statically via the base class interface as the calling conventions may not match.

This is a similar case to how the compiler needs to handle overrides that can have covariant return-types that return a pointer/reference to some type that is derived from the return-type of the base class function.

In many cases the ABI of the override that returns a derived type is the same as the ABI of the base class function.

For example:

```
struct base {
    virtual base* clone();
};
struct derived : base {
    derived* clone() override;
};
```

In this case, the derived class has the base class as its first sub-object and so the address of the derived object returned by `derived::clone()` can be reused as the address of the base object and so the vtable entries for calling `base::clone()` and `derived::clone()` can be the same.

However, if you consider:

```
struct base {
    virtual base* clone();
};
struct other_base {
    virtual std::string to_string();
};
struct derived : other_base, base {
    derived* clone() override;
};
```

Now, the base object is not necessarily the first sub-object of a derived and so the pointer returned by `derived::clone()` may need to be adjusted by an offset when called via the `base::clone()` interface so that the returned pointer refers to the base sub-object.

This means that the vtable entry for `base::clone()` cannot just contain the address of the `derived::clone()` function as it would return an address that was not a pointer to the base sub-object.

Instead, the table entry for `base::clone()` in the derived type's vtable contains the address of a "thunk" - a function stub that forwards the call onto the real implementation and then adjusts the returned address by applying the necessary offset.

When a call is made via the `derived::clone()` interface, it instead dispatches to a separate vtable entry that directly calls the `derived::clone()` function.

The same approach would be required whenever a base-class virtual function is overridden by a derived class that has an exception-specification that differs such that it has a different calling convention and is thus ABI incompatible with the ABI of the base class function's ABI.

These thunks would be the kind of thunk that is generated when you cast a function to a function-pointer with a different exception-specification, described in [Static exception specifications are part of the function type](#).

5.9. Concepts

A `requires` expression allows checking that certain expressions are `noexcept`.

For example:

```
template<typename T, typename U>
concept nothrow_assignable_from =
    requires(T t, U u) {
        { static_cast<T&&>(t) = static_cast<U&&>(u) } noexcept;
    };
```

This concept checking that the assignment expression is valid and that the set of potentially thrown exceptions for the expression is the empty set.

It is equivalent to the following, only without the repetition of expressions:

```
template<typename T, typename U>
concept nothrow_assignable_from =
    requires(T t, U u) {
        static_cast<T&&>(t) = static_cast<U&&>(u);
    } &&
    noexcept(std::declval<T&&>() = static_cast<U&&>(u));
```

If we are to introduce static exception specifications then it seems reasonable to also want to extend the `requires`-expression syntax with the ability to check that the set of potentially-thrown exceptions from an expression satisfies some requirement.

For example, say we wanted to check that a given member-function-call could only throw `std::bad_alloc`, we could allow the user to write something like:

```
template<typename T>
concept foo_factory =
    requires(T& obj) {
        { obj.make_foo() } throw(std::bad_alloc) -> foo_concept;
    };
```

However, from this syntax it's not clear exactly what the semantics of such a check should be. It could be:

- Check that the set of potentially-thrown exceptions is a subset of the listed types. e.g. because it needs to be callable from a function with a `throw(std::bad_alloc)` specifier.
- Check that the set of potentially-thrown exceptions would be caught by a handler whose exception-declaration is `std::bad_alloc`. e.g. because the expression needs to be callable from within a `noexcept` function but is surrounded by a `try/catch` that catches and handles `std::bad_alloc`.

This confusion/ambiguity is similar to the one that was raised during the design of concepts for C++20 - is a `{ expr } -> T;` clause in a `requires`-expression checking that the expression results in exactly type `T`, or is it checking that the expression is convertible to type `T`?

The approach taken to resolve this was to require that you specify a concept that the result of the expression needs to match. This forces the user to be explicit about intent. i.e. either write `std::same_as<T>` or `std::convertible_to<T>`.

Thus this paper proposes taking a similar approach. i.e. to extend the *compound-requirement* as follows:

```
/compound-requirement/:
    { /expression/ } /exception-requirement[opt]/ /return-type-requirement[opt]/ ;
```

```

/exception-requirement/:
    noexcept
    /throw-requirement/

/throw-requirement/:
    throw ( /type-constraint/ )

```

The proposed semantics of providing a *throw-requirement* as part of a *compound-requirement* are as follows.

The set of potentially-thrown exceptions is computed by applying `declthrow()` query to the expression.

The requirement is satisfied if, for each type produced in the expanded pack produced by the `declthrow()` query, the type satisfies the specified *type-constraint*.

i.e. If the *throw-requirement* was `throw(C)`, then the requirement is satisfied if `(C<declthrow(expr)> && ...)` is true.

If we wanted the above example to mean "Any exceptions caught by a handler for type `std::bad_alloc`" then we could write the concept as follows:

```

template<typename T>
concept foo_factory =
    requires(T& obj) {
        { obj.make_foo() } throw(std::derived_from<std::bad_alloc>) -> foo_concept;

```

Or, if you want to check that the only exception thrown (if any) is exactly `std::bad_alloc` then replace the use of `std::derived_from` with `std::same_as`.

To handle the case where you want the expression to be one of a set of types, e.g. because the expression is called from a function that is only allowed to emit those exception types, you can use the `one_of` concept helper.

```

template<typename T, typename... Ts>
concept one_of = (std::same_as<T, Ts> || ...);

template<typename T>
concept foo_factory =
    requires(T& obj) {
        { obj.make_foo() } throw(std::one_of<A, B, C>) -> foo_concept;
    };

void usage(foo_factory auto& factory) throw(A, B, C) {
    auto foo = factory.make_foo();
    // ...
}

```

It is expected this should cover most scenarios that users will want to write.

For other scenarios, expressions involving direct use of `declthrow()` queries can be used to for more complicated type-constraints.

5.10. Coroutines

Coroutines have several areas that require updates to support static exception specifications:

- Extensions to `unhandled_exception()` function of a coroutine's promise-type to allow handling static exception objects.

- Rules for computing the exception specification of a coroutine with a deduced throw specification

The rules for computing the set of potentially thrown exceptions for `co_return` statements and `co_await` and `co_yield` expressions are already described above.

5.10.1. Coroutine promise object handling for static exception objects

The current behaviour of a coroutine with a given *function-body* is specified to be as if the *function-body* were replaced by:

```
{
  /promise-type/ promise /promise-constructor-arguments/ ;
  try {
    co_await promise.initial_suspend();
    /function-body/
  } catch (...) {
    if (!initial-await-resume-called)
      throw;
    promise.unhandled_exception();
  }
  final_suspend:
  co_await promise.final_suspend();
}
```

The only way that the `promise.unhandled_exception()` object can observe the current exception object is by either performing a dynamic rethrow expression, `throw;`, or by calling `std::current_exception()`. This would force any static exception object to be copied to a dynamic exception object, which negates the benefit of static exceptions.

Also, we may want to define a coroutine promise type such that the coroutine body itself cannot throw types other than a specified set of potentially-thrown exception types.

To support these use-cases, we can modify the above translation as follows:

```
{
  /promise-type/ promise /promise-constructor-arguments/ ;
  try {
    co_await promise.initial_suspend();
    /function-body/
  } template catch (auto& ex) {
    // Static exception objects caught here
    if (!initial-await-resume-called)
      throw;

    if constexpr (can_handle_exception</promise-type/, decltype(ex)>) {
      promise.unhandled_exception(ex);
    } else if (can_handle_exception</promise-type/, std::exception_ptr>) {
      promise.unhandled_exception(std::make_exception_ptr(std::move(ex)));
    } else {
      promise.unhandled_exception();
    }
  }
  catch (...) {
    // Dynamic exception objects caught here
    if (!initial-await-resume-called)
      throw;

    constexpr bool is_handler_reachable = sizeof...(declthrow(throw)) > 0;
    if constexpr (is_handler_reachable) {
      if constexpr (can_handle_exception</promise-type/, std::exception_ptr>) {
        promise.unhandled_exception(std::current_exception());
      }
    }
  }
}
```

```

        } else {
            promise.unhandled_exception();
        }
    }
}
final_suspend:
    co_await promise.final_suspend();
}

```

Where the following exposition-only concept is defined:

```

template<typename P, typename E>
concept can_handle_exception =
    requires(P& promise, E& ex) {
        p.unhandled_exception(ex);
    };

```

If a thrown static exception object, `ex`, escapes the body of the coroutine then the exception is caught and one of the following expressions is evaluated, tried in-order;

- Call `promise.unhandled_exception(ex)`, if that expression is valid; otherwise
- Call `promise.unhandled_exception(std::current_exception())`, if that expression is valid; otherwise
- Call `promise.unhandled_exception()` if that expression is valid; otherwise
- The program is ill-formed

If a thrown dynamic exception object escapes the body of the coroutine then the exception is caught and one of the following expressions is evaluated, tried in-order;

- Call `promise.unhandled_exception(std::current_exception())`, if that expression is valid; otherwise
- Call `promise.unhandled_exception()`, if that expression is valid; otherwise
- The program is ill-formed

Some key points to note here:

- The `unhandled_exception()` method can now optionally take a parameter that accepts an lvalue-reference to the static exception object
 - It is a non-const reference to allow the implementation to move/mutate the exception object if desired.
- The `unhandled_exception()` method can also optionally take a parameter of type `std::exception_ptr` to refer to the current exception.
 - This can be potentially more efficient on some platforms than having the `unhandled_exception()` method call `std::current_exception()` since, in the case of a static exception object being thrown, it has a reference to the exception object already and so doesn't necessarily have to access the thread-local values inside `std::current_exception()`.
 - It also allows some freestanding implementations that do not support thread-locals and thus do not support `std::current_exception()` to still allow type-erasing exception objects in a `std::exception_ptr`.
- We only try to form a call to `promise.unhandled_exception()` if the handler is potentially-reachable. i.e. if either the `await_resume()` method on the initial-suspend awaiter object is potentially-throwing or if the *function-body* of the coroutine is potentially-throwing.
 - This differs from C++20 which always requires that a promise type has an `unhandled_exception()` object.
 - This also means that a coroutine promise-type that does not define any `unhandled_exception()` member function can still be used as long as the set of potentially-thrown exceptions from the function-body is the empty set.

Together, these changes will allow a coroutine promise type to store and/or inspect an exception object without having to first dynamically rethrow the exception, or if [P2927](#) is adopted, have to allocate a dynamic exception object and call `eptr.try_cast<T>()` to check if it has a particular type.

For example, say we wanted to write a coroutine type that only permitted exceptions of type A and B to exit the coroutine body, we could define it to have a promise-type that had the following `unhandled_exception()` member-function.

```
struct my_promise_type {  
    // ... rest of promise type omitted for brevity  
  
    void unhandled_exception(one_of<A, B> auto& ex) throw() {  
        constexpr std::size_t index = std::same_as<decltype(ex), A&> ? 2 : 3;  
        result.emplace<index>(std::move(ex));  
    }  
  
    std::variant<std::monostate, value_type, A, B> result;  
};
```

If any other exception than A or B escapes the function body then the compiler will fail to find a suitable overload for a call to `promise.unhandled_exception()` and will be considered ill-formed. This allows the author of a coroutine type to mimic the equivalent throw-specification checking that normal functions with a `throw(A, B)` specification does.

This will allow async coroutines to efficiently propagate static-exception objects through chains of coroutines without needing to invoke any of the dynamic exception machinery of the C++ runtime.

5.10.2. Computing the exception specification of a coroutine with a deduced throw specification

When a coroutine is invoked, the call to the coroutine does not necessarily invoke the entire body of the coroutine before returning. The coroutine may suspend at the initial-suspend point and then return immediately.

Also, once the execution of the coroutine reaches a certain point during the execution of the initial-suspend expression, any exceptions that exit the coroutine body from that point are caught and directed to the promise object's `unhandled_exception()` member-function, as described in the previous section.

This means that the set of potentially-thrown exceptions of the initial call to the coroutine is different from the set of potentially-thrown exceptions of the coroutine function-body that the user writes.

This section defines the steps for computing the deduced throw specification of a coroutine that accurately matches the set of potentially-thrown exception types of the initial invocation of a coroutine.

The steps involved in the initial call to a coroutine are:

- allocating the coroutine frame, calling `promise_type::operator new()` if present and viable, otherwise calls global `operator new`.
- copying/moving parameters of the function to the coroutine-frame
- constructing the promise object, optionally passing lvalues of the parameter copies to the promise constructor
- calling `promise.get_return_object()`
- calling `promise.initial_suspend()`
- calling `operator co_await()` on the returned object, if one is defined
- calling `await_ready()` on the awaiter
- calling `await_suspend()` on the awaiter

- any implicit conversions required to convert the result of `promise.get_return_object()` to the return-type of the coroutine.

If any of these expressions throw an exception then this exception will propagate out of the initial call to the coroutine and so the deduced exception specification should include the union of the sets of potentially-thrown exception types of these expressions.

Any subsequent expressions evaluated as part of the coroutine body, from the initial-suspend-point's `await_resume()` call onwards, will be evaluated inside the try/catch around the coroutine body and thus will be filtered through the `promise.unhandled_exception()` logic described above.

As it's possible that the coroutine may not suspend at the initial suspend-point and may execute the body of the coroutine, and thus can potentially execute any of the reachable handlers that call `promise.unhandled_exception()`. If any of these calls to `promise.unhandled_exception()` exit with an exception then this exception would also propagate from the initial call to the coroutine.

Further, if the coroutine first suspends at a suspend-point that symmetrically transfers execution to another coroutine by returning a `coroutine_handle` from `await_suspend()` then if that other coroutine has a promise-type with a potentially-throwing `unhandled_exception()` method then this could still exit with any exception thrown by that other coroutine's `unhandled_exception()`.

Or that other coroutine could suspend and symmetrically transfer execution to some other coroutine which has a throwing `promise.unhandled_exception()`. This could continue transitively for an arbitrary chain of coroutines.

This possibility for the initial call to a coroutine potentially throwing any exception that could be thrown by a coroutine that is symmetrically transferred to transitively from the initial call makes it difficult in general to deduce an accurate exception specification that is anything other than a dynamic exception specification.

It is currently impossible to tell whether a call to `coroutine_handle::resume()` might throw or not, which makes it difficult to compute an accurate deduced exception specification for any coroutine that suspends and symmetrically transfers to another coroutine.

This in turn means that we need to conservatively assume that every coroutine with a symmetric-transfer has a dynamic exception specification, which also means that we cannot permit such a coroutine to have a static exception specification.

It's possible that we can later relax this by allowing defining coroutines that have a non-throwing resume which in turn only allow symmetric transfer to other coroutines that have a non-throwing resume and which require the coroutine promise-type's `unhandled_exception()` functions to also be non-throwing.

This direction requires more investigation before a design proposal can be made to address this issue, however.

5.11. Type traits

The standard library has a number of traits queries that let you query properties about certain operations and whether or not they are "nothrow".

5.11.1. Nothrow queries

The existing `std::is_nothrow_*` traits are specified in terms of `noexcept` expressions.

As the `noexcept` operator will still "do the right thing" for expressions that have static exception specifications (i.e. will return `false` if there is a non-empty static exception specification) these existing type-traits should not require any changes in specification.

It is not expected that implementations will require any changes to these traits.

5.11.2. `std::is_function` and `std::is_member_function_pointer`

While the specification of these traits need not change, as they are just defined in terms of whether or not a given type is a function type, there may be some changes to standard library implementations which use partial specialization of all of the different forms of functions.

It is noted that `libc++`, `libstdc++` and `MSSTL` all either are defined in terms of compiler intrinsics, which would need to be updated as part of implementing this feature, or are defined in terms of other properties of the type. For example, by computing `is_function_v<T>` by checking whether the result of evaluating `!is_const_v<const T> && !is_reference_v<T>` is true.

5.11.3. Additional traits

It's possible we may want to add some additional traits for querying information about static exception specifications of certain expressions, however this has not yet been explored.

It is expected that any such traits could be added in future as and when a need for them has been established.

5.12. Freestanding

This paper proposes that the following facilities be available in conforming freestanding implementations:

- static throw expressions
- static rethrow expressions
- try/catch
- template catch handlers
- functions with throw specifications

The following facilities would not be required in a freestanding implementation:

- dynamic throw expression
- dynamic rethrow expression
- `std::exception_ptr`
- `std::uncaught_exceptions()`
- `std::current_exception()`

6. Prior Work

There have been a number of prior proposals and prior work that have looked at this area of exception specifications and various improvements to exceptions.

This section tries to summarise some of the history of the prior art.

6.1. Throw specifications and `noexcept`

- [N2855](#) - Rvalue References and Exception Safety (2009)
 - Discusses the problem with move-constructors and providing strong exception-safe guarantee, which motivates some way for the library to check whether an expression/move-ctor can throw.
 - Proposes introducing the `noexcept` specifier
 - Originally proposed to have `noexcept` functions be ill-formed if any exception could potentially escape the function.

- Proposed to use syntax `throw(...)` to mean "this function can throw any exception." This eventually became `noexcept(false)`.
- Proposes making destructors `noexcept` by default.
- Proposes adding a `noexcept` block that allows telling the compiler to assume that no exceptions will propagate out of this block. e.g. where exceptions are a dynamic property that is guarded against by other means.

```
double sqrt(double);

noexcept void f(double& x) {
    if (x > 0) {
        noexcept { x = sqrt(x); } // okay: if sqrt(x) throws, invokes undefined be
    }
}
```

- Proposes that exception specifications are deprecated
 - Lack of static checking has limited usability and confused users
 - Provide few benefits for compilers
 - Not useful in generic code, where functions need to know whether an exception can be thrown or not, but don't know (or care) what kind of exceptions can be thrown.
 - In fact, the `noexcept` specifier, along with the ability to detect whether an operation is `noexcept` via concepts, provides precisely the statically-checked exception specifications that are required in C++ code.
- [N2983](#) Allowing Move Constructors to Throw (2009)
 - Proposes use of `std::move_if_noexcept()` in move-constructors that require strong exception-safety guarantee.
 - Proposes new `noexcept(<expr>)` expression.
 - Proposes a parameterised `noexcept(<bool-expr>)` function specifier.
 - Suggests making destructors `noexcept` by default.
- [N3051](#) - Deprecating exception specifications
 - Talks about shortcoming of original throw-specifications
 - Run-time checking
 - Offers programmer no guarantees all exceptions have been handled.
 - `std::unexpected()` does not lend itself to recovery.
 - Run-time overhead
 - Run-time checking requires compiler to generate extra code, which hampers optimisations.
 - Unusable in generic code
 - Not generally possible to know what types of exceptions may be thrown from operations on template arguments, so precise exception specification cannot be written.
 - Claims that there are only two useful exception-specifications: `throws-something` and `throws-nothing`.
 - Proposes deprecation of `throw()` specifications as `noexcept` covers the two useful cases.
 - Also proposed that `noexcept` was equivalent to `throw()` on a declaration, but differed in semantics when it was placed on the definition.
- [N3103](#) - Security impact of `noexcept`
 - Says that a program that continues after `noexcept` function exits with an exception can lead to undefined/unexpected-behaviour that can be exploited by a malicious user to bypass security restrictions and/or cause denial-of-service attacks.
 - Proposes mandating that the program should terminate if there is ever an unhandled exception that is about to exit a `noexcept` function.
- [N3248](#) - `noexcept` prevents library validation (2011)
 - The "Lakos Rule" paper

- don't put noexcept on functions with narrow contracts
- so we can test assertions/preconditions
- The risk from overly aggressive use of noexcept specifications is that programs with hidden terminate calls are produced
- The risk of under-specifying noexcept specifications is that they become difficult to add in a later revision of the standard, as the noexcept operator becomes an observable part of the ABI.
- Long list of library changes to roll back use of noexcept in standard library.
- [N3279](#) - Conservative use of noexcept in the Library (2011)
 - Summary of N3248 that just describes the guidelines for use of noexcept
- N4133 - Cleanup for exception-specification and throw-expression
 - Revised by N4285
 - Editorial wording cleanup
 - Introduces "exception specification" semantic concept separate from the grammar term exception-specification.
 - Wording changes from "throws an exception" to "exits via an exception"
- N4320 - Make exception specifications be part of the type system (2014)
 - Revised by N4518, N4533, P0012R0, P0012R1
 - Proposes to make noexcept specifier part of the type of a function and function-pointer
- P0003R0 - "Removing Deprecated Dynamic Exception Specifications (2015)
 - Revised by P0003R2, P0003R5 (2016)
 - Proposes to remove the throw specifiers deprecated in C++11.
 - This was adopted and throw specifiers were removed in C++17.

6.2. Java Checked Exceptions

6.2.1. Background On How Exceptions work in Java

In the Java exception model all exception types inherit from the Throwable base class. A throw expression can only throw objects that inherit from Throwable.

There are a number of important base-classes for exceptions defined in

- Throwable - Base-class for all types that can be thrown
 - Error - Base-class for bugs (assertion failures) or more serious problems with the runtime (e.g. StackOverflowException, OutOfMemory). Applications are not expected to handle these errors.
 - Exception - Base-class for conditions that an application may need to handle.
 - RuntimeException - Base-class of conditions that might be raised at any time by the runtime. e.g. NullPointerException, IndexOutOfBoundsException, etc.

Types inheriting from Throwable, directly or indirectly, are classified as either "checked" exceptions, or "unchecked" exceptions.

Types inheriting from either Error or RuntimeException are "unchecked" exceptions. These could potentially be raised by the runtime at any time and may not require an explicit throw expression in user-code.

Types inheriting from Throwable that are not "unchecked" exceptions are classified as "checked" exceptions.

This property of being "checked" or "unchecked" is a property of the type. A checked exception type is always a checked exception type, regardless of context.

Functions in Java have an optional throws clause that is placed after the argument list.

For example:

```
public void foo() throws IOException {  
    // ...  
}
```

If a function can potentially throw a checked exception then it must list that checked exception type, or one of its base-classes, in the throws clause, otherwise the program is ill-formed.

A function is allowed to list unchecked exception types in its throws clause, but is not required to.

6.2.2. Criticisms of Java Checked exceptions

In the interview "The trouble with checked exceptions" [Trouble] between Bruce Eckel and Anders Hejlsberg they talk about checked exceptions in Java and some of the problems they had found in practice with this design.

The interview discussed two main problems:

- versioning and evolvability of interfaces
- scalability of the design at high levels

1. Versioning and evolvability of interfaces

The problem with versioning and evolvability of interfaces with Java checked exceptions arises when an API that is initially released and is in use by users later wants to evolve to add additional error-conditions for a given operation.

Consider a function that initially specifies that it throws A, B or C but then later wants to add some features that means it can fail in new ways and wants to now add D to the list. Adding D to the list of potentially thrown exceptions is now a breaking change, potentially requiring updating each caller to either handle that exception, or if it does not handle that error, then adding that exception to its throw specification.

This in turn can require recursively updating many calling functions with new error-conditions.

To avoid having to go and update callers to add additional exceptions to their throw specifiers programmers often just list `throws Exception` which allows it to transparently throw any type of exception - which basically defeats the purpose of checked exceptions.

This is arguably already the status quo in C++, where a dynamic exception specification already just says "I can throw anything".

This allows C++ functions to later be modified to throw new exceptions without that introducing a compilation error. However, in many cases this can still be a breaking change if existing callers were not expecting this new exception to be thrown - potentially leading to bugs relating to unhandled exceptions.

One of the arguments against checked exceptions in Java was that for a lot of use-cases people don't want to handle the exceptions. If something fails then they just want to let these errors propagate up to some top-level handler and just let it log/report the error. Forcing the programmer to write a lot of boiler-plate to accurately list exception types in cases where callers do not handle specific errors does not add a lot of benefit and so writing `throws Exception` may make sense in that situation.

This paper holds the position that adding a new exception to the list of potentially thrown exceptions is indeed a potentially breaking change to a function, regardless of whether this is represented in the signature of the function or not.

The set of ways in which a function can potentially fail is a key aspect of its interface and should be considered as part of its design.

The design proposed in this paper provides a few additional tools compared to Java to help manage evolution of a function's exception specifications.

It provides the ability to introspect the set of potentially-thrown exceptions for some expression via `declthrow`, allowing computation of derived exception specifications in terms of the exception specifications of other functions. This can help with allowing the signatures of calling functions to automatically adapt when callees are modified to throw additional exception types.

It also provides the ability to deduce the exception specification from the body by using `throw(auto)`. This is largely just a convenience to manually writing out complex `declthrow` expressions to compute the exception specification, but requires that the body of the function is visible to the caller.

Functions that want to have an extensible set of error-conditions that can be emitted can also take the approach of using types that have extensible values, such as `std::error_code`, which can allow new error-categories and error-conditions to be represented in a single value in an extensible way.

This requires calling code to manually check for each error-condition, however, and have a fallback for error-conditions it doesn't yet support. If a new error-condition should require callers to handle it then it may be better to represent this as a new exception type which callers need to handle.

2. Scalability of checked exceptions

The second issue discussed in the [Trouble] interview was the use of checked exceptions in large systems.

The point was made that checked exceptions look good in the small when you are looking at leaf functions with a few exception-types and their callers, but when it comes to composing multiple high-level systems it is easy for those high-level calls to end up with 50+ exception-types being listed in each function, affecting readability of the code.

To avoid such large lists of exceptions, Java code ends up either just listing base-class exceptions like `throws Exception`, or ends up catching and ignoring the exception via a `catch` with an empty handler.

Note that the status quo in C++ is that any function not marked `noexcept` has an exception specification that is the equivalent of Java's `throws Exception`, so if dynamic exceptions are suitable for your use-cases then you can continue to use them.

With this paper, if you want to use `throw`-specifiers and static exceptions all the way up into the high-level systems then it's possible there would indeed be a need to list a large number of exceptions in such high-level functions.

There are additional tools in C++ that could help make this more manageable compared to Java, however.

- Pack aliases, proposed by [P3115R0], could be used to define shorter names for reusable sets of exceptions thrown by each subsystem. Then high-level functions could list the pack-alias names, `throw(foo_errors..., bar_errors..., baz_errors...)` instead of listing every exception that might be thrown by each subsystem.
- Use of `throw(auto)` could be used on functions that compose multiple systems to allow the compiler to compute the exception-specification - with the tradeoff that these functions must have their definitions visible to users. Alternatively, use of `throw` specifications using `declthrow()` queries could be used in the declaration if separate compilation is required.
- Multiple subsystems could throw types like `std::error_code` to represent error conditions so that there is overlap between sets of potentially-thrown exceptions from different subsystems and the errors.

6.3. Midori

Midori was an internal research/incubation project at Microsoft that explored ways of innovating the software stack, including programming languages, compilers, operating-systems, services, applications and overall programming models.

Joe Duffy has written an excellent [blog series](#) about the Midori project, which is generally a good read which I highly recommend. However, there is one article in particular on the [Midori Error Model](#) that is worth discussing here as it contains a lot of good ideas and also critique of error models of other languages, including Java's checked exceptions and C++ exceptions.

The critique of what he calls the "unchecked exceptions" model:

In this model, any function call – and sometimes any statement – can throw an exception, transferring control non-locally somewhere else. Where? Who knows. There are no annotations or type system artifacts to guide your analysis. As a result, it's difficult for anyone to reason about a program's state at the time of the throw, the state changes that occur while that exception is propagated up the call stack – and possibly across threads in a concurrent program – and the resulting state by the time it gets caught or goes unhandled.

TODO: Finish writing this.

6.4. Deducing exception specifications

There have been several papers that have explored the idea of deducing exception specifications of a function. This section tries to summarise them.

- [N3202](#) - To which extent can noexcept be deduced (Stroustrup) (2010)
 - Summarises [N3227](#) "Please reconsider noexcept" (2010)
 - "almost every statement in function templates leak into the noexcept declaration"
 - "a user-maintained noexcept increases the likelihood that the specification is not correct. In turn this implies (a) an increased chance that client code terminates unexpectedly, or (b) that optimization opportunities are lost. (Note tha providing correct warnings is also undecidable.)"
 - client code can still change (fail to compile, different runtime behaviour) if noexcept is added or removed from a library.
 - Questions of consistency of deducing noexcept declarations:

The idea of noexcept is to allow code to be written to take advantage of knowing that code will not throw. The key observation is that if we fail to deem a function noexcept even though it doesn't throw the worst that can happen is that sub-optimal, but still correct, code will be executed. In other words, as long as we don't mistakenly deem a throwing function noexcept, not serious harm is done.

- Walks through some cases where there might be inconsistencies in deducing the noexcepttness in different contexts.
 - This seems to be based on the assumption that it needs to be an implicit deduction rather than explicit opt-in for each function.
- [N3207](#) - noexcept(auto) (2010)
 - Highlights issues with implicit deduction in N3202
 - Easy to accidentally introduce ODR violations
 - Issues with eager function template instantiation to determine function signature/noexceptness (not SFINAE friendly).
 - Proposes noexcept(auto) as in-between compromise between having to duplicate body in noexcept declaration, and fully implicit, which has issues.
 - Doesn't let mutually recursive functions all have their noexcept-specification deduced

```
struct A {
    void f(int i) noexcept(auto)
    { if (i > 1) g(i-1); } // call to g() is ill-formed as it's noexcept specific
    void g(int i) noexcept(auto)
    { if (i > 1) f(i-1); }
};
```

- o Also mentions this example as being ill-formed:

```
template<bool> struct M;
template<> struct M<true> { int large[100]; };
template<> struct M<false> { char small; };
struct B {
    template<bool> void maybe_throw();
    template<> void maybe_throw<true>() noexcept(auto) { throw 0; } // deduced no
    template<> void maybe_throw<false>() noexcept(auto) { } // deduced noexcept
    void f() noexcept(auto) { maybe_throw<(sizeof(B) > 10)>(); };
    M<noexcept(f())> data; // ill-formed because the noexcept-specification for
                        // because definition of f() is not available until e
};
// Definition of f() isn't available until the end of the class-definition, he
```

- o Recursion is also interesting:

```
int f(int i) noexcept(auto)
{
    if (i == 0)
        return i;
    else
        return f(i-1) + i;
}
```

Should in theory be deducible, but ill-formed similar to mutually recursive functions

- Minutes:
 - dislike of `noexcept(auto)` with SFINAE on exception-specifications; having only the latter without the former is ok
 - you can't overload on exception-specifications, why do you want to SFINAE on it?
 - can delay determining exception-specification until the function was selected in overload resolution
- Issue with non-template deduced member inside class template

```
template<class T1, class T2>
struct C {
    C(C&& other) noexcept(auto)
    : first(std::move(other)), second(std::move(other))
    { }
    T1 first;
    T2 second;
};
```

```
C<int, int> x; // will instantiate the body of C::C(&&) right here
```

The implicit instantiation of the body of the move constructor should only be performed if ODR-used. This would make `noexcept(auto)` more like explicitly enumerating the expressions. Ideally, the `noexcept` specification should only be deduced if overload selected. May need to be deduced even if used in unevaluated operand - e.g.

`noexcept(f())`

- Issue with debug builds w/ assertions that throw in move-ctor with deduced `noexcept` meaning that `std::vector` copies instead of throwing and never calls move ctor.

- Paper was struck from core motions in 2010-11 Prague meeting

- I w

- Presentation Notes https://wiki.edg.com/pub/Wg21batavia/Documents/noexcept_auto.pdf Lists two perils

- Adding print statements changes the deduced exception-specification (e.g. using `cout`)
 - Can workaround by adding `try/catch`.
- Adding assertions can change deduced exception-specification.
 - An issue if assertion macro throws.
 - Not an issue if it terminates.

- [N3227](#) - Please reconsider `noexcept` (2010)

- Draft: https://wiki.edg.com/pub/Wg21batavia/EvolutionWorkingGroup/reconsider_noexcept.html
- <https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2010/n3227.html>
- Suggests that we should support deducing `noexcept`-ness of a function
- Talked about avoiding use of flow-analysis to determine `noexcept`ness of a function (undecidable/hard problem - Rice's theorem).
- Concerns about inconsistencies across compiler with flow-analysis leading to some programs being well-formed on some compilers but ill-formed on others based on whether they deduced the `noexcept` the same way.
- Not a problem if you ignore flow-analysis and just look at whether there are any potentially-throwing expressions that do not catch exceptions.
- Has some expected objections and some counter-arguments

- [N3204](#) - Deducing "noexcept" for destructors (2010)

- Short paper
- Wording for default exception-specification user-defined destructors to be the same as for an implicit destructor. i.e. deduced from the exception-specifications of the data-members/base-classes.

- N3386 - return type deduction for normal functions (2012)

- N3638 - Return-type deduction for normal functions (2013)
- maybe relevant to `noexcept(auto)` deduction?
- talks about deduced return types for recursive functions
 - works if there is a prior recursion-breaking return-statement by the time we get to the use of the recursive call.
- Talks about instantiation of function templates even if they are not odr-used e.g. if you use it in a non-evaluated context such as `decltype(f(1))`.
- Proposes adding `decltype(auto)` as well.

- [N4473](#) - `noexcept(auto)`, again (2015) (Voutilainen)

- Tries to open up the discussion about adding `noexcept(auto)` again.
- Not a lot of detail here, other than this is something that is oft-requested, and is a big pain point for some people.

- [Minutes](#)

- Jason M had some proposed wording (not found/attached)
- Had consensus in EWG in Lenexa

- [P0133R0](#) - Putting `noexcept(auto)` on hold, again (2015) (Voutilainen)

- Abandoned `noexcept(auto)` upon realising that you still need to duplicate the expressions for SFINAE cases.
 - Concern should be reduced now that we have concepts as a nicer syntax for SFINAE

6.5. Faster Exceptions

A number of papers have explored the concept of improving the performance of exceptions.

- N4049 - 0-overhead-principle violations in exception handling (2014)
- N4234 - 0-overhead-principle violations in exception handling - part 2 (2014)
- low-latency SG
 - N4456 - Towards improved support for games, graphics, real-time, low latency, embedded systems
 - Mentions wanting guaranteed support for `-fno-exceptions`, `-fno-rtti` No detail on why/motivation/issues.
- P0709 Zero-overhead deterministic exceptions (Herb Sutter)
 - Intro sections have a lot of good motivation for fixing exceptions.
 - Makes the claim that the overheads of dynamic exceptions cannot be avoided by a better implementation strategy.
 - Section 4.1 contains EWG polls indicating that exception-handling is something that they want to improve
 - Marking function with `throws` turns return type into a union of `R + E` with a bool flag to indicate whether it's a result/error.
 - Basically baking `std::expected` into the language
 - Requires a new function-call ABI.
 - `throws(cond)` can be a bool value, or an `except_t` value (`no_except`, `static_except`, `dynamic_except`)
 - Proposes a new `std::error` type
 - Tries to map error values to exception type when propagating out of a function. e.g. `std::errc::ENOMEM <-> std::bad_alloc`
 - For types where there is no obvious standard mapping, it would just wrap an `exception_ptr`
 - This mapping seems like it would be complicated, and difficult to specify/extend.
 - At call sites (that propagate or handle an error), a potential downside of the if-error-goto-handler implementation model is that it injects branches that can interfere with optimizations.
 - Claims that you can still use table-based handling. But not sure what this would look like.
 - Catching `std::error` then requires you to do further conditional branches to determine which of the many possible error-conditions it might be.
 - The callee knew which error they returned with, yet this information has been type-erased in the `std::error` object, and the type information now needs to be extracted again.
 - `try expression / statement`
 - Require every potentially throwing expressions/statement inside a `throws` function to be covered by a `try` expression.
 - Also proposes a `catch(E) { ... }` without an opening `try { ... }` block. Instead, could have `try` expressions scattered throughout code between enclosing open-brace and `catch` clause.
 - This would have issues with the programmer determining what variables are in-scope inside the `catch` block. Every variable whose scope begins after the first `try` expression would potentially not exist and not be available in the `catch` block.
 - Suggests adding a `throws{E}` syntax for specifying a single error type that would be thrown instead of `std::error`.

7. Design Discussion

7.1. Code Evolution

TODO:

- How do we evolve a function to add new exception types?
- How do we evolve a function to remove exception types from throw-specification?
- What about changing from dynamic to static?
- Or from static to dynamic?

7.2. A new calling convention / function-type

TODO: Describe impacts of adding another category of function-types.

- specialization of function-traits
- function-object wrappers
- type-erasing facilities

TODO: Discuss trade-offs of allowing cast from `throw(T1, T2)` function-pointers to `throw(...)` / `noexcept(false)` function-pointers

- ABI / calling-convention impacts
- thunks

7.3. Changing the implementation changes the calling convention

TODO: Expand on these points in more detail

- this was a problem with the paper [P1235R0] "implicit constexpr"

counter points

- this is already a problem for things with deduced return-type
 - mangled names don't include the return type
 - `std::execution` / other expression template libraries can change the return-type of a function if they change the implementation (i.e. the type of the expression-object being returned)
- lots of template code that wants to be `noexcept` transparent already has to maintain this by hand - duplicating the body in the `noexcept` clause
 - if the implementation changes then so does the `noexcept` clause
 - this is not eliminating something
- lots of code is currently specified as "expression equivalent to", which implies having the equivalent `noexceptness`.
 - using `throw(auto)` is a more direct way of expressing that the exception specification is "expression equivalent to" the function body

7.4. How accurate does the potentially-thrown exception type calculation need to be?

This paper currently proposes rules for computing the set of potentially-thrown exception types in a somewhat conservative way and might include exceptions that a deeper analysis would deduce are unreachable.

It considers only control-flow structure of the function-body and function-signatures of called functions. It does not involve or require any reasoning about whether or not conditions are always true or false except for those conditions required to be constant evaluated. i.e. `if constexpr`.

There are a couple of decisions that were made in the design described in this paper which should be considered further:

- whether to consider all identifier labels as reachable or whether to only consider an identifier label reachable if there is a corresponding goto statement that targets the label that is also reachable.
- whether to consider sub-expressions that are sequenced after other interrupted-flow expressions within the same statement as unreachable and thus not contributing to the set of potentially thrown exceptions. For example: What should the deduced exception specification of `f()` and `g()` be?

```
void f() throw(auto) { // deduces to throw(A) or throw(A, B)
    (throw A{}, throw B{}); // throw B{} is unreachable
}

[[noreturn]] int always_throws() throw(A);
void takes_an_int(int x) throw(B);

void g() throw(auto) { // deduces to throw(A) or throw(A, B)
    takes_an_int(never_returns()); // takes_an_int() is never invoked
}
```

7.5. Permission to throw types derived from `std::bad_alloc` / `std::bad_cast`

The definition of global operator `new` and `dynamic_cast` facilities permit implementations to throw exceptions that will be caught by a handler for `std::bad_alloc` or `std::bad_cast`.

This allows implementations to throw types that are derived from these types.

If we want to allow these expressions to have static exception specifications then we would need to constrain these implementations to throw types that were exactly `std::bad_alloc` or `std::bad_cast`. However, doing so is a potential breaking change for any implementations that are throwing some type derived from these types.

And whilst portable C++ code should not be relying on catching a particular exception-type derived from these classes, implementations could potentially define the semantics for their implementations of these facilities to throw `vendor::bad_alloc` or `vendor::bad_cast`.

Having these expressions actually have a static throw-specification would also introduce a dependency of those expressions on the corresponding library-defined types, which could be problematic for some implementations. There is presumably already this dependency, however, since the compiler needs to generate code to throw `std::bad_cast` for `dynamic_cast` expressions.

7.6. Avoiding dependencies on thread-locals

The design proposed by this paper allows eliminating some of the overhead of dynamic exceptions; in particular the dynamic allocation, reference-counting and run-time type-information needed to perform dynamic matching of handlers.

However, there is still some overhead that remains and that is the dependency on thread-local variables needed to support `std::uncaught_exceptions()`, `std::current_exception()` and dynamic-rethrow expressions.

As mentioned in the "Error Handling" section of Ben Craig's paper, [P2268R0](#) "Freestanding Roadmap", there are some freestanding environments that do not support thread-locals. P2268 suggests some potential directions we could take to try to eliminate the dependency on thread-locals for freestanding environments and this paper discusses them further here.

7.6.1. Avoiding overhead implied by `std::current_exception()` and `throw`;

The `std::current_exception()` function and `throw;` expression can be called from any function dynamically within the scope of a handler - they don't need to be called from code lexically within the handler.

However, this means that, because they could be called from an arbitrary number of different contexts, they don't know anything about the type of the exception objects that might be returned or rethrown - so they necessarily need to deal with dynamic exception objects.

These facilities also rely on accessing thread-local state to be able to find the current exception object.

When a thrown static exception object is caught by a handler, the exception object will be living on the stack. However, if we want a call to `std::current_exception()` to be able to return an `exception_ptr` that references a dynamic exception object that is a copy of the stack-allocated object on the stack, then the handler needs to register the address of the static exception object along with some basic type-information and information about how to copy that object to a new location.

This then allows a subsequent call to `std::current_exception()` within the scope of the handler to find the object, allocate new storage for a dynamic-exception-object of the right size, copy the static-exception-object to the dynamic-exception-object and then return an `exception_ptr` referencing that dynamic-exception-object.

Then on exit from the handler, the handler needs to deregister this information from the thread-local.

All of this generated code for registering/unregistering the static-exception-object adds to the cost of handling an exception, both in terms of execution and in terms of binary size.

If the compiler can prove that there will be no calls to `std::current_exception()` or dynamic rethrow it could theoretically optimize out the registration/deregistration logic. But this is difficult to prove as soon as the handler contains any calls to functions that weren't inlined.

However, if we were to annotate the handler with some kind of marker that told the compiler that within the dynamic scope of this handler there will be no calls to `std::current_exception()` or any dynamic-rethrow expressions, then the compiler could avoid needing to register the information about the exception object.

1. Using `catch static` to reduce overhead of dynamic exception facilities

We could consider adding some kind of annotation/specifier to a handler that allows the author to indicate that they will not be making use of the dynamic exception facilities within the handler and that the compiler can optimise on this basis.

For example:

```
try { something(); }
catch static (const X& ex) {
    // user promises not to call std::current_exception() or dynamic-rethrow in here
}
```

Annotating a handler thusly signals to the compiler that, while this handler is the active handler, that:

- evaluating a call to `std::current_exception()` will return an unspecified `std::exception_ptr`. It is unspecified since if the handler has caught a dynamic exception object then there may not be any overhead in registering the exception object if the act of throwing the exception already registered it.
- evaluating a dynamic-rethrow expression either calls `std::terminate()` or is undefined-behaviour (which one is TBD).

Note that the handler can still rethrow the exception by evaluating a `throw;` expression that is lexically associated with the handler - the compiler knows where to go to get the exception object in this case.

2. Catching `std::exception_ptr` as an alternative to calling `std::current_exception()`

Another avenue to explore that might reduce the need for thread-local access is to allow the user to write:

```
try { something(); }
catch /* static */ (std::exception_ptr e) {
    // stash 'e' somewhere
}
```

as an alternative to writing:

```
try { something(); }
catch (...) {
    auto e = std::current_exception();
    // stash 'e' somewhere
}
```

The alternative formulation potentially allows the compiler to directly construct the `exception_ptr` and pass it to the handler without needing to access the thread-local state.

If the thrown exception object was a dynamic-exception-object then it might not be any more efficient and so might lower to calling `std::current_exception()` and just passing the result.

If the thrown exception object was a static-exception-object, however, then the compiler could directly construct a new dynamic-exception-object from the static-exception-object and pass the pointer without having to access the thread-locals. e.g. as if by `std::make_exception_ptr(obj)`.

On the throwing side, we could consider making a `throw expr` expression where argument is a possibly-cv-qualified `std::exception_ptr` as equivalent to calling `std::rethrow_exception()`.

This would have great benefits for generic programming if we could treat `std::exception_ptr` as if it were just another exception object that could be thrown/caught.

The major challenge to a syntax like this, however, is that it would be a breaking change. There may be code-bases out there that are intentionally throwing an exception of type `std::exception_ptr` and catching `std::exception_ptr` rather than using `std::rethrow_exception()` to rethrow the contained dynamic-exception-object.

Adopting the semantics described above would mean their `catch(std::exception_ptr)` handler would now start matching all exceptions, not just a thrown `std::exception_ptr`.

Also, if we decided that, like `catch(...)`, that a `catch(std::exception_ptr)` must appear as the last handler in the sequence of handlers for a try-block, then it may cause existing code to become ill-formed if `catch(std::exception_ptr)` does not appear as the last handler.

A quick search in <https://codesearch.isocpp.org/> showed 2 matches for handlers that were catching a `std::exception_ptr`.

One was a unit-test¹ in `cpprestsdk` that was making sure that when an `exception_ptr` was being rethrown by an async task library that it was using `std::rethrow_exception(eptr)` instead of `throw eptr` and was checking that the `catch(std::exception_ptr)` handler was not being entered when the exception was rethrown.

The other was similarly for an async framework in the Ice project which was intentionally catching an `exception_ptr` and then immediately calling `std::rethrow_exception()` to propagate the exception.

Upon further investigation, I found an issue² that indicated a desire to move away from this and a search of the latest version of the codebase yielded no usage.

A search of github yielded one other hit:

- [heisenberg-go hnsu_wrapper.cc](#) This looks like it might be a bug and may have intended to write `catch(const std::exception&)` as I could not find any throw expressions that were throwing an `exception_ptr` in the project.

Usage of this pattern in open-source projects does not seem to be wide-spread. While not representative of code-bases at large, it may be worth considering a syntax like this to both allow freestanding platforms that don't have access to thread-locals an alternative API for obtaining the current exception an `exception_ptr` and also to simplify generic code that needs to deal both dynamic and generic exceptions by giving them similar forms.

7.6.2. Avoiding overhead implied by `std::unhandled_exceptions()`

The `std::unhandled_exceptions()` function reports a count of the number of in-flight exceptions on the current thread - ones for which a throw expression has been evaluated but for which the corresponding handler has not yet been activated.

Multiple exceptions can be in-flight at once in the case that a destructor calls some function that throws an exception which is then caught within the scope of the destructor.

This is used by facilities such as `scope_success` or `scope_failure` scope-guard objects that conditionally execute some logic if the destructor is called during the normal scope exit path or during exception unwind. These classes capture the number of uncaught exceptions in their constructor and then compare that number to the number of uncaught exceptions in their destructor decide whether to execute the logic based on the result of that comparison.

For example: A `scope_failure` can be used to roll back a transaction if something fails

```
{
    scope_failure cancel_on_fail{[&] { transaction.Rollback(); } };
    for (auto& x : items) {
        transaction.execute(generate_sql(x)); // might throw
    }
}
```

To make this work, however, the C++ runtime needs to store a thread-local count of the number of in-flight exceptions and then increment it whenever executing a throw expression and decrement it whenever a handler is activated.

For example, on the Itanium ABI, there is the following interface relevant to `std::uncaught_exceptions()`

```
struct __cxa_eh_globals {
    __cxa_exception* caughtExceptions;
    unsigned int uncaughtExceptions;
};

// Gets a pointer to the current thread's exception state
__cxa_eh_globals* __cxa_get_globals(void);
```

When an exception is thrown, the compiler needs to ensure that something equivalent to the following:

```
++__cxa_get_globals()->uncaughtExceptions;
```

and then in the handler (immediately after initializing any exception-object parameter)

```
--__cxa_get_globals()->uncaughtExceptions;
```

Usually the increment/decrement is wrapped up inside the `__cxa_throw()` and `__cxa_begin_catch()` functions.

The use of a thread-local here is a somewhat indirect mechanism to detect which context the compiler is calling the destructor and has known deficiencies when used in conjunction with coroutines or fibers which can suspend and then resume in a different context, potentially with a different number of in-flight exceptions.

One direction to explore, which has been previously hinted at in both P0709 and P2268 (and possibly others) is to allow either having two destructors, one to execute during exception unwind and another to execute during normal scope exits, or to pass a flag that indicates the disposition in which the destructor is being called.

For example, the strawman syntax of a destructor that takes a parameter of type `std::unexpect_t`:

```
template<std::invocable F>
struct scope_failure {
    F func;

    // Normal destruction
    ~scope_failure() {}

    // Unwind destruction
    ~scope_failure(std::unexpect_t) {
        func();
    }
};

template<std::invocable F>
scope_failure(F) -> scope_failure<F>;
```

The compiler already knows in what contexts it is calling the destructors - the calls to destructors during unwind is typically separate from the code that calls destructors during normal scope exit, and so it would be straightforward for compilers to call a different overload in exceptional contexts.

While this would allow classes like `scope_success` and `scope_failure` to be implemented without a dependency on thread-locals, it would be a fairly large design change and potentially require a lot of viral changes to standard library types to support.

More exploration is needed to determine the viability of this option.

In the meantime, it's possible that freestanding platforms that do not support thread-locals could provide basic exception support but without providing `std::uncaught_exceptions()`.

7.7. Calling C functions

When importing APIs from C libraries which were not written with consumption by C++ in-mind, C function declarations will generally not have any exception-specification listed on them, thus defaulting the exception-specification to `noexcept(false)`.

When a C++ program calls such functions, the compiler must assume that they could potentially throw an exception and so may end up generating additional code to handle the case that these functions throw an

exception, even if the vast majority of these functions will never emit an exception.

When you call these C functions from a `noexcept` function then the worst you get is some suboptimal code-gen that bloats your binary and maybe misses out on some optimizations.

However, within a function that has a static-exception-specification, calling such functions would require surrounding the call in a `try { } catch (...) { }` to ensure that all exceptions are handled.

For example:

```
extern "C" int some_c_api(void*);

void some_caller() throw(std::error_code) {
    // ...

    void* handle = /* ... */;

    int result = some_c_api(handle); // ERROR: some_c_api() may throw a dynamic exception v
    if (result < 0) {
        throw std::error_code{errno, std::system_category()}
    }

    // ...
}
```

This is helpful that the compiler can identify places where control flow may cause an unhandled exception to be emitted, however in this case may be an example of a false-positive.

The code can be amended either by adding a `try/catch(...)` around the call, with either a `std::unreachable()` or `std::terminate()` call, although this still ends up generating sub-optimal code.

Alternatively, if the developer knows that the function will not throw, the function-pointer can be `reinterpret_cast` to an equivalent function-pointer annotated with `noexcept` and then call through that function-pointer. While non-portable, this will work on most platforms and gives better codegen (without code for handlers).

For example: Given the following helpers

```
template<typename T> struct noexcept_cast_helper;
template<typename R, typename... As>
struct noexcept_cast_helper<R(*) (As...)> {
    using type = R(*) (As...) noexcept;
};
template<typename F>
[[clang::always_inline]] auto noexcept_cast(F func) noexcept {
    return reinterpret_cast<typename noexcept_cast_helper<F::type>(func);
}
```

you can write the following (which is now well-formed):

```
void some_caller() throw(std::error_code) {
    // ...

    void* handle = /* ... */;

    int result = noexcept_cast(some_c_api)(handle); // OK: This is now a call to a noexcept
```

```

    if (result < 0) {
        throw std::error_code{errno, std::system_category()}
    }

    // ...
}

```

Another direction suggested by Ben Craig was to add something similar to the `extern "C"` block to allow declaring that all functions in a given block are `noexcept`.

For example: The following would

```

extern "C" { noexcept {
#include "the_c_api.h"
} }

```

One concern with this approach would be that it could transitively apply the `noexcept` qualifier to all headers included by `"the_c_api.h"`, not necessarily just the APIs directly declared by that header.

Another potential direction we could explore is the use of `try` expressions. For example: A `try/else` expression that evaluates a first sub-expression and if that exits with an exception then evaluates a second sub-expression as a fallback value.

```

int result = try some_c_api(handle) else std::terminate();
int result = try some_c_api(handle) else nothrow_unreachable(); // compiler optimizes this
int result = try some_c_api(handle) else -1;

```

This could help simplify calling C APIs that have not been marked non-throwing but for which the programmer knows it cannot throw.

Note that this paper does not propose either of these directions at this time, only list them as potential directions for further investigation.

7.8. Interaction with Contracts

The contracts proposal [P2900R6] adds the ability to add pre-conditions and post-conditions to function declarations and also allows adding `contract_assert()` expressions.

When a contract that is checked is violated, a violation handler is invoked. As currently specified, implementations can permit the violation handler to throw an exception. This means that `contract_assert()` statements can potentially throw exceptions.

Whether the violation handler is a throwing handler or not is not necessarily known by the compiler at the time that a `contract_assert()` statement is compiled, and the current contracts facility provides no way of detecting whether or not a contract-assertion is potentially throwing.

Further, throwing violation handlers are not restricted to throwing an exception of a particular type, so if treating a `contract_assert()` statement as potentially-throwing then we would need to consider it as potentially-throwing any exception type - i.e. as if it had a dynamic exception specification.

However, one stated design principle of the contracts facility, listed in [P2900R6]:

Concepts do not see Contracts - if the mere presence of a contract assertion, independent of the predicate within that assertion, on a function or in a block of code would change when the satisfiability of a concept then a contained program could be substantially changed by simply using contracts in such a way. Therefore, we remove the ability to do this. As a corollary, the addition or removal of a contract assertion must not change the result of SFINAE, the result of overload resolution, the result of noexcept operator, or which branch is selected by an if constexpr.

However, this principle, in combination with potentially-throwing contract violation handlers, seems to be incompatible with several of the features proposed in this paper.

The design in P2900R6 makes contract_assert a statement so that you cannot ask the question whether it is noexcept or not as a noexcept expression requires its operand to also be an expression. This defers needing to answer the question of whether or not contract_assert is potentially throwing.

However, this question needs to be answered if we wanted to be able to use contracts in combination with the features proposed in this paper - something I think we should support

For example: What is the deduced exception specification of these functions?

```
int example1(int x) throw(auto) {
    contract_assert(x != 0);
    return 42 / x;
}
int example2(int x) throw(auto) pre(x != 0) {
    return 42 / x;
}
```

For example: Are the following functions well-formed?

```
void example3(int x) throw() {
    contract_assert(x != 0); // ill-formed if potentially-throwing
}
void example4(int x) throw() pre(x != 0) {
    // does a function with a precondition have a potentially-throwing function-body?
}
```

For example: Is the call to set_error() below a discarded statement or not?

```
void example5(int x) throw() {
    try {
        contract_assert(x != 0);
    } catch (...) {
        constexpr bool is_reachable = sizeof...(declthrow(throw)) > 0;
        if constexpr (is_reachable) {
            set_error(std::current_exception());
        }
    }
}
```

If we follow the principle that introducing contracts should not change the satisfiability of a concept then, for the purposes of computing the set of potentially-thrown exception types of a contract_assert statement, we would need to treat it as if it were non-throwing. Otherwise, adding a contract_assert might change the signature of a function with a deduced exception specification, or might make a function with a static exception specification ill-formed.

However, this then seems at odds with the desire for people to make use of throwing violation handlers to recover from program bugs, as code surrounding contract-assertions is going to assume that they do not throw - which they don't, for a correct program which does not violate any of its contracts.

I think this conflict helps us get to the core question of what does an exception-specification mean *in the presence of contract checks that might fail*.

If I mark a function as `noexcept`, does that mean that it guarantees that calling it will not ever throw an exception? Or does it mean it will not throw *for an evaluation that does not violate any contract checks*?

I don't have any solutions to propose here at this stage, but I would like to try to find a solution that allows both static exceptions and contracts to work well together.

7.9. Standard Library Impacts

7.9.1. Whether implementations add `noexcept` to "Throws: Nothing" functions can now affect well-formedness of a program

A function in the standard library specified as "Throws: Nothing" but for which there is a pre-condition are often specified as being `noexcept(false)` functions. The main rationale here is to allow for the possibility that an implementation might want to throw an exception if the pre-condition is violated, since violating a pre-condition is undefined-behaviour the implementation is technically free to do anything, including throw an exception.

However, implementations that do not throw exceptions on pre-condition violations and instead choose to terminate/abort are currently free to add `noexcept` to function declarations wherever they feel is appropriate.

This permission is granted to them by [\[res.on.exception.handling\]](#) p5 which says

An implementation may strengthen the exception specification for a non-virtual function by adding a non-throwing exception specification.

For example MS-STL, `libc++` and `libstdc++` all declare `std::vector::operator[]` as `noexcept` despite it having a pre-condition that the index is less-than `size()`. It is worth noting, however, the same standard library implementations do not mark `std::vector<bool>::operator[]` with `noexcept`.

With this paper and the ability to either declare a function as having a deduced throw-specification or to call these functions from a context where the exception-specification is checked, the implementation differences on `noexcept` qualification of these member functions can potentially affect the well-formedness of the program.

For example: The following function would be well-formed on current major standard library implementations which mark `std::vector::operator[]` as `noexcept`, but might be ill-formed on other conforming implementations that do not mark `std::vector::operator[]` as `noexcept`, and thus would not be portable C++ code.

```
void example(const std::vector<int>& v) throw() {
    int sum = 0;
    for (size_t i = 0; i < v.size(); ++i) {
        sum += v[i];
    }
    return sum;
}
```

A portable implementation would either need to declare this function as `noexcept` and allow the possibility of a runtime `std::terminate()` call if the implementation of `example()` happens to contain a bug that violates a pre-condition and the implementation of `vector::operator[]` handles this by throwing an exception.

Or you would need to surround the body in a *function-try-block* that calls either `std::terminate()` or, if you are confident in the correctness of the code, calls `std::unreachable()` in the handler. For example:

```
void example2(const std::vector<int>& v) throw() try {
    int sum = 0;
    for (size_t i = 0; i < v.size(); ++i) {
        sum += v[i];
    }
    return sum;
} catch (...) { std::terminate(); }
```

Of course, this is then just semantically equivalent to declaring the function `noexcept`.

Ideally we'd be able to declare that this handler was unreachable by calling `std::unreachable()` instead, however `std::unreachable()` is not specified as `noexcept` and so might again throw an exception which would violate the `throw()` specifier checking.

The `std::terminate()` function, on the other hand, is specified to be `noexcept`, and so can be called in this case to portably indicate that the function does not exit with an exception.

Alternatively, the user can define a new `nothrow_unreachable()` function as follows and call that instead of `std::terminate()`:

```
[[noreturn]] void nothrow_unreachable() noexcept {
    std::unreachable();
}
```

This would allow the compiler to assume that the handler is unreachable in modes where calling `std::unreachable()` does not throw an exception, and be equivalent to `std::terminate()` in modes where it does throw an exception.

7.9.2. Should implementations be allowed to strengthen "Throws: something" exception specification to a static exception specification?

Similar to the case where a function that is specified as "Throws: Nothing", there are also functions that are specified as "Throws: <some-concrete-list-of-types>".

For example:

- `std::vector::at(size_t n)` is specified to throw `std::out_of_range` if `n >= size()`.
- `std::optional::value()` is specified to throw `std::bad_optional_access` if the optional does not contain a value.
- `std::function::operator()` is specified to throw either `std::bad_function_call` or any exception thrown by the target object.

There are also many other cases where a function is specified to throw whatever some other expression throws.

In the same vein as the "Throws: Nothing" rule, should we also grant standard library implementations permission to declare functions that can throw exceptions to have a narrower exception specification than `noexcept(false)`. i.e. to have a static-exception-specification.

For example, should we allow conforming implementations to declare `std::vector::at()` with a `throw(std::out_of_range)` exception specifier?

This would greatly assist with calling such functions from a function with a checked exception specification, which would otherwise require surrounding such calls in `try/catch(...)`, and would also allow those functions to have deterministic performance for the exceptional code-paths.

For example: A portable implementation would need to write an extra `catch(...)` to handle any exceptions that are not specified

```
std::uint32_t walk_graph(const std::vector<std::optional<std::uint32_t>>& v, std::uint32_t
std::uint32_t current = start_at;
try {
    while (true) {
        current = v.at(current).value();
    }
}
catch (std::out_of_range) {}
catch (std::bad_optional_access) {}
catch (...) { [] noexcept { std::unreachable(); }(); } // This line would be unnecessary
return current;
}
```

Having a static exception specification for functions like `std::vector::at()` and `std::optional::value()` would actually make them more useful as their performance could now be guaranteed to be on the order of normal control flow you could write yourself with extra if-checks, rather than the many thousands of cycles that the current dynamic exception mechanisms would take.

However, user code written assuming such narrow exception declarations were placed on the standard library functions would, again, be non-portable, because other implementations might not put the tighter exception declarations on their implementations.

Note that specifying that these functions must have a static exception specification would likely be an ABI break for many implementations, so we are unlikely to be able to require implementations to do that.

7.9.3. Usage of the standard library in functions with static throw-specifications will be painful without more `noexcept`

Trying to use standard library functions with pre-conditions, and thus a `noexcept(false)` exception specification, or even library functions with a *Throws: some-type* exception specification from within a function with a static exception specification is going to require callers to surround the calls in a `try/catch(...)` to ensure that the exception-specification is not violated.

Thus, if this paper is adopted then there will be some pressure on vendors and on the standard library to provide APIs with narrower exception specifications for these kinds of functions to allow them to be used in contexts with static exception specifications without having to write `catch(...)` in any handlers.

7.9.4. Function-wrapper types

1. `std::function`

This type is parameterised by a function signature of form `R(Args...)`.

This class is undefined for function signatures of the form `R(Args...) noexcept`, largely because doing so would have no effect on the resulting class.

The `std::function::operator()` call can throw the `std::bad_function_call` exception if invoked on a `std::function` instance that is empty and so the `operator()` could not be marked `noexcept` anyway.

The existing `std::function` class could be used as-is to wrap functions and function-objects whose call-operator as a non-empty static-exception specification. Although we would need to extend the function-pointer casting rules to allow casting from `void(*)() throw(X)` to `void(*)() throw(...)`.

We could also consider defining an additional partial-specialization of `std::function` that allowed specifying a function-signature with a static-exception specification and then computed a static-exception specification for `operator()` that added `std::bad_function_call` to that exception-list.

2. `std::copyable_function` and `std::move_only_function`

These types are parameterised on a function-type that includes the exception-specification.

We would need to add additional partial specializations for cases where the function type passed as the first template argument has a non-empty static exception specification to have this apply that exception specification to its `operator()`, and also to require that the wrapped callable has a compatible exception specification.

3. `std::reference_wrapper`

This type could have its `operator()` modified to have a deduced exception specification so that if the referenced object has an `operator()` with a static exception specification then the `reference_wrapper::operator()` also has an equivalent exception specification.

4. Function objects

The following types could be extended to have call operators annotated with `throw(auto)` to have them deduce the exception specification based on the exception specification of the underlying operator implementation.

Arithmetic function objects

- `std::plus`
- `std::minus`
- `std::multiplies`
- `std::divides`
- `std::modulus`
- `std::negate`

Comparison function object

- `std::less`
- `std::greater`
- `std::less_equal`
- `std::greater_equal`
- `std::equal_to`
- `std::not_equal_to`
- `std::compare_three_way`

Logical Operations

- `std::logical_and`
- `std::logical_or`
- `std::logical_not`

Bitwise Operations

- `std::bit_and`
- `std::bit_or`
- `std::bit_xor`
- `std::bit_not`

7.9.5. Algorithms

We could also consider adapting algorithms that take lambdas/function-objects as parameters that the user provides as being transparent to exceptions such that if the user-provided iterators/ranges and function-objects have static exception specifications then the algorithm itself has a static exception specification.

At the moment, algorithms tend to have `noexcept(false)` exception specifications and so would require a `try/catch(...)` around calls to them if used inside a function with a static exception specification.

8. Implementation Strategies

This section describes some potential implementation strategies that could be taken for implementing the semantics described above.

Note that this is just a sketch that talks through some of the potential challenges and techniques that could be used. It has not yet been implemented in a compiler, so treat this section as speculative until implementation experience has been gained.

8.1. Multiple return-paths/return-addresses

One of the intents of the semantics described above for functions with static-exception specifications is to model them effectively as a function that can either return a value object for the success return-path or can return an error object for each of a set of possible exception-types listed in the exception specification.

With a normal function call there is usually a single return-address that the callee must transfer control to when the callee completes.

The return-address is usually passed from the caller to the callee on the stack. Many hardware architectures have dedicated hardware instructions for pushing the address of the next instruction onto the stack before then jumping to the entry-point of the callee. When returning there is also often hardware support for popping this return address from the stack and then jumping to the return address.

If the callee is potentially throwing then there is, logically, a second potential return-address that the callee might transfer execution to when it exits - the exception unwind path. The unwind path is the path that, logically, exits scopes, destroying automatic-storage duration variables, until it reaches a matching handler and then transfers execution to that handler.

To avoid overhead in having to pass multiple return-addresses to a function and to take advantage of the dedicated hardware instructions for passing a single return-address and returning to that return-address upon completion, implementations often use auxiliary data-structures stored within the binary that allow looking up the exceptional return-address for the current function based on the return-address for the normal path.

With functions with static exception specifications we have a function that has 1 normal return path and zero or more possible exceptional return-paths.

If the caller knows what all of the possible exception types that might be thrown are, it can statically compute what the unwind path should be for each potential exception type thrown. e.g. whether it is handled locally within

the function or is propagated through to its caller.

If the caller could pass, somehow, a list of return-addresses to the callee, one for each possible return-path, then the callee can then return directly to the return-address corresponding to either the normal return-path or a particular exception-type return-path.

However, we want to do so without increasing substantially the cost of making a normal call. For example, if we were to have to generate assembly code to push 5 separate return-addresses onto the stack for a function call to a function that could throw 4 different exception types then this would increase the cost of making the function call and the stack-space used by the calling convention.

Also, many CPUs optimise their branch predictor for call/ret instructions to maintain a stack of recent return-addresses pushed onto the stack so that it can predict the target that a ret instruction will jump to, assuming that for every call instruction there is eventually a corresponding ret instruction. Any scheme that doesn't maintain balanced call/ret instructions can result in a misaligned return-address branch predictor cache which can significantly increase the number of subsequent branch mispredictions.

One approach to implementing this with a lower cost for a function call would be to provide a jump-table of possible return-addresses and to instead push a pointer to the jump-table as part of the call instruction.

On x86/x86-64 architectures this can be done relatively cheaply in one of two ways without increasing the amount of stack-space and would only require one additional instruction to be executed on the normal return path compared to a normal function-call.

8.1.1. Inline Jump Tables

The first option would be to immediately follow a `call` instruction with a series of 5-byte `jmp` instructions that jump to the relevant block for handling the corresponding return-path. The first `jmp` instruction would jump to the code for the normal-return path, and subsequent `jmp` instructions for each exception path.

This calling convention is backwards compatible with existing calling conventions. The caller issues a `call` instruction as per normal and the callee when it returns a value just issues a `ret` instruction to return, which transfers to the first `jmp` instruction which then unconditionally jumps to the code that handles this path.

The additional direct `jmp` instruction should be reasonably able to be predicted/pipelined by a CPU so shouldn't add much in the way of additional latency (NOTE: to be confirmed with data).

For example, given the following code:

```
int divide(int num, int den) throw(divide_by_zero, overflow) {
    if (den == 0) throw divide_by_zero{};
    if (den == -1 && num == INT_MIN) throw overflow{};
    return num/den;
}

int x, y;

int example() {
    try {
        int result = divide(x, y);
        return 2 * result;
    } catch (divide_by_zero e) {
        return -1;
    } catch (overflow) {
        return -2;
    }
}
```

The caller could be translated into assembly similar to the following:

```
x:
    .data 4

y:
    .data 4

example:
    mov edi, dword ptr [x]    ; Load 'x' into register for param 1
    mov esi, dword ptr [y]    ; Load 'y' into register for param 2
    call divide                ; Jump to 'divide' and push address of .JUMP_TABLE onto stack
.JUMP_TABLE:                  ; Each entry is a 'JMP rel32' encoded instruction which is 5 bytes
    jmp .SUCCESS              ; entry[0] - success path
    jmp .ERR_0                 ; entry[1] - error path for divide_by_zero
    jmp .ERR_1                 ; entry[2] - error path for overflow
.SUCCESS:
    # result is in eax
    mul eax, 2
    ret
.ERR_0:
    mov eax, -1
    ret
.ERR_1:
    mov eax, -2
    ret
```

Note that for simplicity, this is assuming that the `dividebyzero` and overflow types are empty, trivially destructible types.

The callee function could be translated into assembly similar to the following:

```
divide:
    ;# NOTE: [rsp] points to address of .JUMP_TABLE
    test esi, esi
    je .THROW1
    cmp esi, -1
    jne .LBL1:
    cmp edi, 0x7fffffff
    jne .THROW2
.LBL1:
    mov eax, edi
    cdq
    idiv esi
    ret                ; return to entry[0]
.THROW1:
    ; [rsp] contains address of entry[0] instruction
    ; need to add '5' (size of JMP instruction) to this to get address of entry[1]
    ; do this in-place to avoid using an extra register to compute the address
    add qword ptr [rsp], 5
    ret                ; return to entry[1]
.THROW2:
    ; [rsp] contains address of entry[0] instruction
    ; need to add '10' (size of 2 * JMP instruction) to this to get address of entry[2]
    ; do this in-place to avoid using an extra register to compute the address
    add qword ptr [rsp], 10
    ret                ; return to entry[2]
```

A few things are worth noting here.

- The calling code still just passes arguments in registers, calls the function with a call-instruction, which returns on the normal return-path back to the instruction after the call-instruction.
- The callee still executes a normal 'ret' instruction to return on the normal code-path.
- On the error return-path, the callee adjusts the return address by adding a constant to the location on the stack where the return-address is stored. On x86 this can be done with a single instruction and does not require any additional registers.
- After returning to the appropriate return-address, the caller then needs to jump to the appropriate location to execute the logic for that code-path.
 - For the success case, this is the one extra instruction that needs to be executed compared to traditional table-based exception-throwing calling conventions.
- There is no need for additional exception-unwind-lookup tables or runtime-type-information. All of the data/logic for handling the exceptions is local and inline within the caller and callee.
- There are no additional branches to execute on the caller side here. Once the callee has branched to the necessary code path that executes the throw expression, it returns directly to the code-path in the caller that handles that case.

Of particular note is that overhead of returning via the exception code-path is one additional instruction.

A slight space-optimisation could be to inline the final code-path (in the above example, the code-path corresponding to the 'overflow' exception case) in lieu of having a jump instruction for the final block of code.

For example: Inlining the last case into the jump table

```
example:
    mov edi, dword ptr [x]    ; Load 'x' into register for param 1
    mov esi, dword ptr [y]    ; Load 'y' into register for param 2
    call divide                ; Jump to 'divide' and push address of .JUMP_TABLE onto stack
.JUMP_TABLE:                  ; Each entry is a 'JMP rel32' encoded instruction which is 5 bytes
    jmp .SUCCESS              ; entry[0] - success path
    jmp .ERR_0                 ; entry[1] - error path for divide_by_zero
.ERR_1:                       ; entry[2] - error path for overflow (inlined as last jump table entry)
    mov eax, -2.              ; entry[2] - error path for overflow (inlined as last jump table entry)
    ret
.SUCCESS:
    ; result is in eax
    mul eax, 2
    ret
.ERR_0:
    mov eax, -1
    ret
```

One concern with this approach is that the normal return-path is no longer inline immediately after the call instruction as there is now a number of jmp instructions interspersed between the call and the logic that handles the success case.

This can potentially have a detrimental effect on instruction-cache miss rate, particularly if the number of possible exception types is large. e.g. if there were more than 12 possible exception types listed in the signature, each of which required their own jump table entry, then you're pretty much guaranteed that the call instruction will live in a different cache-line to the normal return-path and so may be more likely to result in an instruction-cache miss. For cases with a small number of exception types, the overhead on the normal return-path is expected to be small, however.

The other (minor) concern is that returning with a ret instruction with a modified return-address is basically always going to mis-predict the return-address on CPUs that use a return-address cache to predict the target of ret addresses.

While a branch mis-predict would still be much faster than table-based, dynamic exception throwing, there may be use-cases for which this is significant. If it becomes an important enough performance problem, future CPU designs could potentially solve the mis-prediction by introducing a dedicated `retn` instruction that encodes the index of the jump table in an immediate value, which would allow the branch predictor to correctly predict the alternative return-paths.

Implementation experience is required to evaluate the actual impact of these issues on the performance of the normal call-path and on the performance of the error path.

8.1.2. "NOP Stuffing" Jump Tables

Another alternative approach of implementing the jump-table is to somehow embed a pointer/offset to an external jump-table in a location relative to the call instruction.

On x86/64 there are some `nop` instruction encodings that have unused bytes within them and this can be used to encode some data in the instruction stream without that instruction having any effect on the state of the CPU registers (other than advancing the instruction pointer).

You can place this instruction immediately following a `call` instruction and encode into the spare bits of the instruction, an offset to the jump table. As this instruction is a no-op, you can just immediately follow the instruction with the logic for handling the normal return-path. In this case, there is much less of a concern with instruction cache misses.

So in the above case, the caller would be replaced with:

```
example:
    mov edi, dword ptr [x]    ; Load 'x' into register for param 1
    mov esi, dword ptr [y]    ; Load 'y' into register for param 2
    call divide               ; Jump to 'divide' and push address of .JUMP_TABLE onto stack
.DATA_NOP
    nop DWORD (.JUMP_TABLE - .DATA_NOP) ; 3-byte prefix + 4-bytes jump table offset
    mul eax, 2
    ret

.JUMP_TABLE:                ; Each entry of table is a pointer to return-address
    data DWORD (.ERR_0 - .DATA_NOP)
    data DWORD (.ERR_1 - .DATA_NOP)

.ERR_0:
    mov eax, -1.             ; entry[1] - error path for divide_by_zero
    ret

.ERR_1:
    mov eax, -2.             ; entry[2] - error path for overflow
    ret
```

Then the callee would be updated to look something like the following:

```
divide:
    ; NOTE: [rsp] points to address of nop encoding offset to .JUMP_TABLE
    test esi, esi
    je .THROW1
    cmp esi, -1
    jne .LBL1:
    cmp edi, 0x7fffffff
    jne .THROW2
.LBL1:
    mov eax, edi
```

```

    cdq
    idiv esi
    ret      ; return to nop instruction (normal return)
.THROW1:
    ; [rsp] contains address of nop instruction (i.e. return address)
    mov rax, QWORD PTR [rsp]      ; load the address of the nop instruction
    mov edx, DWORD PTR [rax+3]    ; load the offset-to-jump-table part of the nop instruction
    mov edx, DWORD PTR [rax+rdx]  ; load the offset in entry[0] of table
    add QWORD PTR [rsp], rdx      ; add offset to return-address
    ret                          ; return to unwind path for 'divide_by_zero'
.THROW2:
    mov rax, QWORD PTR [rsp]      ; load the address of the nop instruction
    mov edx, DWORD PTR [rax+3]    ; load the offset-to-jump-table part of the nop instruction
    mov edx, DWORD PTR [rax+rdx+4] ; load the offset in entry[1] of table
    add QWORD PTR [rsp], rdx      ; add offset to return-address
    ret                          ; return to unwind path for 'overflow'

```

Note that with this approach, the success return path can now place the rest of the code inline after the NOP instruction, and so is more likely to be in-cache.

However, this comes at the cost of additional instructions for each of the exceptional return paths, including one additional memory load to load the jump table entry's offset.

It's unclear whether the "inline jump table" or "nop stuffing jump table" approach would be more efficient or smaller. The "inline jump table" approach is simpler and more efficient, but the "nop stuffing jump table" approach is more compact and may be more efficient in some cases.

Other architectures may have alternative approaches that are possible or may have additional constraints - these will all need to be investigated.

8.2. Multiple return-value-slots

The previous section focused mostly on how to implement multiple return-addresses for a given function in an efficient way. We also need to consider how to return the exception objects themselves.

With the normal return path, there are a couple of common conventions for passing the return value back to the caller. If the return type is void there is nothing to do as the return-path does not need to pass a value back.

If the return-type is trivially copyable or trivially movable and the return-value fits within a register or two then the return-value can sometimes be passed back in designated register(s). e.g. on x86-64 this is often the `eax/rax` register

Otherwise, in order to support guaranteed copy-elision, the return object will need to be constructed into storage provided by the caller. For normal return-values the caller usually provides the address of this storage by passing it as an additional (hidden) parameter to the function, usually in a register, which the callee then uses to construct the return object before returning.

When returning an exception object from a function with a static exception specification we can use a lot of the same conventions for passing the exception value back.

If the exception object is trivially copyable/movable and empty then we just return to the corresponding return-address without consuming a register.

If the exception object is trivially copyable/movable and fits within a register or two, then we can put the exception object in a register before returning to corresponding return-address. The caller can then just read the exception object value from the register(s).

If the exception object needs to be placed in memory then to get the same copy-elision behaviour as for normal return-values the caller needs to provide the address of the storage for that particular exception type.

This presents somewhat of a challenge, because now, instead of passing a single address for the return value, we potentially need to pass N addresses, one for the return object and one for each of the exception object types.

It is also worth noting here that if the called function accepts a return-value-slot parameter then the storage for the exception object must be distinct from the storage for the return-value. This is because an exception might be thrown while constructing the return-value - the exception-object might need to be constructed while the return-value is still partially initialized.

This paper describes two possible approaches to passing the storage address for exception objects. Other approaches are certainly possible, this paper does not seek to provide an exhaustive list of them, but rather to show that efficient implementations exist and also to discuss some of the tradeoffs.

8.3. Passing an exception-value-slot parameter

The first approach described is similar to the approach described in [Renwick].

If any of the potentially-thrown exception types of the function being called are non-trivial and thus unable to be returned in registers, the caller passes the address of storage for any non-trivial exception objects as an additional (hidden) parameter to the function call. This parameter is referred to as the exception-value-slot. This is in addition to any return-value-slot parameter.

The address must point to storage that is suitably sized and aligned to hold *any* of the non-trivial exception types listed in that call's throw specification.

If the caller handles all potentially-thrown exceptions locally, and does not rethrow any of the exceptions to its caller, then the caller can allocate storage for the exception object within its stack-frame (or coroutine-frame) and then pass the address of that storage as the exception-value-slot parameter.

If the caller does not locally handle all of the exceptions and the caller has a dynamic exception specification, then this is handled as-if the function had a `try { /function-body/ } template catch (auto& e) { throw std::move(e); }` wrapped around the function body. This then rethrows any static exception objects that escape the function as dynamic exception objects - initializing the dynamic exception object by move-constructing it from the static exception object.

If the caller itself has a static exception specification and does not locally handle any of the potentially thrown exceptions then the caller can just pass through the address passed to it in its exception-value-slot parameter.

In this situation, as all of the exception-types are propagated through to the caller's caller then this implies that the caller's throw-specification is a super-set of the exception types listed in the callee's throw specification, otherwise the function would be ill-formed. Therefore, the storage provided by the caller's caller should already be guaranteed to be sized and aligned sufficiently to handle any of the thrown exceptions.

This leaves the more interesting situation where the caller handles some of the exceptions and propagates some exceptions to its caller.

For example: Assuming A, B and C are non-trivial exception types

```
void callee() throw(A, B, C);

void caller() throw(C) {
    try {
        callee();
    }
}
```

```

    catch (const A& a) { print(a); }
    catch (const B& b) { print(b); }
}

void parent_caller() throw() {
    try {
        caller();
    }
    catch (const C& c) { print(c); }
}

```

In this example, if the size and alignment of the storage provided by `parent_caller()` to `caller()` for the exception object `C` is also sufficient for storing an object of either type `A` or `B`, then `caller()` can still pass along the that storage to `callee()`, it just needs to make sure that if an `A` or `B` is thrown then the exception object is destroyed before control returns to `parent_caller()`.

For example, it could be lowered to something (roughly) equivalent to the following example. Note that below, functions with multiple return-addresses are returning an `int` to indicate which return-address it should return to. When lowered to assembly, assume this would instead use one of the approaches described above for returning to the corresponding return address.

```

int callee(void* __exception_value_slot);

int caller(void* __exception_value_slot) {
    switch (callee(__exception_value_slot)) {
        case 0: goto normal_return;
        case 1: goto throw_a;
        case 2: goto throw_b;
        case 3: goto throw_c;
    }

    throw_a:
        const auto& a = *reinterpret_cast<A*>(__exception_value_slot);
        print(a);
        a.~A();
        goto normal_return;

    throw_b:
        const auto& b = *reinterpret_cast<B*>(__exception_value_slot);
        print(b);
        b.~B();
        goto normal_return;

    throw_c:
        // Propagate C exception to caller.
        return 1;

    normal_return:
        return 0;
}

void parent_caller() {
    alignof(C) unsigned char __exception_storage[sizeof(C)];

    switch(caller(&__exception_storage[0])) {
        case 0: goto normal_return;
        case 1: goto throw_c;
    }

    throw_c:
        const auto& c = *reinterpret_cast<C*>(&__exception_storage[0]);
        print(c);
}

```

```

    c.~C();
    goto normal_return;

normal_return:
    return;
}

```

However, if, say, the B object was larger than the C object, then the `caller()` function cannot just pass the storage passed to its exception-value-slot onto `callee()` as this storage is only guaranteed to be able to fit a C.

So in this case, the `caller()` function needs to allocate its own local storage, sufficiently sized for storing either an A, B or C, and then pass that to `callee()`.

However, this means then that in the case that `callee()` throws a C, which we want to propagate to `parent_caller()`, then we need to move-construct the C object from `caller()`'s local storage to its exception-value-slot.

The `caller()` function from above would need to change to:

```

int caller(void* __exception_value_slot) {
    alignas(A) alignas(B) alignas(C)
    unsigned char __exception_storage[std::max(sizeof(A), std::max(sizeof(B), sizeof(C)))];

    switch (callee(&__exception_storage)) {
    case 0: goto normal_return;
    case 1: goto throw_a;
    case 2: goto throw_b;
    case 3: goto throw_c;
    }

    throw_a:
        const auto& a = *reinterpret_cast<A*>(__exception_value_slot);
        print(a);
        a.~A();
        goto normal_return;

    throw_b:
        const auto& b = *reinterpret_cast<B*>(__exception_value_slot);
        print(b);
        b.~B();
        goto normal_return;

    throw_c:
        // Move exception object to storage for caller.
        C& __exception_object = *reinterpret_cast<C*>(&__exception_storage[0]);
        ::new(__exception_value_slot) C(std::move(__exception_object));
        __exception_object.~C();
        return 1;

    normal_return:
        return 0;
}

```

This need to allocate local storage and then move a resulting exception object into the exception-value-slot storage is also required in the situation where an exception is caught and then, inside the handler, a new exception object is thrown and this new exception escapes the function.

This is because, at the point of construction of the new exception object (which will need to be constructed into storage passed in exception-value-slot) the existing exception object which has been caught is still alive and so its storage must be distinct from the exception-value slot storage.

Using this approach is expected to be highly efficient for cases where all exceptions are propagated, with the main cost being the need to pass an extra pointer to the function, using an extra register, possibly causing more spills of registers to the stack.

However, there will also be cases where an exception object must be moved multiple times from local storage to caller's storage as the exception propagates up the stack, which has the potential to be a performance issue for large exception objects.

8.4. Walking stack to find storage

If we want to avoid needing to move/copy exception objects as they propagate up the stack then we need to be able to construct the object in its final location. However, different exception object types are potentially going to be caught in different callers up the stack.

For example: If `foo()` throws `C`, the final storage is in `bar()`, if it throws `B`, the final storage is in `baz()`, and if it throws `A`, the final storage is in `main()`.

```
void foo() throw(A, B, C);

void bar() throw(A, B) {
    try { foo(); } catch(const C&) {}
}

void baz() throw(A) {
    try { bar(); } catch(const B&) {}
}

int main() throw() {
    try { baz(); } catch(const A&) {}
}
```

Since each of these locations is likely going to have different addresses, this means we effectively need to pass `N` exception-object addresses to a call that can potentially throw `N` non-trivial exception types.

The naive approach of adding `N` additional parameters to the function may work for a handful of exception-types, but the overhead of making a function call increases linearly with the number of addresses that need to be passed as each call needs to either push onto the stack or load into registers `N` different pointers.

Instead, we ideally want a solution that has minimal to no overhead on the success path and runtime overhead on the error path that does not increase with the number of thrown exception types.

To do this, we can leverage either of the jump-table approaches described above and add an additional entry to the jump-table for each non-trivial exception type.

The callee can then call this jump-table entry, passing the address of the parent stack-frame, to compute the address of the exception object.

If the immediate caller handles the exception locally then this returns an address that is some offset into the stack-frame address. Otherwise, it loads the address of its caller's stack-frame and then tail-calls the corresponding jump-table entry for the exception-type from its return-address.

For example, given the C++ code above, we might expect the `bar()` function to be compiled to something like the following:

```

bar:
    push rbp
    mov rbp, rsp
    sub rsp, 8    ; allocate storage for C exception

    ; Call foo() with inline jump-table convention
    call foo
    jmp .SUCCESS
    jmp .THROW_A
    jmp .GET_A
    jmp .THROW_B
    jmp .GET_B
    jmp .THROW_C
    jmp .GET_C

.SUCCESS:
.RETURN:
    add rsp, 8
    pop rbp
    ret

.THROW_A:
    add QWORD PTR [rbp+8], 5 ; adjust return-address to entry[1]
    jmp .RETURN

.THROW_B:
    add QWORD PTR [rbp+8], 15 ; adjust return-address to entry[3]
    jmp .RETURN

.THROW_C:
    ; C handled locally
    ; Address of exception object in rax
    mov rdi, rax
    call C::~~C()
    jmp .SUCCESS

.GET_A:
    ; Query for address of A exception will call here.
    ; Forward query to caller
    ; Current stack frame in rdi
    mov rax, QWORD PTR [rdi+8] ; load return-address
    mov rdi, QWORD PTR [rdi]   ; load parent frame-ptr
    add rax, 10                ; compute address of entry[2]
    jmp rax                   ; tail-call to entry[2]

.GET_B:
    ; Query for address of B exception will call here.
    ; Forward query to caller
    mov rax, QWORD PTR [rdi+8] ; load return-address
    mov rdi, QWORD PTR [rdi]   ; load parent frame-ptr
    add rax, 20                ; compute address of entry[4]
    jmp rax                   ; tail-call to entry[4]

.GET_C:
    ; Query for address of C exception, handle locally
    ; Current stack frame in rdi
    lea rax, [rdi-8] ; compute address of storage for C on stack
    ret              ; return to function that is about to throw

```

In the above code, if the call to `foo()` completes normally then execution returns to the `jmp .SUCCESS` and this goes on to return from the function. The overhead here should be negligible - the direct `jmp` instruction is the only additional instruction to be executed on the success path.

Otherwise, if the call to `foo()` throws an exception then first there will be a call to one of the "GET" jump targets to obtain the address to construct the exception object in. Then the `foo()` function will construct the exception object at that address, perform any local unwinding and then return to the corresponding "THROW" jump target.

In the above example, the `GETA` and `GETB` targets forward through to the corresponding queries on `bar()`'s caller's jump table, while `GETC` is able to compute the value locally. In the case of `GETA`, `bar()`'s implementation would then forward on to its caller - `main()` in the example above.

This is effectively doing a stack-walk to find the caller that is willing to provide storage. Only instead of having a generic stack-walking algorithm in a loop it is performing a series of tail-jumps to code in each of the callers corresponding to that exception type.

The walk is terminated by a GET entry that knows how to compute the address of the object within its stack-frame.

For example: In `foo()`, code that throws an A exception might look like the following:

```
foo:
; ... snip

;;;
; throw A{}
;;;

; first get the address for the exception object
mov rax, QWORD PTR [rbp+8] ; load return-address / jump-table-address
mov rdi, QWORD PTR [rbp]   ; load parent frame address
add rax, 10                ; adjust address to entry[2]
call rax                   ; call the address query
; when this returns, the address to use will be in rax
; construct the A() object
mov rdi, rax
call A::A()
; ... any local unwind goes here
; now return to entry[1]
add QWORD PTR [rbp+8], 5 ; adjust return-address
add rsp, 24
pop rbp
ret

; ... snip
```

Walking the stack like this to obtain the storage location allows constructing the object in its final place and minimizes the number of moves of the exception object as it propagates, even if it is caught and rethrown.

Once the exception object is constructed, then any local objects in scope are destroyed and then the function returns to the corresponding entry in the jump table.

Each entry returned to during unwind that is not a handler will just execute a sequence of calls to destructors before then returning to the corresponding for that type in its parent's jump-table. This continues until control returns to some entry that enters a handler for the exception.

Some benefits of this approach:

- No need for external exception tables to be consulted during exception unwinding.
- No need for run-time type information.
- Less copying/moving of exception objects as it propagates up the stack.

There are a couple of potential down-sides to this approach, which will need to be quantified once an implementation is available.

- The stack walk can potentially incur a number of cache misses during the walk, depending on how many frames need to be walked.
- The stack walk may also incur some branch mis-predictions as it jumps to different entries up the call-stack.
- There is more code-generation required compared to the exception-value-slot approach. The size of code grows with the number of exception types that might be thrown from the call.
- Worst case overhead of stack walking is where the immediate caller typically handles but rarely conditionally rethrows the exception which then propagates up to a handler many levels up the call-stack. The stack walk needs to walk all of these frames to find storage, but rarely benefits from placing the storage at this final handler.

Whether this approach ends up being more or less efficient than the exception-value-slot may depend on the context and types involved.

8.5. Unwinding through scopes with destructors

When unwinding due to an exception thrown from a function with a static exception specification, execution will return to an address that handles that particular exception being thrown.

The function will then need to call destructors for any in-scope variables, before then either entering a handler, or propagating the exception to its caller by returning to its

In the case that there are multiple exception types that propagate through a function, all of the code-paths for each exception type will need to call the same set of destructors. Also, there will be overlap in the set of destructors to call depending on which scope the exception was thrown at.

For example: Given the following function, `f()`.

```
struct X { X() throw(); ~X(); int data; };
struct Y { Y() throw(); ~Y(); int data; };
void f(X&) throw(A, B);
void g(Y&) throw(A, B);

void caller(bool cond) throw(A, B) {
    X x;
    f(x);

    Y y;
    g(y);
}
```

If the call to `f()` throws an A then it needs to call `X::~~X()` and then return to entry[1]. If the call to `f()` throw a B then it need to call `X::~~X()` and then return to entry[3]. If the call to `g()` throws an A then it needs to call `Y::~~Y()` and `X::~~X()` and then return to entry[1]. If the call to `g()` throws a B then it needs to call `Y::~~Y()` and `X::~~X()` and then return to entry[3].

There is a lot of overlap in these unwind paths which can be merged to avoid bloating the generated code for these unwind paths with duplicated instructions.

For example: the above function could be compiled to the following assembly:

```
caller:
    push rbp
    mov rbp, rsp
```

```

    sub rsp, 8    ; reserve storage for X and Y
    lea rdi, [rsp+4]
    call X::X()
    lea rdi, [rsp+4]
    call f(X&)
    jmp .F_SUCCESS
    jmp .F_THROW_A
    jmp .GET_A
    jmp .F_THROW_B
    jmp .GET_B

.F_SUCCESS:
    mov rdi, rsp
    call Y::Y()
    mov rdi, rsp
    call g(Y&)
    jmp .G_SUCCESS
    jmp .G_THROW_A
    jmp .GET_A
    jmp .G_THROW_B
    jmp .GET_B

.G_SUCCESS:
    mov rdi, rsp
    call Y::~~Y()
    lea rdi, [rsp+4]
    call X::~~X()

.RETURN:
    add rsp, 8
    pop rbp
    ret

.F_THROW_A:
    add QWORD PTR [rbp+8], 5    ; adjust return-address to entry[1] (type A)
    jmp .UNWIND1

.F_THROW_B:
    add QWORD PTR [rbp+8], 15   ; adjust return-address to entry[3] (type B)

.UNWIND1:
    push rax                    ; save address of exception-object
    jmp .LBL1

.G_THROW_A:
    add QWORD PTR [rbp+8], 5    ; adjust return-address to entry[1] (type A)
    jmp .UNWIND2

.G_THROW_B:
    add QWORD PTR [rbp+8], 15   ; adjust return-address to entry[3] (type B)

.UNWIND2:
    push rax                    ; save address of exception-object
    lea rdi, [rsp+8]            ; address of 'y'
    call Y::~~Y()

.LBL1:
    lea rdi, [rsp+12]           ; address of 'x'
    call X::~~X()
    pop rax                     ; restore address of exception-object
    jmp .RETURN

.GET_A:
    mov rax, QWORD PTR [rdi+8]  ; load return address
    mov rdi, QWORD PTR [rdi]    ; load parent frame ptr
    add rax, 10                  ; adjust to entry[2]
    jmp rax                      ; tail-call

.GET_B:
    mov rax, QWORD PTR [rdi+8]  ; load return address
    mov rdi, QWORD PTR [rdi]    ; load parent frame ptr
    add rax, 20                  ; adjust to entry[4]
    jmp rax                      ; tail-call

```

Notice here how the calls to the destructors during unwind can be rolled together so that we only need code for calling the destructor of each local object on unwind once, despite there being 4 different cases that need to be handled.

It can do this by first adjusting the return-address to the one corresponding to the exception type immediately after jumping to the jump-table target and then after this the code for calling destructors and returning to the caller is the same for all exception types.

8.6. Interop between static-exception functions and dynamic-exception functions

There are several cases that need to be considered with regards to interaction between functions with a checked static-exception specification and functions with dynamic-exception specifications.

When a function with a checked static exception specification calls a function with a dynamic-exception specification, the calling must surround the call in a `try/catch(...)`, otherwise the set of potentially-thrown exceptions computed for the function-body would make the function ill-formed.

The function therefore needs to catch and explicitly either rethrow an exception whose type is marked `final` or needs to throw a new exception object. Both cases would need to initialize a new static exception object in order to propagate the exception to its caller.

For example:

```
// Can potentially throw anything, but docs say it will
// only throw something inherited from std::bad_alloc
void callee();

void caller() throw(std::bad_alloc) {
    try {
        callee();
    } catch (const std::bad_alloc& e) {
        // Rethrow a copy of the exception as a new bad_alloc exception
        throw e;
    } catch (...) {
        // throwing anything else would be a violation of its documented contract
        // contract_assert(false);
        std::terminate(); // or nothrow_unreachable()
    }
}
```

If the existing dynamic exception facility uses table-based unwind, then the calling function still needs to have unwind table entries for catching the exception.

When the situation is reversed, and instead a function with a dynamic exception specification is calling a function with a static exception specification then if the exception is not handled locally, it must implicitly propagate out of the calling function.

This means that at the boundary of this function, the exception needs to be translated from a static exception object to a dynamic exception object and then rethrown. Thus the calling function will need to insert code to allocate a new dynamic exception and then move-construct the static exception into this allocated storage.

For example, given the following:

```
void callee() throw(std::bad_alloc, std::system_error);

void caller() {
```

```
callee();  
}
```

the compiler may lower `caller()` it to something equivalent to the following:

```
void caller() try {  
    callee();  
} template catch (auto& e) {  
    throw std::move(e); // rethrow as a new dynamic exception at boundary  
}
```

8.7. Virtual function calls

Virtual functions generally act the same as normal functions do with regards to static exceptions, with the exception of the case where an override declares a narrower exception specification than the base class.

The following cases need to be handled specially:

- the base class function is declared with a dynamic exception specification and the derived class is declared with a non-empty static exception specification.
- the base class function is declared with a non-empty static exception-specification and the derived class is declared with a static exception specification with a subset of the exceptions listed in the base class' function declaration.

In both of these cases, the calling convention for calling the override via the base class interface is different from the calling convention for calling the override via the derived class interface.

Therefore, the use of a thunk is required to adapt between the calling conventions.

In the case that the base class signature has a dynamic exception specification and the derived class has a static exception specification, the thunk will need to allocate storage on the stack for the static exception, and call the override with the appropriate jump table for handling each exception type and then if the call completes with an exception then rethrow the static exception object as a dynamic exception.

8.8. Interaction with `std::uncaught_exceptions()`

During exception unwind due to a thrown static exception object, we will still want the `std::uncaught_exceptions()` function to return a value that indicates a new uncaught exception is in-flight.

This means that, after constructing the exception object, but before starting the unwind process, the function needs to ensure that the thread-local count of uncaught exceptions is incremented. Then upon entering a handler, after initializing the object in the exception-declaration of the handler, it needs to decrement the count of uncaught exceptions.

For example, on platforms that use the Itanium C++ ABI, the thread-local exception state is obtained through the following API:

```
struct __cxa_exception { /* fields that describe an exception object */ };  
  
struct __cxa_eh_globals {  
    __cxa_exception* caughtExceptions; // linked list of caught exceptions – most recent f:  
    unsigned int uncaughtExceptions; // count of thrown but not yet caught exceptions  
};
```

```
__cxa_eh_globals* __cxa_get_globals(void); // get pointer to thread-local exception state
__cxa_eh_globals* __cxa_get_globals_fast(void); // get pointer to thread-local exception state
```

To modify the uncaught exceptions, the program needs to call `__cxa_get_globals()` and then increment/decrement the `uncaughtExceptions` member of the object pointed to by the return value at the appropriate point during exception throwing/handling.

If the `__cxa_eh_globals()` function is annotated with appropriate attributes (e.g. `__attribute__((const))`) which let the compiler assume that it always returns the same value and has no side effects, then the compiler can call `__cxa_eh_globals()` at the unwind site and handler at the same time and can see that there are no opaque calls or references to the `uncaughtExceptions` member during unwind, then it can even optimize out the thread-local access and increment/decrement.

It should be relative straight-forward to ensure that an implementation of static exceptions gives the specified behaviour of `std::uncaught_exceptions()`.

8.9. Static exceptions and the Itanium C++ ABI

See <https://godbolt.org/z/8578rKh6j> for an proof-of-concept of how a local throw/catch within a function, which under this proposal would be required to be a static exception object, would interact with the Itanium C++ ABI and facilities such as `std::uncaught_exceptions()`, `std::current_exception()` and `throw;`.

Support for static exception objects is likely to require some changes to the implementations of the `__cxa_*` functions that form the ABI, and thus would require relinking with the ABI libraries (either statically or dynamically) but should otherwise be able to link against code previously compiled against the existing ABI.

Additional work is required here to validate this approach.

9. Acknowledgements

Special thanks to Ben Craig and Corentin Jabot for providing feedback on drafts of this paper.

10. References

- [FastCasting] "Fast dynamic casting" (Gibbs, Stroustrup)
https://www.stroustrup.com/fast_dynamic_casting.pdf
- [N3227] "Please reconsider noexcept" (Ottosen, 2010)
<https://wg21.link/N3227>
- [N3202] "To which extent can noexcept be deduced?" (Stroustrup, 2010)
<https://wg21.link/N3202>
- [N3207] "noexcept(auto)" (Merrill, 2010)
<https://wg21.link/N3207>
- [N4473] "noexcept(auto), again" (Voutilainen, 2015)
<https://wg21.link/N4473>
- [P0133R0] "Putting noexcept(auto) on hold, again" (Voutilainen, 2015)
<https://wg21.link/P0133R0>
- [P0709R4] "Zero-overhead deterministic exceptions: Throwing values" (Sutter)
<https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2019/p0709r4.pdf>
- [P1947R0] "C++ exceptions and alternatives" (Stroustrup)
<https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2019/p1947r0.pdf>

- [P2268R0] "Freestanding Roadmap" (Craig) <https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2020/p2268r0.html>
- [P2300] "std::execution" (Multiple authors) <https://isocpp.org/files/papers/P2300R8.html>
- [P2900R6] "Contracts for C++" (Multiple authors) <https://isocpp.org/files/papers/P2900R6.pdf>
- [P2830R1] "Standardized Type Ordering" (Nichols, Ažman) <https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2023/p2830r1.pdf>
- [P2809R2] "Trivial infinite loops are not Undefined Behavior" (Bastien) <https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2023/p2809r2.html>
- [P3068R0] "Allowing exception-throwing in constant evaluation" (Dusíková) <https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2024/p3068r0.pdf>
- [Renwick] "Low-cost deterministic C++ exceptions for embedded systems" (Renwick, Spink, Franke) <https://dl.acm.org/doi/10.1145/3302516.3307346>

Footnotes:

¹

https://github.com/microsoft/cpprestsdk/blob/master/Release/tests/functional/pplx/pplx_test/pplx_op_test.cpp#L198

² <https://github.com/zeroc-ice/ice/issues/582>

Date: 2024-03-16

Author: Lewis Baker

Created: 2024-03-17 Sun 00:22

[Validate](#)